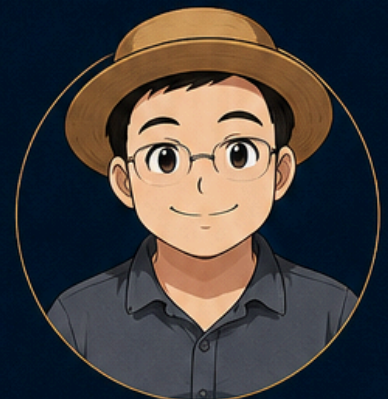


AI HARNESS ENGINEERING

ออกแบบระบบรอบ LLM ให้ AI Agent
ทำงานได้จริง เชื่อถือได้ และตรวจสอบได้



VERSION 2 · LECTURE-GROUNDED EDITION

AI Harness Engineering

ออกแบบระบบรอบ LLM ให้ AI Agent ทำงานได้จริง
เชื่อถือได้ และตรวจสอบได้

Aekanun Thongtae

ฉบับที่ 2 · สิงหาคม 2026

ปรับความลึกจากคำบรรยายจริง โค้ดที่ใช้สอน และกรอบวิศวกรรมสำหรับระบบ Production

แนวทางของหนังสือ

AI Harness Engineering คือการออกแบบระบบรอบ LLM ซึ่งจัดเตรียม Context, Tools, State, Memory, Control, Evidence และ Evaluation เพื่อเปลี่ยนการตัดสินใจที่มีความไม่แน่นอนของโมเดล ให้กลายเป็นการทำงานที่มีขอบเขต สังเกตได้ ตรวจสอบได้ และหยุดได้อย่างปลอดภัย

ที่มาของเนื้อหา

Lecture

แนวคิด อุปมา และข้อสังเกตจาก
บันทึกการสอนของผู้เขียน

Real code

พฤติกรรมที่อ่านและตรวจได้จาก
Source Code ที่ใช้ประกอบการ
สอน

Engineering synthesis

กรอบต่อยอดที่ติดป้ายกำกับและมี
แหล่งอ้างอิงชัดเจน

หมายเหตุฉบับเผยแพร่

ชื่อหน่วยงาน ลูกค้า บุคคล และข้อมูลตัวอย่างที่ระบุตัวตนถูกตัดออก เนื้อหาที่ใช้แหล่งข้อมูลมีเลขอ้างอิงในวงเล็บ
เหลี่ยมและรายการเอกสารอ้างอิงท้ายบท คำบรรยายของผู้เขียนระบุเป็น *Author's Lecture Transcript* ในนาม
Aekanun Thongtae

สารบัญ

ภาค 1 — จาก Model สู่ Harness

- 01 โมเดลยังไม่ใช่ Agent
- 02 LLM สร้างคำตอบอย่างไร
- 03 Agent Loop: จากคำตอบสู่การลงมือทำ

ภาค 2 — Harness สำหรับการลงมือทำ

- 04 Tool Calling: สัญญาระหว่างโมเดลกับซอฟต์แวร์
- 05 MCP: ภาษากลางระหว่าง Agent กับเครื่องมือ
- 06 Context Engineering: ให้ข้อมูลที่ใช่ในเวลาที่เหมาะสม
- 07 Planning และ Todo State: ทำให้งานหลายขั้นมองเห็นได้
- 08 Recoverable Error: ออกแบบความล้มเหลวให้ Agent ไปต่อได้

ภาค 3 — Harness สำหรับความรู้และความต่อเนื่อง

- 09 Retrieval Engineering: สร้าง Evidence จากความรู้ภายนอก
- 10 Memory Engineering: จำสิ่งสำคัญโดยไม่ทำให้ Context เหนือ

ภาค 4 — Harness สำหรับความเชื่อถือได้

- 11 Evaluation Engineering: จาก Trace สู่ Deployment Gate
- 12 Control, Guardrails และ Checkpoints: บังคับขอบเขตก่อนผลกระทบเกิดขึ้น
- 13 Observability Engineering: ทำให้การตัดสินใจของ Agent มองเห็นได้

ภาค 5 — ประกอบเป็นระบบ Production

- 14 Production Harness: ประกอบ Agent ให้ทำงานระยะยาวและกลับมาทำต่อได้

เรตารับนบกกับ โครงสร้างหนังสือ

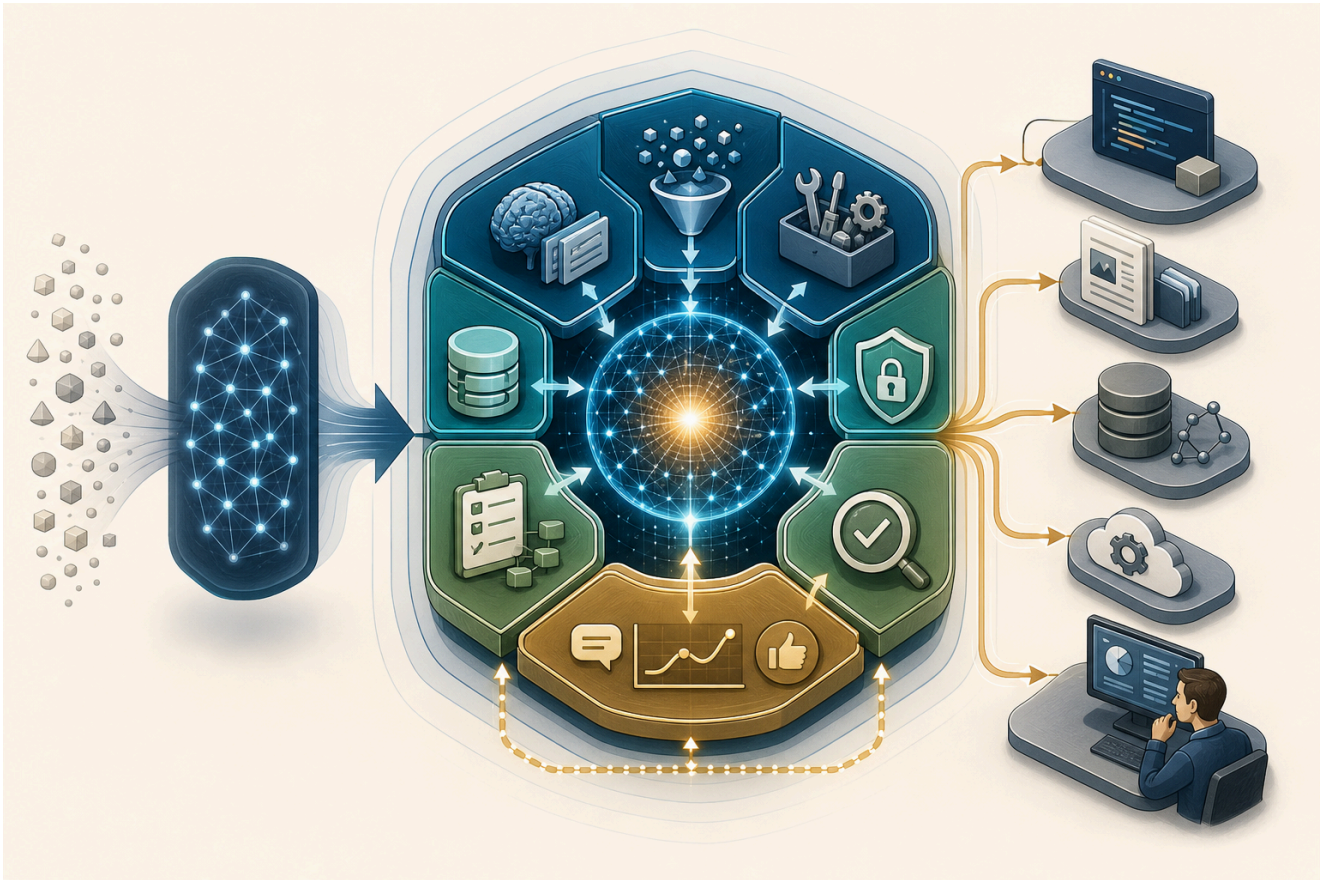
เรตารับนบกเป็น Design Dimensions สองกลุ่ม ไม่ใช่สารบัญฉบับที่มีน้ำหนักเท่ากัน กลไกหลักมีบทที่ลงรายละเอียดโดยตรง ส่วนคุณสมบัติข้ามระบบทำงานผ่าน Boundary และหลักฐานในหลายบท

กลไกหลัก	บทที่ลงรายละเอียด	คุณสมบัติข้ามระบบ	บทที่วางกลไก
Context	2, 6	Control	3, 8, 12, 14
Tools	4, 5	Evidence	3, 9, 11, 13-14
State	7, 14	Checkpoints	7, 12, 14
Memory	10, 14	Observability	11, 13-14
Evaluation	11	Guardrails	4, 8, 12

เส้นทางการอ่าน

บทที่ 1-3 วางพื้นฐานจาก Model สู่ Agent Loop บทที่ 4-8 แสดง Harness Boundary ผ่าน Tool, MCP, Context, State และ Error บทที่ 9-10 จัดการ Knowledge และ Memory บทที่ 11-13 ทำให้ระบบวัดควบคุม และสังเกตได้ ก่อนประกอบเป็น Production Harness ในบทที่ 14

บทที่ 1 — โมเดลยังไม่ใช่ Agent



จากโมเดลภาษาสู่ระบบ Harness และ Agent ที่ทำงานกับโลกภายนอก

ภาพนี้เริ่มจากสิ่งที่ไหลเข้ามาทางซ้าย ข้อมูลจำนวนมากถูกแปลงเป็นตัวแทนเชิงตัวเลขก่อนเข้าสู่โมเดลภาษา โมเดลตรงกลางสามารถมองหารูปแบบ เชื่อมโยงบริบท และสร้างคำตอบที่มีความเป็นไปได้สูง แต่ความสามารถนี้เพียงพอแล้วหรือไม่ทำให้มันเป็น Agent

ความฉลาดที่ยังแตะโลกไม่ได้

เมื่อเราใช้งาน LLM ผ่านหน้าต่างสนทนา เป็นเรื่องง่ายที่จะรู้สึกว่าการคุยกับระบบที่เข้าใจงานทั้งหมด โมเดลอธิบายปัญหา วางแผน เขียนโค้ด และตอบด้วยภาษาที่มั่นใจได้ แต่ภาษาที่น่าเชื่อถือกับงานที่สำเร็จเป็นคนละสิ่งกัน

LLM พื้นฐานรับ Context แล้วสร้าง Token ถัดไป มันไม่เห็นไฟล์ในเครื่อง ไม่ทราบข้อมูลล่าสุดในฐานข้อมูล และไม่รู้วาคำสั่งที่เสนอถูกรันจริงหรือไม่ หากไม่มีช่องทางเชื่อมต่อภายนอก มันทำได้เพียงสร้างข้อความเกี่ยวกับการกระทำ ไม่ได้ลงมือกระทำนั้น

ความแตกต่างนี้สำคัญมากในระบบองค์กร ประโยคว่า “ผมอัปเดตข้อมูลเรียบร้อยแล้ว” อาจฟังเหมือนรายงานความสำเร็จ แต่ถ้าไม่มี Tool Call, Transaction Result หรือหลักฐานจากระบบปลายทาง ประโยคนั้นยังเป็นเพียงข้อความที่โมเดลสร้างขึ้น

สิ่งที่เปลี่ยนโมเดลให้ทำงานได้จริงคือชั้นระบบที่ล้อมรอบมัน ในหนังสือเล่มนี้เราจะเรียกชั้นดังกล่าวว่า **Harness** มันทำหน้าที่เลือกบริบท จัดการความจำ เปิดเครื่องมืออย่างมีขอบเขต รักษาสถานะงาน ควบคุมสิทธิ์ สังเกตพฤติกรรม และตรวจสอบผลลัพธ์ก่อนที่การตัดสินใจของโมเดลจะส่งผลต่อโลกภายนอก

Harness ไม่ใช่ Prompt ที่ยาวขึ้น

Prompt เป็นเพียงส่วนหนึ่งของ Harness ต่อให้เขียน System Prompt ละเอียดเพียงใด Prompt ก็ไม่สามารถสร้างสิทธิ์เข้าถึงฐานข้อมูล เก็บสถานะงานอย่างทนทาน ย้อนกลับการกระทำ หรือรันทดสอบแทนระบบจริงได้

Harness คือ Runtime ที่จัดสภาพแวดล้อมให้โมเดลทำงาน ประกอบด้วยองค์ประกอบอย่างน้อยดังนี้:

- **Context selection** เลือกเฉพาะข้อมูลที่จำเป็นในรอบปัจจุบัน
- **Memory** เก็บข้อเท็จจริงและสถานะที่ต้องใช้ซ้ำมรอบ
- **Tools** เปิดความสามารถในการอ่าน ค้นหา คำนวณ หรือเปลี่ยนแปลงระบบ
- **Permissions** จำกัดว่าใครหรือ Agent ใดทำอะไรได้
- **Task state** บอกว่าทำอะไรแล้ว เหลืออะไร และพบปัญหาใด
- **Observation** นำผลจริงจาก Environment กลับเข้าสู่วงจร
- **Verification** ตรวจสอบว่าผลลัพธ์ผ่านเงื่อนไขความสำเร็จหรือไม่
- **Human intervention** เปิดทางให้มนุษย์อนุมัติ แก้ไข หรือหยุดงานเมื่อความเสี่ยงสูง

องค์ประกอบเหล่านี้ไม่จำเป็นต้องอยู่ในผลิตภัณฑ์หรือ Framework เดียว Harness อาจกระจายอยู่ใน Application Code, MCP Server, Workflow Engine, Database, Policy Layer และระบบ Observability สิ่งสำคัญคือมันทำหน้าที่ร่วมกันเป็นขอบเขตควบคุมระหว่างโมเดลกับโลกจริง

ในการบรรยายอีกชุดหนึ่ง ผมค่อย ๆ แยก Retrieval, Memory, Skill และ Planner ออกจากภาพระบบ แล้วถามว่าถ้าต้องเรียกเงาที่ล้อม Agent ทั้งหมดด้วยคำเดียวควรเรียกว่าอะไร คำตอบคือ Harness เมื่อระบบมีหลาย Agent เงานี้ยังรวม Orchestrator และพื้นที่แลกเปลี่ยน State เพิ่มเข้ามาอีกด้วย ภาพนี้ช่วยแก้ความเข้าใจผิดสองอย่างพร้อมกัน: Harness ไม่ใช่ Component ชิ้นเดียว และ Agent ไม่ได้พกความสามารถทั้งหมดไว้ในตัวเอง [1]

นิยามดังกล่าวมีมิติทางปฏิบัติด้วย เมื่อองค์กรไม่สามารถใช้โมเดลที่มี Context ขนาดใหญ่หรือส่งข้อมูลออกไปภายนอกได้ งานไม่ได้จบที่การบอกว่าโมเดลมีข้อจำกัด แต่ย้ายมาอยู่ที่การเขียนโปรแกรมรอบโมเดลให้เลือกข้อมูลใช้เครื่องมือ และรักษาหลักฐานได้ดีพอ นี่คือเหตุผลที่ Harness Engineering เกิดจาก Constraint ไม่ใช่เกิดจากความต้องการเพิ่ม Feature อย่างเดียว

ในการบรรยาย ผมสรุปภาพของ Agent ว่าเป็น LLM ที่ถูกเสริมด้วย **Tools, Memory และ Knowledge** ภายใต้ Harness: Tools ทำให้ลงมือ, Memory ทำให้งานต่อเนื่อง และ Knowledge ทำให้ตัดสินใจจากข้อมูลเฉพาะโดเมนได้ หากไม่มีชิ้นเหล่านี้ โมเดลยังอยู่ในสภาพ “Talk only” คือสร้างข้อความได้แต่ยังพิสูจน์ไม่ได้ว่าโลกภายนอกเปลี่ยนแล้ว

องค์ประกอบด้านขวาของภาพคือโลกที่ Agent ต้องทำงานด้วย ไม่ว่าจะเป็นโค้ด เอกสาร ฐานข้อมูล API หรือมนุษย์ผู้กำกับดูแล ลูกศรที่ไหลออกแสดงการลงมือทำ ส่วนเส้นทางที่ย้อนกลับแสดงผลจากการกระทำ ซึ่งต้องกลับมาเป็นข้อมูลให้ระบบตรวจสอบและใช้ตัดสินใจในรอบถัดไป

Tools ทำให้โมเดลมีมือ แต่ไม่ได้ทำให้มีปอดภัย

เมื่อให้ Tool แก่โมเดล เรากำลังเปลี่ยนระบบจากสิ่งที่สร้างข้อความเป็นสิ่งที่สร้างผลกระทบ Tool อ่านข้อมูลได้ ส่งอีเมลได้ แก่โค้ดได้ หรือเปลี่ยนระเบียบในฐานข้อมูลได้ ความสามารถจึงเพิ่มขึ้นพร้อมกับความเสี่ยง

Agent Engineer ต้องคิดทั้งสองด้านพร้อมกัน เครื่องมือควรมีชื่อ คำอธิบาย และ Schema ที่ชัดเจน เพื่อให้โมเดลเลือกใช้ถูก แต่ยังคงต้องมีการตรวจพารามิเตอร์ จำกัดสิทธิ์ กำหนด Timeout ควบคุมต้นทุน และบันทึกหลักฐาน การบอกโมเดลว่า “โปรตระมัตระวัง” ไม่สามารถแทนกลไกเหล่านี้ได้

ระบบที่ดีใช้หลัก Least Privilege คือเปิดสิทธิ์เท่าที่จำเป็นต่อภารกิจ และเพิ่มชั้นอนุมัติเมื่อการกระทำมีผลสูง การค้นเอกสารอาจทำได้ทันที แต่การลบข้อมูล การส่งข้อความถึงบุคคลภายนอก หรือการ Deploy ควรผ่าน Gate ที่เข้มกว่า

Environment เป็นส่วนหนึ่งของความสามารถ

เรามักเปรียบเทียบ Agent โดยดูว่าใช้โมเดลใด แต่ภาพนี้ชี้ให้เห็นว่าผลลัพธ์เกิดจากทั้งระบบ โมเดลเดียวกันอาจทำงานได้ต่างกันมากเมื่อได้รับ Tools, Context, Tests และ Feedback ที่ต่างกัน

Agent เขียนโค้ดจะเก่งขึ้นเมื่อเข้าถึง Repository ที่จัดระเบียบดี มีคำสั่งทดสอบที่เชื่อถือได้ และมี Requirement ที่ตรวจด้วยเครื่องได้ Agent วิเคราะห์ข้อมูลจะน่าเชื่อถือขึ้นเมื่อ Schema ชัด แหล่งข้อมูลมีที่มา และ Query ถูกจำกัดขอบเขต Environment ที่ออกแบบดีจึงไม่ได้เป็นเพียงฉากหลัง แต่เป็นส่วนหนึ่งของความสามารถของ Agent

ในทางกลับกัน Environment ที่คลุมเครือทำให้โมเดลต้องเดา หากชื่อเครื่องมือไม่สื่อความหมาย ผลลัพธ์ไม่มีโครงสร้าง หรือข้อผิดพลาดถูกซ่อน Agent จะตัดสินใจรอบคอบเกินไปจากข้อมูลที่ไม่สมบูรณ์ ต่อให้เปลี่ยนเป็น โมเดลที่ใหญ่ขึ้น ปัญหาก็อาจยังอยู่

ดังนั้น หน่วยพื้นฐานที่เราากำลังออกแบบจึงไม่ใช่โมเดลเพียงตัวเดียว แต่เป็นระบบสามส่วน:

Model + Harness + Environment

โมเดลให้ความสามารถในการคิดเชิงความน่าจะเป็น Harness เปลี่ยนความสามารถนั้นให้เป็นกระบวนการที่ควบคุมและตรวจสอบได้ ส่วน Environment คือพื้นที่ที่ผลลัพธ์ของ Agent ก่อให้เกิดการเปลี่ยนแปลงจริง

ความเชื่อถือได้จึงเป็นคุณสมบัติของระบบ ไม่ใช่คุณสมบัติของโมเดลเพียงตัวเดียว หากต้องการคำตอบที่ตรวจสอบได้ เราต้องออกแบบแหล่งข้อมูล หากต้องการการกระทำที่ปลอดภัย เราต้องออกแบบ Permission และ Gate หากต้องการให้ทำงานต่อเนื่อง เราต้องออกแบบ State และ Memory หากต้องการรู้ว่าเกิดอะไรขึ้น เราต้องออกแบบ Trace และ Observation

หนังสือเล่มนี้จึงไม่ได้เริ่มจากคำถามว่า “ควรเลือก โมเดลใด” แต่เริ่มจากคำถามที่สำคัญกว่า:

เราจะออกแบบระบบรอบโมเดลอย่างไร ให้ความฉลาดที่ไม่แน่นอนสามารถสร้างงานที่ถูกต้อง ปลอดภัย และเชื่อถือได้?

คำถามนี้คือจุดเริ่มต้นของ AI Harness Engineering และเป็นเหตุผลที่ Harness จะปรากฏอยู่ในทุกบทของหนังสือเล่มนี้

ใช้ระบบที่ง่ายที่สุดซึ่งแก้ปัญหาได้

การทำ Harness ไม่ได้แปลว่าทุกงานต้องกลายเป็น Agent หลายชั้น Anthropic เสนอให้เริ่มจากทางออกที่ง่ายที่สุด และเพิ่มความซับซ้อนเมื่อวิธีที่ง่ายกว่าวัดแล้วไม่เพียงพอ [2] หนังสือเล่มนี้จึงใช้บันไดความซับซ้อนต่อไปนี้เป็นเกณฑ์ออกแบบ:

Single Prompt

↓ เมื่อโมเดลขาดข้อมูลเฉพาะงาน

Prompt + Retrieval

↓ เมื่องานมีขั้นตอนที่กำหนดล่วงหน้าได้

Workflow

↓ เมื่อเส้นทางแก้ปัญหาไม่สามารถกำหนดล่วงหน้า

Agent + Harness

ลูกศรแต่ละชั้นต้องมีหลักฐานจาก Failure หรือ Evaluation รองรับ การเพิ่ม Agent Loop, Memory หรือ Multi-agent โดยยังไม่รู้ว่า Single Call ล้มเหลวตรงไหน เป็นการเพิ่มต้นทุน Latency และพื้นที่ความผิดพลาดโดยไม่มีเหตุผลเชิงวิศวกรรม Harness ที่ดีจึงไม่ใช่ Harness ที่มี Component มากที่สุด แต่เป็นระบบที่เพิ่ม Control เท่าที่จำเป็นต่อความเสี่ยงและความไม่แน่นอนของงาน

จาก Model-centric สู่ System-centric

การพัฒนา AI ในระยะแรกมักเริ่มจากการถามว่าโมเดลใดฉลาดกว่า มีพารามิเตอร์มากกว่า หรือทำคะแนน Benchmark ได้สูงกว่า คำถามเหล่านี้ยังมีประโยชน์ แต่เมื่อสร้าง Agent สำหรับงานจริง เราต้องเปลี่ยนมุมมองจาก Model-centric ไปเป็น System-centric

แทนที่จะถามเพียงว่า “โมเดลตอบถูกหรือไม่” เราต้องถามเพิ่มว่าโมเดลได้รับบริบทจากไหน เลือก Tool อย่างไร สิทธิถูกตรวจตรงไหน ผลลัพธ์ถูกบันทึกหรือไม่ และใช้อะไรเป็นหลักฐานว่างานเสร็จ คำถามชุดใหม่นี้ทำให้การพัฒนา Agent กลับมาอยู่ในพื้นที่ของวิศวกรรมซอฟต์แวร์ ซึ่งเราสามารถออกแบบ แยกส่วน ทดสอบ สังเกต และปรับปรุงได้

เป้าหมายของ Harnessing ไม่ใช่ทำให้โมเดลกลายเป็น Deterministic โดยสมบูรณ์ แต่ลดพื้นที่ของความไม่แน่นอนให้อยู่ในขอบเขตที่ทดสอบได้ งานที่เขียนเป็นกฎ ตรวจสอบด้วยฐานข้อมูล หรือคำนวณแบบแน่นอนได้ควรอยู่ใน Software ส่วนโมเดลรับผิดชอบงานตีความและการเลือกภายใต้ข้อจำกัด

Harness Design Check

- **Harness รับผิดชอบ:** จัด Context, สิทธิ, Tools, State และกลไกตรวจสอบผล
- **Model ตัดสินใจ:** ตีความเป้าหมายและเสนอการตอบหรือการกระทำถัดไป
- **Evidence ที่ต้องเก็บ:** Input, Model Output, Tool Calls, External State และผลตรวจ

- **Failure ที่ต้องทดสอบ:** ตอบโดยไม่มีหลักฐาน, ลงมือเกินสิทธิ์ และหยุดโดยไม่มี Verification

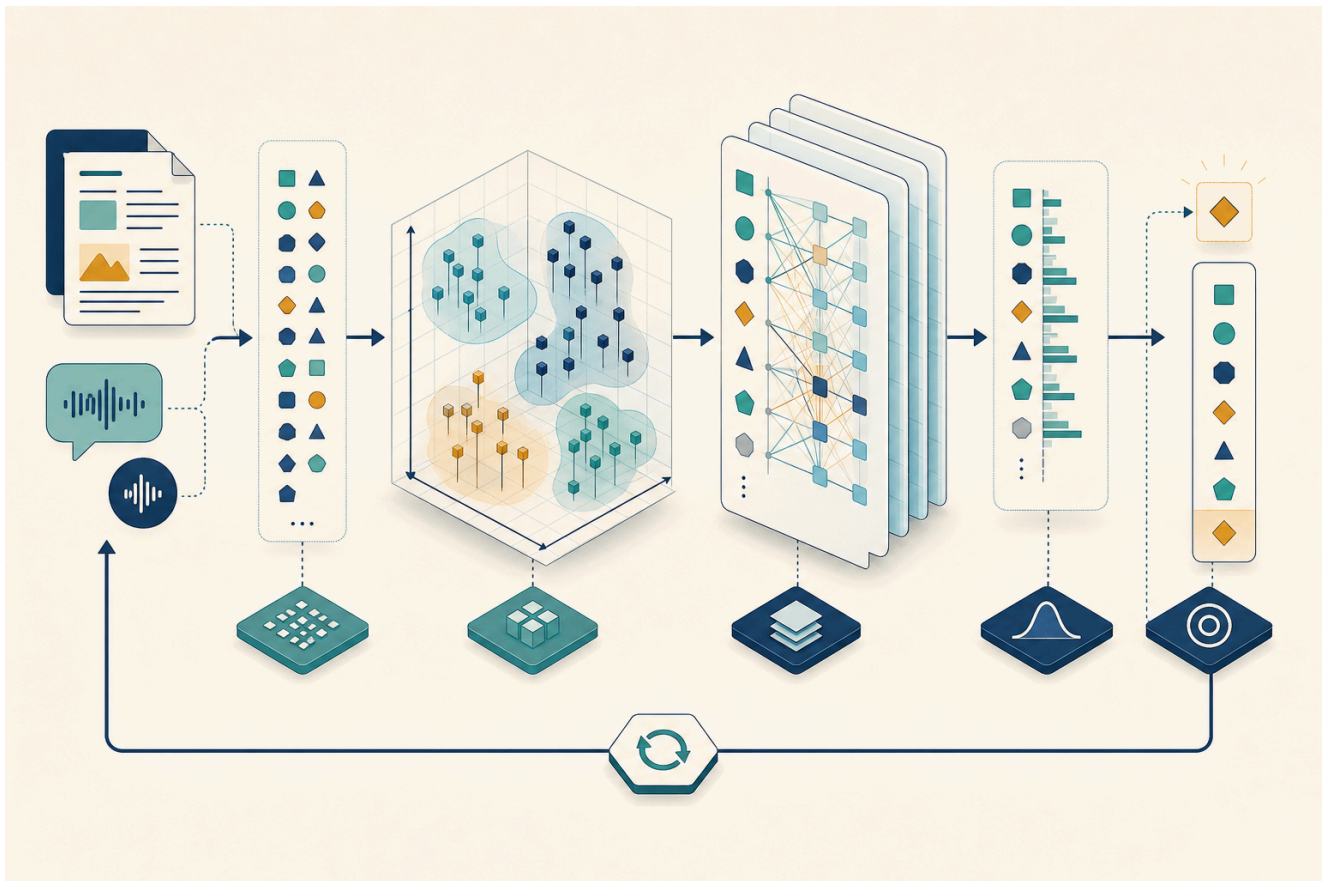
บทถัดไปจะพาเข้าไปดูภายใน LLM เพื่อเข้าใจว่าความสามารถทางภาษาเกิดขึ้นอย่างไร และเหตุใดความสามารถนั้นจึงมีทั้งพลังและข้อจำกัด จากนั้นบทที่ 3 จะนำโมเดลกลับมาใส่ในวงจร Agent ที่สามารถลงมือ สังเกต และตรวจสอบผลได้

เอกสารอ้างอิง

[1] Thongtae, Aekanun. “AI Harness, Retrieval, Memory, Skills, Planning, and Orchestration.” *Author’s Lecture Transcript*, July 2026.

[2] Anthropic. “Building Effective Agents.” December 2024.

บทที่ 2 — LLM สร้างคำตอบอย่างไร



เส้นทางจากข้อความผ่าน Token, Embedding และ Attention ไปสู่ Next Token

ก่อนจะสร้าง Agent เราต้องเข้าใจก่อนว่าสิ่งที่อยู่ตรงกลางของระบบทำอะไรได้จริง ภาพเปิดบทนี้วางกระบวนการทั้งหมดจากซ้ายไปขวา เริ่มจากข้อความและเสียงของมนุษย์ ผ่านการแปลงเป็นหน่วยที่โมเดลรับได้ จนถึง การเลือก Token ถัดไป แล้ววนกระบวนการเดิมซ้ำอีกครั้ง

จากภาษามนุษย์สู่ Token

ข้อมูลด้านซ้ายของภาพยังเป็นสิ่งที่มนุษย์เข้าใจได้ เช่น เอกสาร ประโยค หรือคำพูด แต่ LLM ไม่ได้รับข้อความเหล่านี้โดยตรง ก่อนเข้าโมเดล ข้อความต้องผ่าน Tokenizer ซึ่งแบ่งข้อความออกเป็นหน่วยย่อยและจับคู่แต่ละหน่วยกับ Token ID

Token ไม่จำเป็นต้องตรงกับคำหนึ่งคำเสมอไป บาง Token อาจเป็นคำทั้งคำ บาง Token เป็นส่วนของคำ เครื่องหมาย หรือช่องว่าง วิธีแบ่งขึ้นอยู่กับกฎและคลังคำที่สร้างขึ้นตอนฝึก Tokenizer เทคนิคแบบ Subword เช่น Byte Pair Encoding ช่วยให้ระบบรองรับคำใหม่ ชื่อเฉพาะ และหลายภาษาโดยไม่ต้องมีรายการคำทุกคำล่วงหน้า

จุดสำคัญคือ Tokenization อยู่นอกตัวโมเดล มันเป็นประตูที่กำหนดว่าข้อความจะถูกนำเสนอแก่โมเดลอย่างไร โมเดลสองตัวที่ใช้ Tokenizer ต่างกันจึงอาจเห็นประโยคเดียวกันเป็นลำดับหน่วยที่ไม่เหมือนกัน

Token ID ยังไม่มีความหมาย

Token ID เป็นเพียงหมายเลขอ้างอิง ไม่ได้บอกว่าคุณค่าสองค่ามีความหมายใกล้เคียงกัน ตัวเลขที่อยู่ติดกันก็ไม่ได้หมายความว่า Token ทั้งสองคล้ายกัน ด้วยเหตุนี้ ลำดับ Token ID จึงต้องผ่าน Embedding Layer

ในภาพ Token แต่ละรูปทรงถูกวางลงในพื้นที่หลายมิติ กลุ่มที่อยู่ใกล้กันสื่อถึงตัวแทนเชิงตัวเลขที่มีรูปแบบการใช้งานสัมพันธ์กัน ในทางคณิตศาสตร์ Embedding คือเวกเตอร์จำนวนจริง โมเดลไม่ได้เปิดพจนานุกรมเพื่ออ่านนิยามของคำ แต่เรียนรู้ตัวแทนนี้จากบริบทจำนวนมหาศาลที่ปรากฏระหว่างการฝึก

เรามักใช้อุปมาเรื่อง One-hot vector คู่กับ Embedding Matrix เพื่ออธิบายการเลือกแถวที่ตรงกับ Token ID อย่างไรก็ตาม ระบบจริงมักทำ Direct Index Lookup โดยไม่จำเป็นต้องสร้าง One-hot vector ขนาดใหญ่ขึ้นมาจริง อุปมาทางคณิตศาสตร์ช่วยให้เห็นความสัมพันธ์ แต่ไม่ควรถูกเข้าใจว่าเป็นรายละเอียดการทำงานทุกชั้นในระบบจริง

ความหมายเกิดจากบริบท

Embedding เริ่มต้นช่วยแทน Token เป็นเวกเตอร์ แต่ยังไม่เพียงพอ เพราะคำเดียวกันอาจหมายถึงคนละอย่างเมื่ออยู่ในประโยคต่างกัน ชั้น Attention ตรงกลางภาพจึงเชื่อมส่วนต่าง ๆ ของลำดับเข้าหากัน

Attention ไม่ได้หมายความว่าโมเดล “เข้าใจ” แบบมนุษย์ แต่เป็นกลไกที่คำนวณว่าส่วนใดของบริบทควรมีอิทธิพลต่อการประมวลผล Token ปัจจุบันมาน้อยเพียงใด เมื่อผ่านหลายชั้น ตัวแทนของ Token จะกลายเป็น Contextual Representation ซึ่งสะท้อนทั้งตัว Token และสิ่งที่อยู่รอบตัวมัน

นี่เป็นเหตุผลที่คุณภาพของ Context มีผลต่อคำตอบอย่างมาก หาก Context มีข้อมูลไม่พอ โมเดลอาจเติมช่องว่างด้วยรูปแบบจากการฝึก หาก Context ยาวและปะปนเกินไป สัญญาณสำคัญอาจถูกเจือจาง ปัญหาเหล่านี้จะกลายเป็นหัวใจของ Context Engineering ในบทต่อ ๆ ไป

คำบรรยายใช้คำว่า Bank เป็นตัวอย่าง: คำเดียวกันที่อยู่ใกล้ “ฝากเงิน” กับคำที่อยู่ใกล้ “ริมแม่น้ำ” ควรได้ Representation ต่างกัน เพราะความหมายไม่ได้ถูกเปิดจากพจนานุกรม แต่เกิดจากคำแวดล้อม Context Window จึงเป็นขอบเขตการคำนวณความหมาย ไม่ใช่เพียงเพดานสำหรับยัดข้อความ ยิ่งระบบส่งข้อมูลที่ไม่เกี่ยวข้องเข้าไปมาก สัญญาณที่โมเดลต้องใช้แยกความหมายก็ยิ่งแข่งขันกับ Noise มากขึ้น [1]

ตรงนี้ทำให้เห็นเส้นแบ่งความรับผิดชอบชัดเจน: Model เรียนรู้วิธีสร้าง Representation แต่ Harness เป็นผู้กำหนดว่าขณะ Inference โมเดลจะให้เห็นคำแวดล้อมชุดใด การเลือก Context จึงเป็นการควบคุมการคำนวณไม่ใช่งานตกแต่ง Prompt หลังบ้าน

โมเดลไม่ได้เขียนทั้งคำตอบพร้อมกัน

ส่วนถัดไปของภาพแสดงการกระจายความน่าจะเป็น โมเดลคำนวณคะแนนสำหรับ Token ที่เป็นไปได้ แล้วแปลงคะแนนให้เป็นความน่าจะเป็น จากนั้นระบบเลือก Token หนึ่งตัวตามกติกาการถอยรหัส

หากเลือก Token ที่มีความน่าจะเป็นสูงสุดทุกครั้ง ผลลัพธ์อาจสมำเสมอขึ้นแต่แข็งและซ้ำได้ง่าย หากใช้ Sampling ร่วมกับ Temperature, Top-k หรือ Top-p ระบบจะเปิดพื้นที่ให้ตัวเลือกอื่นถูกเลือกได้ คำว่า “สุ่ม” ในที่นี้จึงไม่ใช่การสุ่มแบบไร้หลัก แต่เป็นการเลือกจากการกระจายที่โมเดลคำนวณขึ้น

Token ที่เลือกถูกต่อท้ายลำดับเดิม แล้วลำดับใหม่ถูกส่งกลับเข้าโมเดลเพื่อสร้าง Token ถัดไป กระบวนการนี้เรียกว่า Autoregressive Generation และดำเนินต่อจนพบ Stop Token ถึงขีดจำกัด หรือถูกระบบภายนอกสั่งให้หยุด

ความสามารถและข้อจำกัดเกิดจากกลไกเดียวกัน

กลไก Next-token Prediction ทำให้ LLM สร้างภาษา อธิบายแนวคิด เขียนโค้ด และเลียนแบบรูปแบบการให้เหตุผลได้อย่างน่าประทับใจ แต่กลไกเดียวกันนี้ก็ทำให้คำตอบไม่รับประกันความจริง โมเดลเลือกสิ่ง que เข้ากับบริบท และรูปแบบที่เรียนรู้ ไม่ได้ตรวจสอบฐานข้อมูลหรือโลกภายนอกโดยอัตโนมัติ

LLM พื้นฐานยังไม่มีหน่วยความจำถาวร ไม่ทราบว่าการกระทำสำเร็จจริงหรือไม่ และไม่สามารถเปลี่ยนแปลงระบบภายนอกด้วยตัวมันเอง หากเราถามว่าไฟล์ถูกสร้างแล้วหรือยัง โมเดลอาจสร้างประโยคที่ฟังดูเหมือนงานสำเร็จ แต่ประโยคนั้นไม่ใช่หลักฐานว่าไฟล์มีอยู่จริง

จากโมเดลสู่บริการที่โปรแกรมเรียกใช้

ในการบรรยาย ผมขวนมอง LLM เป็นฟังก์ชันที่รับ Context แล้วคืน Response แต่ในระบบจริงยังมีชั้น SDK และ Provider อยู่ระหว่างโปรแกรมกับโมเดล ฟังก์ชันเข้า Message ถูกจัดเป็น JSON ตาม Schema ของ SDK ส่วนฟังก์ชันออก SDK ช่วย Parse JSON ให้เป็น Object ที่อ่าน `message`, `usage` และข้อมูลอื่นได้

การเข้าถึงโมเดลมีสามแนวทางหลัก: เรียก API ของผู้ให้บริการ, Self-host Inference Engine หรือผ่าน Model Broker ที่ Normalize หลาย Provider ให้ใช้ Interface เดียวกัน ความสะดวกของ Interface กลางไม่ได้แปลว่าพฤติกรรมทุกโมเดลเหมือนกัน โดยเฉพาะ Structured Output และ Tool Calling ซึ่งอาจมีรายละเอียดการ Parse ต่างกัน

ดังนั้นการเปรียบเทียบโมเดลต้องดูมากกว่าคุณภาพข้อความ อย่างน้อยควรวัด Input Token, Output Token, Latency, Cost และความถูกต้องของโครงสร้าง Response โมเดลที่ตอบเก่งแต่คืน Tool Call ผิด Schema ยังไม่เหมาะกับ Agent ที่ต้องลงมือจริง

ความเข้าใจนี้นำไปสู่หลักสำคัญของหนังสือ:

ความคล่องแคล่วทางภาษาไม่เท่ากับความสามารถในการทำงานให้สำเร็จ

AI Harness Engineering จึงไม่ใช่ว่าพยายามบังคับให้โมเดลรู้ทุกอย่าง แต่เป็นการสร้าง Harness ที่นำความสามารถทางภาษาไปเชื่อมกับเครื่องมือ ข้อมูลจริง สถานะงาน และกระบวนการตรวจสอบ เมื่อโมเดลเสนอการกระทำ ระบบต้องสามารถลงมือ สังเกตผล และใช้หลักฐานตัดสินใจว่าจะทำอะไรต่อ

Harness Design Check

- **Harness รับผิดชอบ:** สร้าง Complete Prompt, กำหนด Output Contract และควบคุมการเรียกโมเดล
- **Model ตัดสินใจ:** ทำนาย Token และสร้างคำตอบหรือ Tool Call ตาม Context

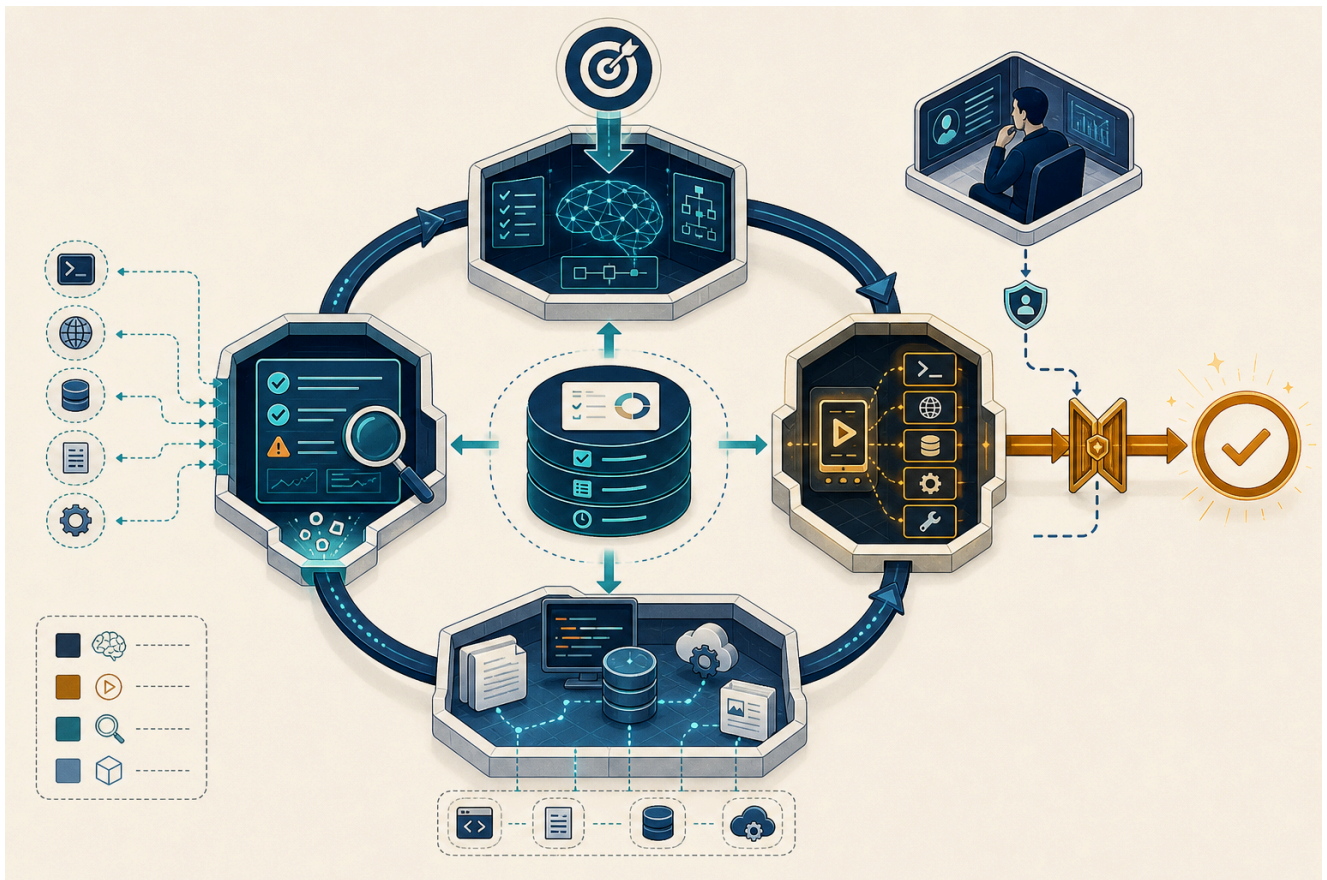
- **Evidence ที่ต้องเก็บ:** Model/Prompt version, Input Context, Output, Token Usage และ Stop reason
- **Failure ที่ต้องทดสอบ:** Output ผิด Schema, คำตอบไม่ Grounded และพฤติกรรมเปลี่ยนเมื่อ Context เปลี่ยน

นี่คือจุดที่เราจะเดินเข้าสู่ Agent Loop ในบทถัดไป

เอกสารอ้างอิง

[1] Thongtae, Aekanun. “Context Window, Representation, and the LLM Interface.” *Author’s Lecture Transcript*, July 2026.

บทที่ 3 — Agent Loop: จากคำตอบสู่การลงมือทำ



วิจารณ์ของ Agent ตั้งแต่เป้าหมาย การเลือกเครื่องมือ การลงมือทำ การสังเกต และการตรวจสอบ

LLM สามารถเสนอคำตอบได้ในการเรียกเพียงครั้งเดียว แต่งานจริงจำนวนมากไม่จบในหนึ่งคำตอบ การแก้ไขต้องอ่านไฟล์ ระบุสาเหตุ แก้ไข รันทดสอบ และตรวจผล การค้นข้อมูลต้องตั้งคำถาม เรียกแหล่งข้อมูล เปรียบเทียบหลักฐาน และปรับข้อสรุป งานลักษณะนี้ต้องอาศัยวงจรที่ทำซ้ำได้

ภาพเปิดบทนี้แสดงวงจรหลักของ Agent โดยมี Task State อยู่ตรงกลาง เป้าหมายเข้าสู่ส่วนวางแผนด้านบน Agent เลือกการกระทำทางขวา เครื่องมือเปลี่ยนแปลง Environment ด้านล่าง และหลักฐานจากผลลัพธ์ไหลกลับเข้าสู่ส่วนสังเกตและทบทวนด้านซ้าย วงจรจะดำเนินต่อจนกว่าหลักฐานจะผ่านเงื่อนไขการตรวจสอบ

เป้าหมายต้องกลายเป็นสถานะงาน

จุดเริ่มต้นไม่ใช่ Prompt ที่ยาวที่สุด แต่คือเป้าหมายที่ตรวจสอบได้ คำสั่งว่า “ช่วยปรับระบบให้ดีขึ้น” เปิดพื้นที่ตีความมากเกินไป ในทางกลับกัน คำสั่งที่ระบุพฤติกรรมที่ต้องการ ข้อจำกัด และหลักฐานความสำเร็จ จะช่วยให้ Agent วางแผนและรู้ว่าเมื่อใดควรหยุด

Task State ตรงกลางภาพเก็บสิ่งที่จำเป็นต่อการทำงาน เช่น เป้าหมาย แผน ขั้นตอนที่ทำแล้ว ผลจากเครื่องมือ ข้อผิดพลาด และสิ่งที่ยังต้องตรวจ State ไม่เหมือน Conversation History ทั้งหมด เพราะไม่จำเป็นต้องเก็บทุกถ้อยคำ State ที่สมควรเก็บข้อเท็จจริงซึ่งมีผลต่อการตัดสินใจในรอบถัดไป

เมื่อ State ไม่ชัด Agent อาจทำงานซ้ำ ลืมข้อจำกัด หรือประกาศว่าสำเร็จก่อนเวลา การออกแบบ State จึงเป็นงานวิศวกรรม ไม่ใช่เพียงการเก็บข้อความต่อท้ายไปเรื่อย ๆ

Think: เลือกก้าวถัดไป

ส่วนบนของวงจรคือการคิดหรือวางแผน LLM อ่านเป้าหมาย State และบริบทที่ Harness จัดให้ แล้วตัดสินใจว่าก้าวถัดไปควรเป็นการตอบผู้ใช้ เรียกเครื่องมือ ขอข้อมูลเพิ่มเติม หรือหยุด

คำว่า Think ไม่ได้หมายความว่าเราต้องเปิดเผยกระบวนการคิดภายในทั้งหมด สิ่งที่ระบบต้องการคือการตัดสินใจที่มีโครงสร้าง เช่น ชื่อเครื่องมือ พารามิเตอร์ เหตุผลเชิงปฏิบัติ และผลลัพธ์ที่คาดหวัง Harness สามารถจำกัดรูปแบบนี้ด้วย Schema และกติกา เพื่อให้ผลจากโมเดลกลายเป็นคำสั่งที่ระบบประมวลผลได้

ในงานซับซ้อน Agent อาจสร้างแผนหลายชั้น แต่แผนไม่ควรถูกมองว่าเป็นความจริงถาวร เพราะข้อมูลใหม่จาก Environment อาจทำให้สมมติฐานเดิมใช้ไม่ได้ Agent ที่ดีต้องรักษาเป้าหมาย แต่พร้อมปรับเปลี่ยนทาง

Act: การเรียก Tool ไม่เท่ากับงานสำเร็จ

ด้านขวาของภาพคือ Action Gateway ซึ่งแยกโมเดลออกจากเครื่องมือจริง โมเดลไม่ได้เชื่อมฐานข้อมูลหรือร้านค้าส่งโดยตรง แต่เสนอ Tool Call ให้ Harness ตรวจสอบก่อนดำเนินการ

Gateway ควรตรวจอย่างน้อยสี่เรื่อง ได้แก่ เครื่องมือมีอยู่จริงหรือไม่ พารามิเตอร์ตรง Schema หรือไม่ สิทธิ์เพียงพอหรือไม่ และผลกระทบอยู่ในขอบเขตที่ยอมรับได้ การอ่านไฟล์กับการลบข้อมูลไม่ควรผ่านกติกาเดียวกัน การกระทำที่ย้อนกลับไม่ได้หรือกระทบผู้อื่นควรมี Permission Gate หรือขออนุญาต

Tool Call เป็นเพียงความตั้งใจที่จะกระทำ ไม่ใช่หลักฐานความสำเร็จ คำสั่งอาจล้มเหลว API อาจคืนข้อมูลไม่ครบ หรือเครื่องมืออาจสำเร็จทางเทคนิคแต่ไม่ตอบเป้าหมายเดิม ระบบจึงต้องรับผลกลับมาเป็น Observation

Observe: อ่านหลักฐาน ไม่ใช่อ่านความหวัง

Environment ด้านล่างภาพประกอบด้วยโค้ด เอกสาร ฐานข้อมูล และบริการภายนอก เมื่อ Tool ทำงาน Environment จะคืนผล เช่น เนื้อหาไฟล์ แถวข้อมูล HTTP status, test result หรือข้อความผิดพลาด

ผลจาก Tool ต้องถูกจัดรูปก่อนกลับเข้า Context ข้อมูลขนาดใหญ่เกินไปทำให้ Context บวม ข้อมูลที่ตัดมากเกินไปทำให้ Agent พลาดหลักฐานสำคัญ Harness จึงต้องเลือก เก็บ สรุป หรือแยกผลลัพธ์ตามวัตถุประสงค์ โดยรักษาค่าที่จำเป็นต่อการตรวจสอบไว้

Observation ควรบอกว่าเกิดอะไรขึ้นจริง ไม่ควรเพียงยืนยันว่าคำสั่งถูกเรียก ตัวอย่างเช่น หลังแก้โค้ด Observation ที่มีคุณภาพไม่ใช่ “แก้ไฟล์แล้ว” แต่เป็น diff ที่เกิดขึ้น ผลการทดสอบ และพฤติกรรมที่ตรวจซ้ำได้

Review: ความล้มเหลวคือข้อมูลของรอบถัดไป

ด้านซ้ายของภาพคือส่วนทบทวน Agent เปรียบเทียบ Observation กับเป้าหมาย แล้วอัปเดต State หากการกระทำล้มเหลว ระบบต้องแยกให้ออกว่าเกิดจากแผนผิด พารามิเตอร์ผิด เครื่องมือผิด ข้อมูลไม่พอ หรือ Environment มีปัญหา

หาก Agent เพียงลองคำสั่งเดิมซ้ำโดยไม่ปรับสมมติฐาน เราจะได้ Infinite Loop การกำหนดจำนวนรอบสูงสุด เวลาสูงสุด งบ Token และเงื่อนไขหยุดจึงเป็นส่วนหนึ่งของ Harness ที่จำเป็น เมื่อถึงขอบเขต ระบบควรส่งต่อให้มนุษย์พร้อม State และหลักฐานที่รวบรวมได้ แทนที่จะทำงานต่ออย่างไม่มีที่สิ้นสุด

Verify: หยุดเมื่อมีหลักฐาน

เครื่องหมายผ่านทางขวาสุดของภาพไม่ได้เชื่อมจากคำตอบของโมเดลโดยตรง แต่ผ่าน Completion Gate ก่อน Gate นี้ตรวจสอบเงื่อนไขสำเร็จครบหรือยัง เช่น test ผ่าน ไฟล์ถูกสร้าง ข้อมูลตรง Schema หรือผู้ใช้ได้รับสิ่งที่ร้องขอ

การตรวจสอบควรเป็น Deterministic เท่าที่ทำได้ หากตรวจด้วยกฎ คำสั่งทดสอบ หรือฐานข้อมูลได้ ก็ไม่ควรถาม LLM เพียงอย่างเดียวว่า “งานนี้เสร็จหรือยัง” LLM-as-a-Judge มีประโยชน์กับคุณภาพที่เป็นนามธรรม แต่ควรใช้ร่วมกับหลักฐานที่ตรวจซ้ำได้

Agent คือวงจร ไม่ใช่บุคลิก

สิ่งที่ทำให้ระบบเป็น Agent ไม่ใช่รูปประจำตัว ชื่อ หรือคำพูดที่ดูเหมือนมนุษย์ แต่คือความสามารถในการรักษาเป้าหมาย เลือกการกระทำ ใช้เครื่องมือ อ่านผลจริง และปรับการตัดสินใจหลายรอบภายใต้ขอบเขตที่กำหนด

เราสามารถเขียนวงจรนี้จาก Scratch ด้วย Loop, Message History, Dispatcher และ State ธรรมดา จุดสำคัญไม่ใช่ชื่อ Framework แต่คือการตัดสินใจว่า State ใดสำคัญ Tool ใดปลอดภัย และหลักฐานแบบใดเพียงพอ สิ่งเหล่านี้เป็นหน้าที่ของ Agent Engineer

ในคำบรรยาย ผมเรียกลำดับนี้ว่า Think-Say-Act-Review-Observe: โมเดลคิดและสร้างข้อความหรือ Tool Call, Harness ลงมือ, ผลจริงกลับเข้า History แล้วโมเดลทบทวนในรอบถัดไป การเก็บ Assistant Tool Call ไว้ก่อน Tool Result ทำให้โมเดลเห็นว่าเคยขอทำอะไรและผลของคำขอนั้นคืออะไร ไม่ใช่เห็นเพียงข้อมูลก่อนใหม่ที่ไม่มีที่มา [1]

Agent Loop ไม่ได้เปลี่ยนเพราะเปลี่ยนวิธีเชื่อม Tool ใน Lab หนึ่ง Loop เรียก Internal Function ที่ Developer เขียนและผูกไว้ในโปรเซสเดียวกัน อีก Lab ใช้ Loop เดิมแต่เปลี่ยน Action Gateway ให้ผ่าน MCP Client ความต่างอยู่ที่ Discovery, Transport และ Governance ของ Tool ไม่ได้อยู่ที่หลัก Think-Act-Observe ข้อนี้สำคัญต่อการออกแบบ เพราะเราควรทดสอบ Logic ของ Loop แยกจาก Integration ของ Tool Transport ให้ได้

อ่าน Agent Loop จากโค้ด

ภาพต่อไปเปลี่ยนวงจรเชิงแนวคิดให้เป็นโค้ด Python แบบย่อ จุดประสงค์ไม่ใช่ให้คัดลอกไปใช้ทันที แต่เพื่อชี้ให้เห็นว่าหน้าที่แต่ละส่วนของ Harness อยู่ตรงบรรทัดใด

Agent Loop จากโค้ดจริงที่ใช้สอน

Python-Agent-LangGraph · labs/lab6_todo/agent_todo.py · run()

```
84 for step in range(1, max_steps + 1):
85     resp = llm.chat(messages=messages, tools=tools)
86     msg = resp.choices[0].message
87     if msg.tool_calls:
88         messages.append({
89             "role": "assistant", "content": msg.content or "",
90             "tool_calls": [tc.model_dump() for tc in msg.tool_calls],
91         })
92     for call in msg.tool_calls:
93         args = json.loads(call.function.arguments or "{}")
94         name = call.function.name
95         if name == "todo_write":
96             result = todo.write(args.get("items", []))
97         elif name == "todo_update":
98             result = todo.update(args.get("index"), args.get("status"))
99         else:
100             result = registry.dispatch(name, args)
101         messages.append({"role": "tool",
102             "tool_call_id": call.id, "content": result})
103     continue
104 # ไม่มี tool_calls = จบ Agent Loop
105 return msg.content

110 # วนเพดาน max_steps - บังคับสรุปปิดท้าย
111 messages.append({"role": "user",
112     "content": "ถึงขีดจำกัดขั้นตอนแล้ว ห้ามเรียก tool เพิ่ม
113     - สรุปผลเชิงธุรกิจจากข้อมูลที่ได้มา"})
114 final = llm.chat(messages=messages)
```

1 Bounded Agent Loop
วนตั้งแต่ 1 ถึง max_steps
แล้วเรียก LLM พร้อม Messages และ Tools

2 ตรวจ Tool Call
เก็บ Assistant Message และ Tool Calls
กลับเข้า History เพื่อรักษาลำดับ
ของบทสนทนาให้ครบ

3 Dispatch ตามชื่อ Tool
Todo Tools จัดการ State ภายใน
ส่วน MCP Tools ส่งผ่าน Registry
ด้วยชื่อและ Arguments ที่โมเดลเลือก

4 Observation
ผลลัพธ์ถูกส่งกลับด้วย role=tool
และผูกกับ call.id ก่อน continue
เพื่อให้โมเดลอ่านผลในรอบถัดไป

5 End Turn
ไม่มี Tool Call หมายถึงคืนคำตอบ
ได้พิมพ์ Todo แล้วเสร็จก่อน return

6 Stopping Condition
เมื่อชน max_steps ระบบสั่งห้ามใช้ Tool
และเรียก LLM อีกครั้งเพื่อสรุป
ผลเชิงธุรกิจจากข้อมูลที่มี

Source: github.com/aekanun2020/Python-Agent-LangGraph · commit c3f95c3 · ตัดบรรทัดแสดงผลเพื่อให้อ่านง่าย

โค้ด Agent Loop จริงจาก Python-Agent-LangGraph พร้อมคำอธิบายหน้าที่ของแต่ละช่วง

โค้ดในภาพมาจาก `labs/lab6_todo/agent_todo.py` ฟังก์ชัน `run()` ใน Repository

`aekanun2020/Python-Agent-LangGraph` ที่ commit `c3f95c3` โดยตัดบรรทัดแสดงผลและรายละเอียดโดเมนออกเพื่อให้เห็น Agent Loop, Todo dispatch, MCP dispatch และ Stopping Condition ในภาพเดียว [3]

กลุ่มบรรทัดที่ 1 ใช้ `for step in range(1, max_steps + 1)` สร้างขอบเขตของงาน หากไม่มีจำนวนรอบสูงสุด โมเดลสามารถเรียก Tool ผิดแล้วลองซ้ำไปเรื่อย ๆ จนเกิด Infinite Loop หรือต้นทุนเกินควบคุม

กลุ่มที่ 2 เรียก `llm.chat(...)` พร้อม History และ Tool Definition ตรงนี้คือช่วงที่โมเดลตัดสินใจ แต่ยังไม่มี การกระทำภายนอกเกิดขึ้น ผลลัพธ์อาจเป็นข้อความธรรมดาหรือรายการ Tool Call

กลุ่มที่ 3 ตรวจสอบว่ามี Tool Call หรือไม่ ในตัวอย่างแบบย่อ หากไม่มี ระบบคืนคำตอบและจบวงจร แต่ Production Agent ควรมี Completion Verification เพิ่ม โดยเฉพาะงานที่ต้องสร้างหรือเปลี่ยนแปลงสิ่งภายนอก

กลุ่มที่ 4 แยก Tool ภายในสองตัวคือ `todo_write` และ `todo_update` ออกจาก MCP Tools อื่น ซึ่งส่งไปยัง `registry.dispatch(name, args)` ตัว Registry จึงเป็นจุดเชื่อมระหว่างชื่อ Tool ที่โมเดลเลือกกับการทำงานจริงของ MCP Tool

กลุ่มที่ 5 นำ `result` มาห่อเป็นข้อความ `role: tool` และผูกกับคำขอเดิมผ่าน `tool_call_id: call.id` นี่คือการ Observation ที่โมเดลจะอ่านในรอบถัดไป จากนั้นคำสั่ง `continue` ส่งการทำงานกลับไปเริ่ม Agent Loop รอบใหม่

กลุ่มสุดท้ายแสดง Stopping Condition ของโค้ดที่ใช้สอนจริง เมื่อชน `max_steps` ระบบเพิ่มข้อความที่สั่งห้ามเรียก Tool แล้วเรียก `llm.chat(messages=messages)` อีกครั้งเพื่อบังคับสรุปผลเชิงธุรกิจจากข้อมูลที่มี วิธีนี้ปิดงานด้วยคำตอบแทนการปล่อยให้งงจวนต่อไป

โค้ดชุดนี้จึงแสดงการแบ่งความรับผิดชอบอย่างชัดเจน: โมเดลเสนอ Tool Call ส่วน Harness ดูแล Messages, Tool Registry, Todo State, การ Dispatch, Observation และขอบเขตการหยุด

หลักการสำคัญของบทนี้จึงสรุปได้ว่า:

Agent ที่เชื่อถือได้ไม่ได้หยุดเมื่อโมเดลบอกว่าเสร็จ แต่หยุดเมื่อ Harness มีหลักฐานว่างานสำเร็จ

Ground-truth Loop

โมเดลสร้างข้อเสนอเกี่ยวกับโลก แต่โลกภายนอกต้องเป็นผู้ยืนยันผล Anthropic เน้นว่า Agent ต้องได้รับ Ground Truth จากสภาพแวดล้อมในแต่ละขั้น เพื่อประเมินความคืบหน้า และควรหยุดรับความเห็นจากมนุษย์เมื่อถึง Checkpoint หรือพบ Blocker [2] เมื่อนำมาวางบน Agent Loop จะได้ลำดับที่ตรวจสอบได้:

```
Model proposes → Harness validates → Environment executes  
→ Observation returns → Evidence is recorded  
→ State changes → Checkpoint decides
```

Tool Call จึงยังไม่ใช่ Evidence และข้อความว่า “ดำเนินการเรียบร้อยแล้ว” ก็ไม่ใช่ Ground Truth หลักฐานต้องมาจากผลการ Execute, สถานะภายนอกที่อ่านกลับได้, Receipt หรือมนุษย์ผู้มีอำนาจยืนยัน หากขั้นตอนมี Side Effect สูง Checkpoint ต้องเกิดก่อนลงมือและหลังลงมือ: ก่อนลงมือเพื่อตรวจสอบสิทธิ์และผลกระทบ หลังลงมือเพื่อยืนยันว่า External State เปลี่ยนตามที่ตั้งใจจริง

Harness Design Check

- **Harness รับผิดชอบ:** เก็บ State, Dispatch Action, ส่ง Observation กลับ และบังคับเงื่อนไขหยุด
- **Model ตัดสินใจ:** เลือกก้าวถัดไปจาก Goal, State และ Observation
- **Evidence ที่ต้องเก็บ:** Step, Tool Call, Result, State transition และ Verification
- **Failure ที่ต้องทดสอบ:** Loop, หยุดเร็ว, ทำซ้ำ และประกาศเสร็จโดยไม่มีหลักฐาน

จากจุดนี้ เราสามารถมององค์ประกอบต่าง ๆ ของหนังสือ—Tool Calling, MCP, Context, Planning, Error Recovery, Retrieval, Memory และ Evaluation—ว่าเป็นส่วนสนับสนุนวงจรเดียวกัน บทถัดไปจะขยายช่วง Act ให้เห็นว่าสิ่งที่โมเดลสร้างเป็นเพียงข้อเสนอ Tool Call และซอฟต์แวร์ต้องตรวจสอบอะไรบ้างก่อนลงมือ

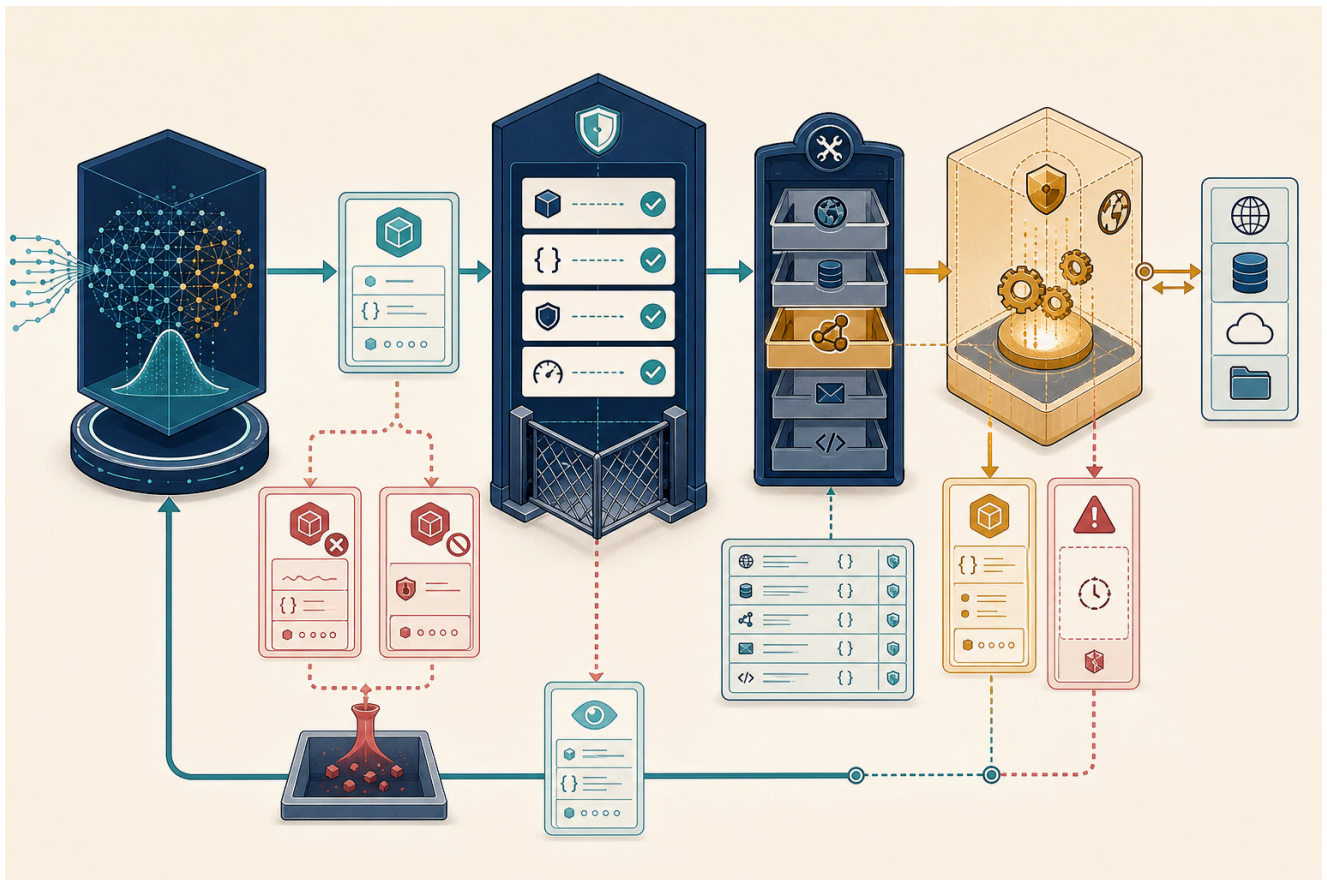
เอกสารอ้างอิง

[1] Thongtae, Aekanun. “Agent Loop, Internal Tools, and MCP Tools.” *Author’s Lecture Transcript*, July 2026.

[2] Anthropic. “Building Effective Agents.” December 2024.

[3] Thongtae, Aekanun. *Python-Agent-LangGraph*, [labs/lab6_todo/agent_todo.py](#), commit [c3f95c3](#), GitHub repository, 2026.

บทที่ 4 — Tool Calling: สัญญาระหว่างโมเดลกับซอฟต์แวร์



เส้นทางของ Tool Call ตั้งแต่ข้อเสนอของโมเดล การตรวจสอบ การทำงาน และผลลัพธ์ที่ย้อนกลับ

ภาพเปิดบทนี้เริ่มจากโมเดลทางซ้าย โมเดลไม่ได้จับฐานข้อมูล เปิดไฟล์ หรือเรียก API โดยตรง สิ่งที่มีนัยสำคัญคือข้อเสนอเชิงโครงสร้างว่าอยากใช้เครื่องมือใด ด้วยพารามิเตอร์อะไร ข้อเสนอเหล่านั้นต้องผ่าน Gate ก่อนที่ซอฟต์แวร์จริงจะยอมลงมือ

จากข้อความเป็นคำสั่งที่ประมวลผลได้

หากเราขอให้ LLM ตอบว่า “ควรเรียกฟังก์ชันอะไร” ด้วยภาษาธรรมชาติ คำตอบอาจมีรูปแบบไม่แน่นอน Tool Calling แก้ปัญหานี้ด้วยสัญญาที่ชัดเจน ระบบส่ง Tool Definition ให้โมเดล ซึ่งโดยทั่วไปประกอบด้วยชื่อ คำอธิบาย และ Schema ของพารามิเตอร์ โมเดลจึงสามารถตอบกลับเป็น Tool Call ที่ซอฟต์แวร์อ่านได้

Tool Definition ไม่ใช่โค้ดของเครื่องมือ และ Tool Call ก็ไม่ใช่ผลลัพธ์ของเครื่องมือ Definition บอกความสามารถ Call แสดงความตั้งใจ ส่วน Result คือสิ่งที่เกิดขึ้นหลัง Runtime ดำเนินการจริง การแยกสามสิ่งนี้ช่วยให้เราสับสนระหว่าง “โมเดลเลือกถูก” กับ “งานสำเร็จจริง”

คำอธิบายเครื่องมือมีผลต่อการตัดสินใจของโมเดล ชื่อที่กำกวม คำอธิบายที่ไม่บอกข้อจำกัด หรือ Schema ที่เปิดกว้างเกินไป ล้วนเพิ่มโอกาสเลือกผิด Tool ที่ไม่ควรทำหน้าที่เดียวชัดเจน รับพารามิเตอร์เท่าที่จำเป็น และคืนผลลัพธ์ที่ดีความต่อได้

Description และ Docstring คือส่วนหนึ่งของ Runtime Contract

ในการบรรยายเรื่องการสร้าง MCP Server มีการไล่ตั้งแต่การ Register Tool ไปจนถึงการเขียน Docstring ให้ครบ เพราะข้อความเหล่านี้ไม่ได้มีไว้ให้คนอ่านโค้ดเท่านั้น เมื่อระบบทำ Tool Discovery ชื่อ คำอธิบาย และ Schema จะถูกส่งเข้า Context ให้โมเดลใช้เลือก Tool และสร้าง Arguments ด้วย ดังนั้น Description ที่ตกหล่นจึงกลายเป็นความผิดพลาดขณะทำงานได้จริง แม้ฟังก์ชันข้างใต้จะเขียนถูกต้องทั้งหมด [1]

มองสัญญานี้เป็นสามชั้นจะชัดเจน:

1. **Selection contract** — ชื่อและคำอธิบายต้องบอกว่า Tool นี้ควรถูกเลือกเมื่อใด และไม่ควรถูกเลือกเมื่อใด
2. **Argument contract** — Schema ต้องบอกชนิด ฟิลด์บังคับ ขอบเขตค่า หน่วย และรูปแบบที่ยอมรับ
3. **Result contract** — ผลลัพธ์ต้องทำให้ Agent แยกความสำเร็จ ข้อมูล หลักฐาน และข้อผิดพลาดได้

ถ้าโมเดลเลือก Tool ถูกแต่ส่ง Arguments ผิดซ้ำ อย่าเริ่มแก้ที่ Prompt ใหญ่ทันที ให้ตรวจ Description กับ Schema ก่อน เช่น พารามิเตอร์ `timeout` ที่ไม่ระบุหน่วยอาจถูกตีความเป็นวินาทีหรือมิลลิวินาที หรือฟیلด์ `query` ที่ไม่บอกไวยากรณ์อาจได้ทั้งภาษาธรรมชาติและคำสั่งเฉพาะระบบ ความกำกวมเหล่านี้คือหนี้ของ Harness ไม่ใช่เรื่องลุ่มของโมเดลเพียงอย่างเดียว

จาก HCI สู่ ACI

Anthropic เรียกขอบเขตนี้ว่า Agent-Computer Interface หรือ ACI และเสนอว่าการออกแบบ Interface สำหรับ Agent ควรได้รับความใส่ใจพอ ๆ กับที่เราออกแบบ Human-Computer Interface ให้มนุษย์ [2] ความต่างคือ ACI ต้องคำนึงถึงข้อผิดพลาดของโมเดลโดยเฉพาะ เช่น ชื่อคล้ายกัน ค่า Enum ที่กำกวม การ Escape รูปแบบซับซ้อน หรือ Relative Path ที่เปลี่ยนความหมายตาม Working Directory

แนวคิด Poka-yoke จึงใช้กับ Tool ได้: เปลี่ยน Interface เพื่อทำให้ความผิดพลาดเกิดยาก แทนการเขียน Prompt เต็มซ้ำ ตัวอย่างเช่น บังคับ Absolute Path, แยก Tool สำหรับ Read และ Write, ใช้ Enum แทนข้อความอิสระ และให้ Dry-run คิน Preview ก่อน Commit การลงทุนตรง ACI มักลดทั้ง Prompt, Retry และ Error Recovery ที่ต้องตามแก้ภายหลัง

Tool ไม่ควรเป็นเพียง API Wrapper

API เดิมมักออกแบบตาม Resource ของระบบ เช่น `list_users`, `list_events` และ `create_event` แต่ Agent ต้องประกอบหลาย Call เพื่อทำงานเดียว ยังมี Call มาก จำนวนจุดที่ Arguments ผิด Result บวม หรือ State คลาดเคลื่อนก็ยิ่งเพิ่ม Tool สำหรับ Agent จึงควรออกแบบตามงานที่มีความหมาย เช่น `schedule_event` ซึ่งคนผู้เกี่ยวข้อง ตรวจเวลาว่าง และสร้างนัดหมายภายใต้ Transaction เดียว [4]

หลักนี้ไม่ได้แปลว่าต้องรวมทุกอย่างเป็น Tool ยักษ์ ขอบเขตที่เหมาะสมคือหนึ่ง Tool ควรมี Outcome ชัด มี Policy เดียวกัน และคืน Observation ที่ Agent ใช้ตัดสินใจต่อได้ หากขั้นตอนหนึ่งต้องการสิทธิ์ต่างกันหรือควรมี Human Checkpoint ควรแยก Boundary ออกมาให้เห็น

ลดความกำกวมของพื้นที่เลือก

เมื่อมี Tool หลายระบบ ชื่ออย่าง `search`, `get` หรือ `update` จะชนกันง่าย Namespacing เช่น `docs_search`, `tickets_search` และ `calendar_create_event` ช่วยให้ทั้งคนและโมเดลเห็นขอบเขตเจ้าของความสามารถ คำอธิบายยังต้องระบุความต่างจาก Tool ที่ใกล้เคียง ไม่ใช่เพียงบอกหน้าที่ของตัวเอง

Result ควรคืนข้อมูลที่มีความหมายต่อการตัดสินใจ แทน Identifier ที่อ่านไม่ออกเพียงอย่างเดียว เช่นคืน `document_title`, `matched_section` และ `source_url` พร้อม ID สำหรับอ้างอิงกลับ หาก Result อาจยาวให้มี `Pagination`, `limit` และรูปแบบ `concise | detailed` โดยค่าเริ่มต้นเลือกแบบที่กิน Context ต่ำ [4]

```
{
  "status": "ok",
  "summary": "พบเอกสารที่เกี่ยวข้อง 12 รายการ",
  "items": ["...5 รายการแรก..."],
  "next_cursor": "page_2",
  "evidence_ref": "search-run-184"
}
```

รูปแบบนี้บอกทั้ง Outcome, ขอบเขตข้อมูลที่เห็น, วิธีอ่านต่อ และ Reference สำหรับ Audit ต่างจากการคืน Array หลายพันแถวซึ่งโยนภาระการกรองทั้งหมดกลับเข้า Context

Gate สี่ขั้นก่อนการลงมือ

ส่วนกลางของภาพแสดง Gate ที่ตรวจ Tool Call อย่างน้อยสี่มิติ

1. **Availability** — เครื่องมือนี้มีอยู่และพร้อมใช้งานหรือไม่
2. **Schema validity** — ชนิดข้อมูล ฟิลด์บังคับ และรูปแบบพารามิเตอร์ถูกต้องหรือไม่
3. **Permission** — ผู้ใช้และ Agent มีสิทธิ์ทำการกระทำนั้นหรือไม่
4. **Operational policy** — การกระทำอยู่ภายในข้อจำกัดด้านต้นทุน เวลา และความเสี่ยงหรือไม่

การตรวจเหล่านี้ควรอยู่ในโค้ด ไม่ใช่หวังให้โมเดลจำข้อห้ามจาก Prompt เพียงอย่างเดียว เส้นทางสีแดงในภาพแสดง Call ที่ผิดรูปหรือไม่สิทธิ์ ซึ่งถูกปฏิเสธก่อนถึงระบบจริง การปฏิเสธที่ดีควรคืน Error แบบมีโครงสร้าง เพื่อให้ Agent รู้ว่าต้องทำอะไร แทนที่จะได้รับข้อความคลุมเครือแล้วลองเดาซ้ำ

Tool Registry และการเลือกความสามารถ

หลังผ่าน Gate ระบบต้องจับคู่ Call กับเครื่องมือใน Registry แต่การให้โมเดลเห็นเครื่องมือทุกตัวพร้อมกันไม่จำเป็นต้องเป็นทางเลือกที่ดีที่สุด เมื่อจำนวน Tool เพิ่มขึ้น ชื่อและคำอธิบายทั้งหมดจะกิน Context และเพิ่มความสับสน

Harness สามารถทำ Tool Routing ก่อน เช่น เลือกเฉพาะกลุ่มเครื่องมือที่สัมพันธ์กับงาน หรือเปิดรายละเอียดเพิ่มเติมแบบ Progressive Disclosure วิธีนี้ช่วยลดทั้ง Token และพื้นที่ที่ตัดสินใจ เครื่องมือที่ไม่เกี่ยวข้องไม่ควรปรากฏใน Context เพียงเพราะระบบมีมันอยู่

Execution Boundary

กล่องสีทองในภาพคือขอบเขตที่โค้ดทำงานจริง ตรงนี้ต้องมี Timeout, Retry Policy, Rate Limit, Logging และการจัดการ Transaction การเรียก API สำเร็จในระดับเครือข่ายไม่ได้แปลว่าภารกิจสำเร็จในระดับธุรกิจ HTTP 200 อาจคืนข้อมูลว่าง การเขียนฐานข้อมูลอาจสำเร็จเพียงบางส่วน หรือคำสั่งอาจทำงานนานจนเกินงบ

เครื่องมือจึงควรคืนผลลัพธ์ที่แยกสถานะ ข้อมูล หลักฐาน และข้อผิดพลาดออกจากกัน หากมี Side Effect ควรมี Identifier หรือ Receipt สำหรับตรวจสอบย้อนหลัง ในงานสำคัญควรออกแบบ Idempotency เพื่อป้องกันการซ้ำเมื่อ Agent Retry

Result ต้องกลับมาเป็น Observation

เส้นทางด้านล่างของภาพนำ Tool Result กลับเข้าโมเดล ผลลัพธ์นี้ไม่ควรถูกยัดกลับทั้งหมดโดยอัตโนมัติ Log หลายหมื่นบรรทัดหรือข้อมูลทั้งตารางอาจทำให้ Context บวม Harness ต้องตัด เลือก หรือสรุปโดยไม่ทิ้งหลักฐานสำคัญ

Agent อ่าน Observation แล้วตัดสินใจว่าจะตอบผู้ใช้ เรียก Tool อื่น แก่พารามิเตอร์ หรือหยุด การเรียก Tool จึงเป็นส่วนหนึ่งของวงจร ไม่ใช่ปลายทางของงาน

อ่าน Tool Calling จากโค้ดที่ใช้สอน

ภาพต่อไปมาจาก `labs/lab6_todo/agent_todo.py` โดยตรง และแสดงจุดเปลี่ยนจากคำตอบของโมเดลไปสู่การทำงานของ Harness [3]

Tool Calling จาก agent_todo.py: ความตั้งใจ → Dispatch → Observation

โคัดจริงที่ใช้สอน · `labs/lab6_todo/agent_todo.py` · `run()`

```
87 if msg.tool_calls:
88     messages.append({
89         "role": "assistant", "content": msg.content or "",
90         "tool_calls": [tc.model_dump() for tc in msg.tool_calls],
91     })
92
93 for call in msg.tool_calls:
94     args = json.loads(call.function.arguments or "{}")
95     name = call.function.name
96     if name == "todo_write":
97         result = todo.write(args.get("items", []))
98     elif name == "todo_update":
99         result = todo.update(args.get("index"), args.get("status"))
100     else:
101         result = registry.dispatch(name, args)
102     messages.append({"role": "tool",
103                     "tool_call_id": call.id, "content": result})
104
105 continue
```

1 **Tool Call คือข้อเสนอ**
เก็บคำตอบและ Tool Calls ของโมเดลลง History ก่อนทำงานจริง

2 **อ่าน Name และ Arguments**
Arguments เป็น JSON ที่โมเดลสร้างชื่อ Tool กำหนดเส้นทาง Dispatch จึงต้องไม่ถือว่าเป็นข้อมูลที่เชื่อถือได้

3 **Internal Tool หรือ MCP Tool**
Todo Tools เปลี่ยน State ในโปรเจกต์ Tool สิ้นส่งผ่าน `registry.dispatch()` ไปยัง MCP Server เจ้าของ Tool

4 **Result กลายเป็น Observation**
`role=tool` นอกขณิดของข้อความ `call.id` ผูกผลกับคำขอเดิมแล้ว `continue` เพื่อเริ่มรอบถัดไป

Production gap
ควรเพิ่ม Schema validation, authorization และ approval gate

Source: github.com/aekun2020/Python-Agent-LangGraph · commit c3f95c3

Tool Calling จากโคัดจริงใน Python-Agent-LangGraph

เมื่อ `msg.tool_calls` มีค่า โค้ดเก็บ Assistant Message พร้อม Tool Calls ลงใน `messages` ก่อน ขั้นนี้สำคัญ เพราะผลจาก Tool ที่จะตามมาต้องอ้างอิงกลับไปยังคำขอเดิม

แต่ละ Call ถูกแยกเป็น `name` และ `args` โดย Arguments มาจาก JSON ที่โมเดลสร้าง จากนั้น Harness แบ่งเส้นทางเป็นสองกลุ่ม: `todo_write` และ `todo_update` เป็น Internal Tools ที่เปลี่ยน State ในโปรเซส ส่วน Tool อื่นถูกส่งไปยัง `registry.dispatch(name, args)` เพื่อเรียก MCP Server

ผลลัพธ์จริงถูกเพิ่มกลับเข้า History ด้วย `role: "tool"` และ `tool_call_id: call.id` แล้ว `continue` พากการทำงานกลับไปเริ่มรอบใหม่ นี่คือการแปลง Tool Result ให้เป็น Observation ที่โมเดลใช้ตัดสินใจต่อ

โค้ดนี้เหมาะกับการสอนแกน Tool Calling แต่ยังไม่ใช้ Production Security Layer ในตัวมันเอง `json.loads()` ทำหน้าที่ Parse ไม่ใช้ตรวจสอบความปลอดภัย และ `registry.dispatch()` ในฉบับนี้ยังไม่มี Authorization หรือ Human Approval ดังนั้นระบบจริงควรเพิ่ม Schema Validation, Permission Check และ Confirmation Gate ก่อนการกระทำที่มีผลสูง

Tool ที่ดีต้องทดสอบได้โดยไม่ใช้ LLM

ก่อนผูกเครื่องมือกับ Agent เราควรทดสอบ Tool แบบซอฟต์แวร์ปกติ เรียกด้วยพารามิเตอร์ถูก ผิด ขาด เกิน และกรณีขอบ ทดสอบ Permission, Timeout และ Error Contract หากเครื่องมือไม่น่าเชื่อถือเมื่อเรียกตรง การเพิ่ม LLM เข้าไปจะทำให้สาเหตุของความล้มเหลวยิ่งวิเคราะห์ยาก

หลังผ่าน Unit Test จึงทดสอบ ACI ด้วย Task จริง โดยเก็บว่าโมเดลเลือก Tool ถูกหรือไม่ สร้าง Arguments ผ่านครั้งแรกที่เปอร์เซ็นต์ เรียกซ้ำโดยไม่ก้าวหน้าหรือไม่ และ Result มีข้อมูลพอให้มนุษย์ตอบโต้ได้หรือไม่ Failure แต่ละแบบควรย้อนกลับไปแก้ Contract คนละส่วน:

Failure	จุดที่ควรตรวจ
เลือก Tool ผิด	Name, Description, overlap และ Tool subset
Arguments ผิดซ้ำ	Parameter name, type, unit, enum และ example
เรียกหลายครั้งเกินจำเป็น	Tool boundary และ workflow-level operation
Result ใหญ่เกิน Context	Projection, pagination และ response format
Agent ไปต่อไม่ได้หลัง Error	Error code, retryable และ actionable hint
Write ซ้ำหลัง Retry	Idempotency key และ receipt

Availability กับ Effectiveness

คำบรรยายแยกการทดสอบ Tool เป็นสองคำถาม Availability ถามว่า Tool มองเห็นและเรียกได้หรือไม่ ส่วน Effectiveness ถามว่าผลที่คืนมาดีพอสำหรับทำงานต่อหรือไม่ วิธีตรวจเบื้องต้นคือวาง Tool Result ให้มนุษย์อ่านแล้วถามว่า “ข้อมูลเท่านี้ตอบโต้ได้หรือยัง” หากมนุษย์ยังตอบไม่ได้ LLM ก็ไม่ควรถูกคาดหวังให้เติมส่วนที่หายไปเอง

อีก Failure Layer คือ Parsing: `json.loads()` อาจอ่าน JSON ได้แต่ค่าไม่ตรง Schema หรือบาง Inference Engine อาจสร้าง Payload ไม่สมบูรณ์ จึงควร Validate Arguments หลัง Parse เสมอ หาก Agent เรียก Tool เดิมซ้ำโดยไม่ก้าวหน้า ต้องตรวจทั้ง Description, Argument Contract และคุณภาพ Result ไม่ควรรับสรุปว่า โมเดลไม่เก่ง

หลักสำคัญของบทนี้คือ:

โมเดลมีสิทธิ์เสนอการกระทำ แต่ Harness เป็นผู้ตัดสินใจว่าการกระทำนั้นจะเกิดขึ้นจริงหรือไม่

Harness Design Check

- **Harness รับผิดชอบ:** เลือก Tool, Validate Arguments/Permission, Execute และจัดรูป Result
- **Model ตัดสินใจ:** เสนอชื่อ Tool และ Arguments ตาม Schema
- **Evidence ที่ต้องเก็บ:** Tool version, Raw/Normalized arguments, Authorization, Result และ Side Effect
- **Failure ที่ต้องทดสอบ:** Invalid arguments, ผิดลำดับ, ไม่มีสิทธิ์, Duplicate write และ Result ใหญ่เกิน Context

เมื่อมีเครื่องมือจำนวนมากหรือหลาย Agent ต้องใช้ความสามารถเดียวกัน การเขียน Integration แยกทุกคู่จะกลายเป็นภาระ บทดัดไปจึงพาไปดู MCP ซึ่งสร้างภาษากลางสำหรับการค้นพบ เรียกใช้ และรับผลจากเครื่องมือ

เอกสารอ้างอิง

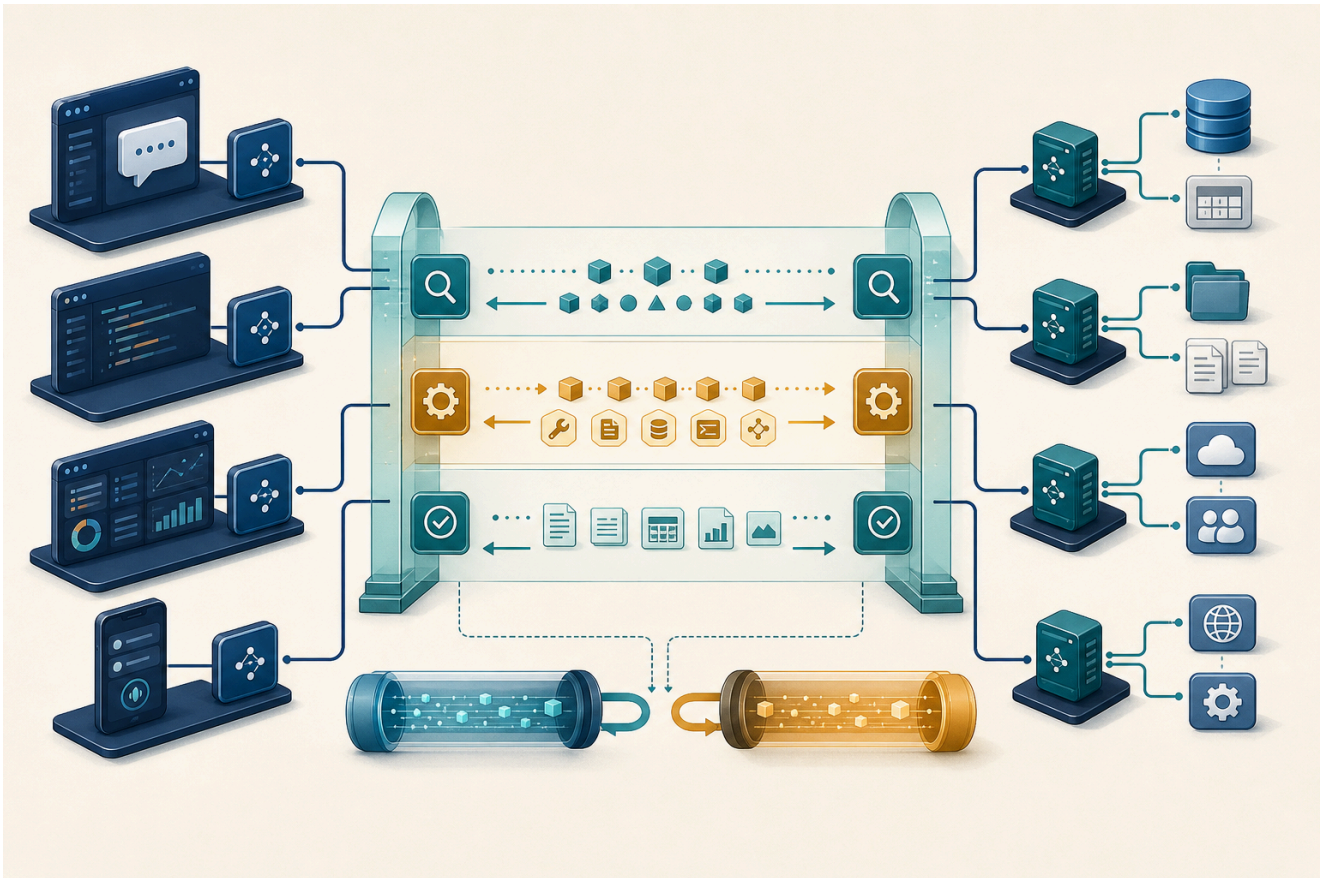
[1] Thongtae, Aekanun. “MCP Tool Registration, Description, Schema, and Error Handling.” *Author’s Lecture Transcript*, August 2026.

[2] Anthropic. “Building Effective Agents: Prompt Engineering Your Tools.” December 2024.

[3] Thongtae, Aekanun. *Python-Agent-LangGraph*, `labs/lab6_todo/agent_todo.py`, commit `c3f95c3`, GitHub repository, 2026.

[4] Anthropic. “Writing Effective Tools for AI Agents.” September 2025.

บทที่ 5 — MCP: ภาษากลางระหว่าง Agent กับ เครื่องมือ



สถาปัตยกรรม many-to-many ของ Host, Client, Protocol, Server และระบบข้อมูลภายนอก

ภาพเปิดบทนี้มีแอปพลิเคชันหลายชนิดอยู่ทางซ้าย และระบบข้อมูลหลายชนิดอยู่ทางขวา สะพานตรงกลางทำให้ทั้งสองฝั่งสื่อสารกันด้วยรูปแบบเดียว นี่ก็คือคุณค่าหลักของ Model Context Protocol หรือ MCP

ปัญหา Integration แบบจับคู่

ก่อนมีมาตรฐาน แอปพลิเคชันแต่ละตัวต้องเขียน Integration สำหรับบริการแต่ละตัว หากมี Agent หลายชนิดและแหล่งข้อมูลหลายระบบ จำนวนการเชื่อมต่อจะเพิ่มขึ้นอย่างรวดเร็ว โค้ดฝั่ง Agent ต้องรู้รายละเอียด Authentication, Endpoint, Payload และ Error ของทุกบริการ

MCP เปลี่ยนความสัมพันธ์จากการเชื่อมแบบคู่ต่อคู่เป็นสถาปัตยกรรม many-to-many ฝั่ง Host ใช้ Client ที่เข้าใจ Protocol ส่วนผู้ให้ความสามารถสร้าง MCP Server ซึ่งห่อหุ้มรายละเอียดของระบบปลายทางไว้ด้านหลัง เมื่อทั้งสองฝั่งใช้สัญญาเดียวกัน Host สามารถพบและเรียกความสามารถใหม่โดยไม่ต้องนำโค้ดภายในของ Server มาผูกไว้กับ Agent

แยก Host, Client และ Server ให้ชัด

Host คือแอปพลิเคชันที่ผู้ใช้หรือ Agent ทำงานอยู่ เช่น Desktop, IDE, ระบบ Workflow หรือบริการ Agent ฟังก์ชัน Host ดูแลโมเดล การสนทนา Permission ระดับผู้ใช้ และประสบการณ์ใช้งาน

Client คือส่วนเชื่อมต่อ MCP ภายใน Host มันเปิด Session กับ Server ขอรายการความสามารถ ส่ง Request และรับ Result Host หนึ่งตัวอาจมี Client หลายตัวเพื่อเชื่อมหลาย Server

Server ประกาศความสามารถและเชื่อมกับระบบจริง เช่น Database, Filesystem, API หรือบริการองค์กร Server ไม่จำเป็นต้องมี LLM อยู่ภายใน และ MCP เองก็ไม่ใช้โมเดลหรือฐานข้อมูล มันเป็น Protocol ที่จัดการการสื่อสาร

Discovery มาก่อน Invocation

แถบบนของสะพานในภาพสื่อถึง Discovery Client ขอให้ Server บอกรายการความสามารถและรายละเอียดที่จำเป็น โมเดลจึงไม่ต้องถูก Hard-code ว่าระบบปลายทางมี Tool อะไรอยู่ตลอดไป

การค้นพบนี้ทำให้เครื่องมือมีลักษณะ Dynamic มากขึ้น แต่ไม่ได้แปลว่าควรส่งทุก Tool เข้าสู่โมเดลทันที Host ยังต้องทำ Filtering ตามสิทธิ์ งาน และงบ Context จากนั้นจึงนำ Tool Definition ที่เกี่ยวข้องไปประกอบ Prompt

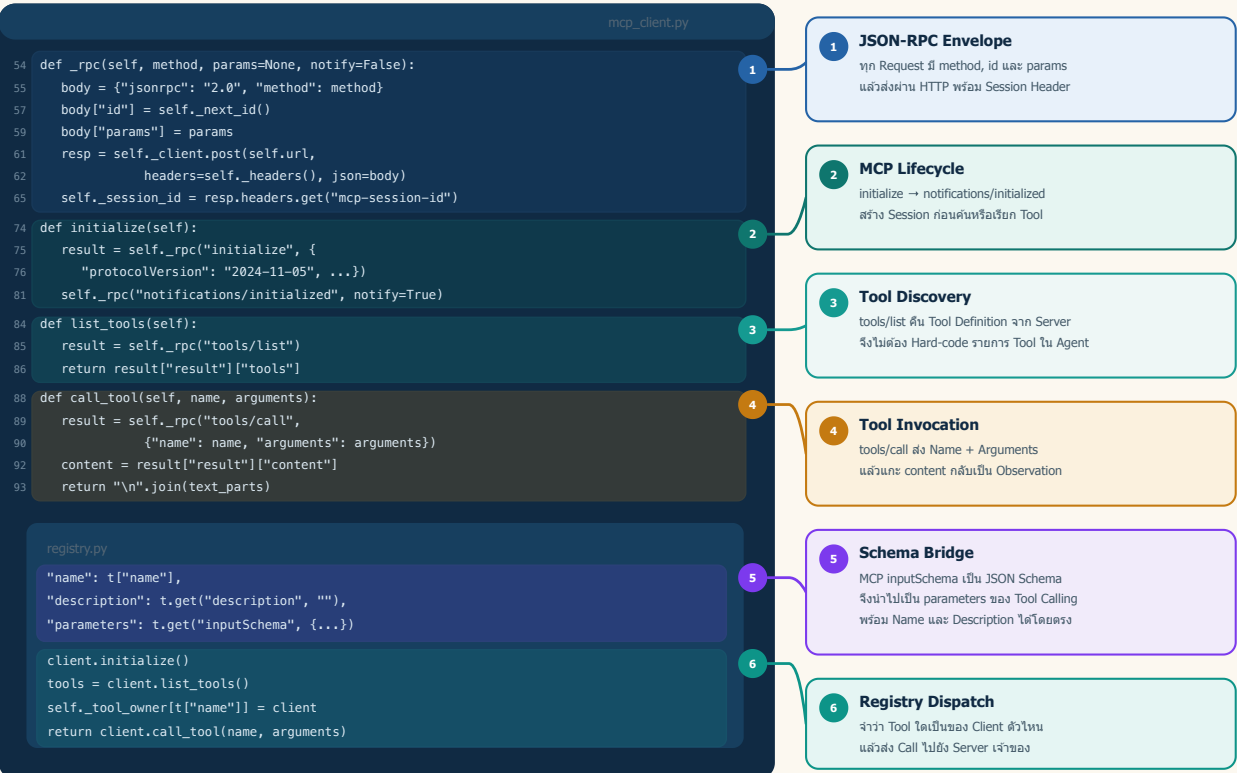
แถวล่างคือ Invocation เมื่อโมเดลเสนอ Tool Call Host ตรวจสอบแล้วส่งคำขอผ่าน Client ไปยัง Server Server เรียก Logic หรือระบบปลายทางจริง และคืนผลที่มีโครงสร้างกลับมา แถวล่างคือ Result และหลักฐานที่ย้อนกลับสู่ Agent Loop

อ่าน MCP Lifecycle จาก Pure Python

ภาพต่อไปประกอบจาก `labs/core/mcp_client.py` และ `labs/core/registry.py` ใน Repository ที่ใช้สอน จึงทำให้สถาปัตยกรรมในภาพเปิดบทย่อยเชื่อมกับบรรทัดที่ทำงานจริงได้ [2]

MCP จากโค้ดจริง: Lifecycle, Discovery, Schema Conversion และ Dispatch

core/mcp_client.py + core/registry.py · Pure Python ผ่าน Streamable HTTP



Source: github.com/aekun2020/Python-Agent-LangGraph · commit c3f95c3 · labs/core/mcp_client.py + registry.py

MCP Lifecycle, Schema Conversion และ Registry Dispatch จากโค้ดจริง

เมธอด `_rpc()` สร้าง JSON-RPC Envelope ที่มี `method`, `id` และ `params` ก่อนส่งผ่าน HTTP เมื่อ Server คืน `mcp-session-id` Client จะเก็บไว้และแนบไปกับ Request ถัดไป ทำให้หลาย Request อยู่ใน Session เดียวกัน

Lifecycle เริ่มจาก `initialize` แล้วส่ง `notifications/initialized` เพื่อแจ้งว่า Client พร้อม หลังจากนั้น `tools/list` จึงใช้ค้น Tool Definition และ `tools/call` ใช้ส่งชื่อพร้อม Arguments ไปเรียก Tool จริง

Schema Bridge: แปลง “สัญญาของ Tool” ระหว่างสองมาตรฐาน

`mcp_to_openai_tools()` ใน Registry ไม่ได้มีหน้าที่เพียงเปลี่ยนรูป JSON ให้สวยงาม แต่เป็น Adapter ระหว่างมาตรฐาน MCP กับรูปแบบ Function Tool ที่ LLM API คาดหวัง หลังจาก `tools/list` ส่ง Tool Definition กลับมา Registry จะนำข้อมูลสามส่วนมาประกอบเป็นสัญญาที่โมเดลอ่านได้

- `name` บอกว่า Tool ชื่ออะไร และเป็นชื่อที่โมเดลต้องส่งกลับมาเมื่อเลือกใช้ Tool
- `description` บอกว่า Tool ทำอะไร ควรใช้เมื่อใด และไม่ควรใช้เมื่อใด ส่วนนี้มีผลต่อการที่โมเดลจะเลือก Tool ถูกตัวหรือไม่
- `inputSchema` บอกโครงสร้างของ Arguments เช่น ชื่อ Field, Data Type, Field ที่บังคับ, ค่า `enum` และ ข้อจำกัดของค่า ส่วนนี้มีผลต่อการที่โมเดลจะสร้าง Arguments ได้ถูกต้อง

จุดที่ต้องสังเกตคือคำเรียกในสองฝั่งไม่เหมือนกัน ฝั่ง MCP ส่งสัญญาณรับข้อมูลมาใน Key ชื่อ `inputSchema` แต่รูปแบบ OpenAI-compatible Function Calling วาง JSON Schema เดียวกันไว้ได้ `function.parameters` ดังนั้นบรรทัด

```
"parameters": t.get("inputSchema", {"type": "object", "properties": {}})
```

จึงเป็นจุดแปลงสัญญาณที่สำคัญ: ไม่ได้ให้ LLM อ่าน MCP โดยตรง แต่ทำให้ Tool Definition ของ MCP อยู่ในรูปที่ API ของโมเดลเข้าใจเสียก่อน ถ้า Adapter ไม่ทำขั้นตอนนี้ หรือแมปชื่อ Key ผิด แม้ Server จะประกาศ Tool ถูกต้อง โมเดลก็อาจมองไม่เห็นโครงสร้าง Arguments และเรียก Tool ไม่ได้ โดยเฉพาะระบบ OpenAI-compatible หรือ vLLM ที่ความเข้ากันได้ของ Adapter อาจต่างกันในระยะยาว

ในขั้นเรียนผมเปรียบเทียบ `name` กับชื่อบนฉลากเครื่องมือ และ `description` กับข้อความที่บอกว่าเครื่องมือชิ้นนั้นใช้ทำอะไร แต่ฉลากที่ดีไม่ควรหยุดแค่คำอธิบายสั้น ๆ Docstring ซึ่งถูกนำมาสร้าง Tool Description ควรบอก Purpose, When to use, When not to use, ความหมายและชนิดของ Argument, ช่วงค่าที่รับได้ ตลอดจนรูปแบบของผลลัพธ์ หากไม่บอกขอบเขตให้ชัด LLM อาจเลือก Tool ผิดตัวแต่ “ยิง Argument สะเปะสะปะ” ได้

อย่างไรก็ตาม Schema เป็นเพียงสัญญาณที่ช่วยให้โมเดลสร้างคำขอ ไม่ใช่หลักประกันว่าค่าที่โมเดลสร้างจะถูกต้องเสมอ Harness ยังต้องตรวจสอบ Arguments ก่อน Dispatch และต้องส่ง Error หรือ Observation ที่มีความหมายกลับเข้า Agent Loop เพื่อให้โมเดลรู้ว่าอะไรผิดและแก้การเรียกครั้งถัดไปได้

เมื่อ Registry เชื่อม Server มันเก็บความสัมพันธ์ `tool_name` → `MCPClient` ไว้ใน `_tool_owner` ตอน Agent เรียก `dispatch(name, arguments)` Registry จึงรู้ว่าต้องส่งคำขอไปยัง Client และ Server ตัวใด นี่คือเหตุผลที่ Agent สามารถค้นพบ Tool แบบ Dynamic โดยไม่ Hard-code รายการทั้งหมดไว้ใน Agent Loop

อ่าน Log ตาม Phase ไม่ใช่อ่านทีละบรรทัด

คำบรรยายแบ่ง Log ของ MCP เป็นสามช่วงเพื่อช่วย Debug [1]:

1. **Initialization** — Client ส่ง `initialize`, Server ตอบ Capability และ Client ยืนยัน `notifications/initialized`
2. **Operation** — `tools/list`, `tools/call` และ Tool Result ซึ่งมักกินพื้นที่มากกว่า 90% ของ Log
3. **Shutdown** — ปิด Session และทรัพยากรที่เกี่ยวข้อง

หาก Initialization ไม่จบ ปัญหาอาจอยู่ที่ Protocol Version หรือ Capability Compatibility และยังไม่ควรกระโดดไปแก้ Tool Function หาก `tools/list` ผ่านแต่ `tools/call` ล้ม จึงค่อยตรวจ Schema, Arguments และ Execution การอ่านตาม Phase ลดพื้นที่ค้นหาสาเหตุได้เร็วกว่าการไล่ Error String โดยไม่มีแบบจำลอง Lifecycle

Tools, Resources และ Prompts

MCP ไม่ได้มีเฉพาะ Tools เท่านั้น ความสามารถสามารถถูกนำเสนอในรูปแบบต่างกันตามธรรมชาติของข้อมูล

- **Tools** เหมาะกับการกระทำหรือการคำนวณที่รับพารามิเตอร์

- **Resources** เหมาะกับข้อมูลที่สามารถระบุและอ่านได้
- **Prompts** ช่วยเผยแพร่แม่แบบการทำงานหรือบริบทที่ Server เตรียมไว้

จาก Function สู่ MCP Tool

ในโค้ด Server ที่ใช้สอน Decorator เช่น `@mcp.tool` ทำหน้าที่ Register Function ธรรมดาให้ Framework นำไปประกาศผ่าน `tools/list` ชื่อ Function กลายเป็น Tool Name, Docstring กลายเป็น Description และ Type/Argument กลายเป็น Input Schema หากไม่ Register ต่อให้ Function ทำงานได้ Agent ก็ไม่สามารถค้นพบผ่าน MCP

`tools/list` และ `tools/call` จึงตอบคนละคำถาม: List บอกว่ามีความสามารถอะไรและต้องส่งข้อมูลรูปแบบใด ส่วน Call ระบุชื่อ Tool กับ Arguments เพื่อให้ Server Execute จึง Tool Result ที่ดีควรเป็น JSON แยก Field สำคัญ เช่น `data`, `source`, `score`, `warning` และ `error` แทนข้อความยาวก๊อปปี้ เพราะ Harness สามารถตรวจ ดัด และอ้างแหล่งข้อมูลได้ง่ายกว่า

การเลือกรูปแบบที่เหมาะสมช่วยให้ขอบเขตชัดเจน ตัวอย่างเช่น เอกสารอ้างอิงที่อ่านอย่างเดียวไม่จำเป็นต้องแก้งทำเป็น Tool ที่มี Side Effect ในทางกลับกัน การอนุมัติธุรกรรมไม่ควรถูกนำเสนอเป็น Resource ธรรมดา

Transport เปลี่ยนได้ แต่สัญญายังคงเดิม

ทั้งสองชนิดด้านล่างภาพแทนวิธีขนส่งที่ต่างกัน การเชื่อมแบบ Local Process เหมาะกับ Server ที่รันบนเครื่องเดียวกับ Host ขณะที่การเชื่อมผ่านเครือข่ายเหมาะกับบริการส่วนกลางหรือระบบหลายผู้ใช้

Transport มีผลต่อ Deployment, Authentication, Latency และ Security แต่ไม่ควรเปลี่ยนความหมายของ Tool การแยก Protocol ออกจาก Transport ทำให้เราพัฒนา Logic เดิมแล้วเลือกวิธีติดตั้งให้เหมาะกับบริบทได้

MCP ลด Coupling แต่ไม่ได้ลบความเสี่ยง

มาตรฐานช่วยให้เชื่อมต่อสะดวกขึ้น แต่ความสะดวกไม่ได้รับประกันความปลอดภัย Host ต้องตัดสินใจว่าจะเชื่อม Server ใด เชื่อถือคำอธิบาย Tool ไหน และอนุญาตการกระทำระดับใด Server ต้องตรวจ Authentication, Authorization และ Validation ซ้ำที่ฝั่งตนเอง ไม่ควรเชื่อว่า Request ปลอดภัยเพียงเพราะมาจาก MCP Client

คำอธิบาย Tool และข้อมูลจาก Server ยังอาจเข้าสู่ Context ของโมเดล จึงต้องคำนึงถึง Prompt Injection และข้อมูลที่ไม่ควรถูกเปิดเผย ระบบที่ดีแยก Trust Boundary บันทึก Tool Call และให้มนุษย์เห็นผลกระทบก่อนอนุมัติงานความเสี่ยงสูง

คุณค่าทางสถาปัตยกรรม

MCP ทำให้ทีมที่ดูแลความรู้หรือระบบธุรกิจสามารถเปิดความสามารถให้ Agent หลายชนิดเรียกใช้ผ่านสัญญาเดียว ในอีกฝั่ง ทีม Agent สามารถเปลี่ยน Framework หรือโมเดลโดยไม่ต้องเขียน Integration ใหม่ทั้งหมด

หลักสำคัญของบทนี้คือ:

MCP ทำให้ Agent ค้นพบและเรียกความสามารถได้อย่างเป็นมาตรฐาน แต่ Harness ยังต้องเลือก ควบคุม และตรวจสอบการใช้งานนั้น

Harness Design Check

- **Harness รับผิดชอบ:** Lifecycle, Discovery, Schema conversion, Session และ Dispatch ไปยังเจ้าของ Tool
- **Model ตัดสินใจ:** เลือกความสามารถจาก Tool Definition ที่ถูกเปิดให้เห็น
- **Evidence ที่ต้องเก็บ:** Server/Session, Discovered schema, Request ID, Tool owner และ Response
- **Failure ที่ต้องทดสอบ:** Schema เปลี่ยน, Session ขาด, Tool ชนชื่อ, Server unavailable และ Result ผิด Contract

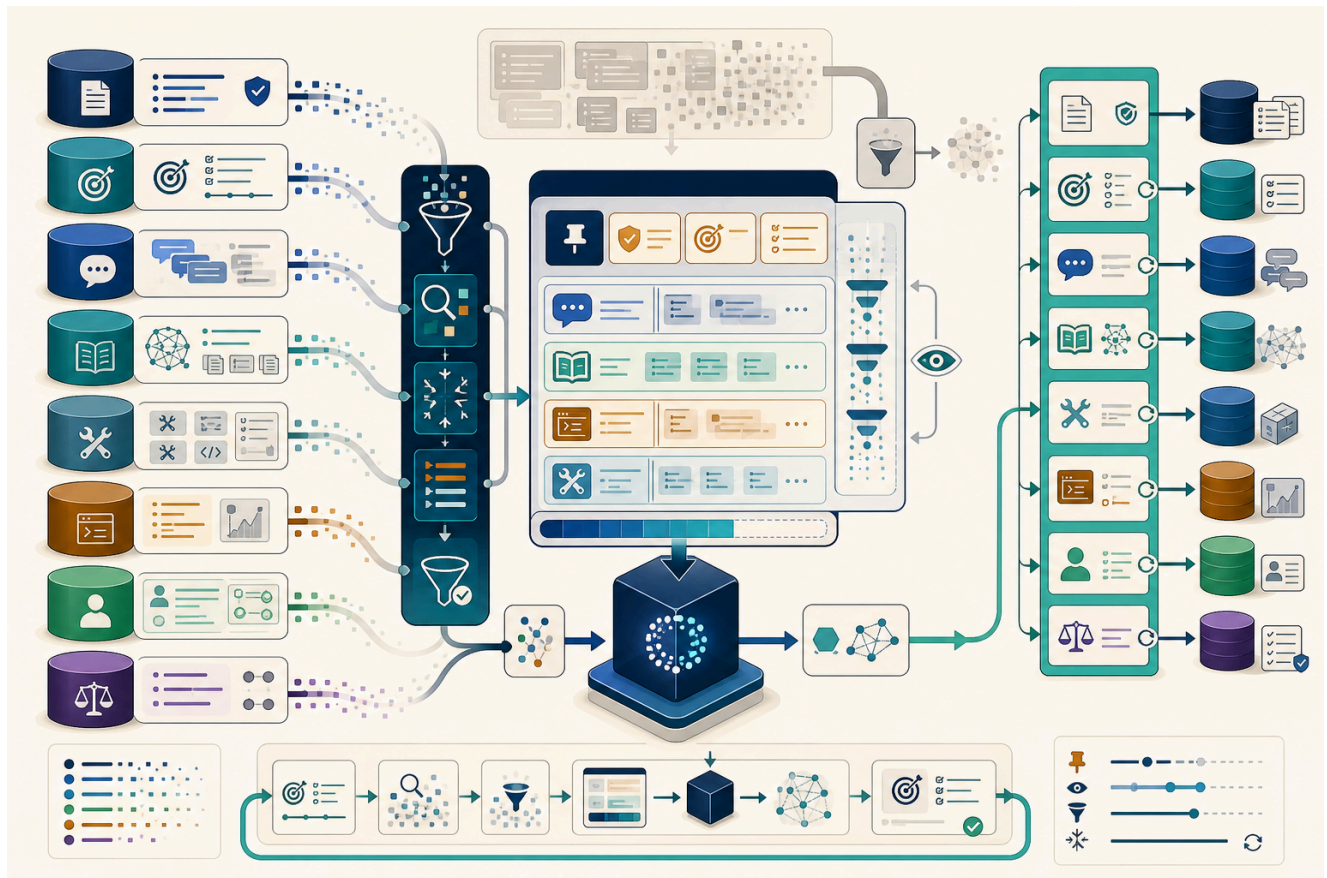
เมื่อ Agent มีเครื่องมือและแหล่งข้อมูลมากขึ้น ปัญหาถัดไปคือไม่สามารถนำข้อมูลทั้งหมดใส่ใน Context พร้อมกันได้ บทดัดไปจึงเปลี่ยนจากการเชื่อมต่อระบบ ไปสู่การเลือกข้อมูลที่โมเดลควรเห็นในแต่ละช่วงเวลา

เอกสารอ้างอิง

[1] Thongtae, Aekanun. “MCP Lifecycle, Tool Discovery, and Log Analysis.” *Author’s Lecture Transcript*, August 2026.

[2] Thongtae, Aekanun. *Python-Agent-LangGraph*, [labs/core/mcp_client.py](#) and [labs/core/registry.py](#), commit [c3f95c3](#), GitHub repository, 2026.

บทที่ 6 — Context Engineering: ให้ข้อมูลที่ ใช้ในเวลาที่ใช่



แหล่งบริบทจำนวนมากถูกคัดเลือก จัดลำดับ ป้อน และประกอบเป็น Working Context ที่มีขอบเขต

ด้านซ้ายของภาพมีแหล่งข้อมูลหลายประเภท แต่มีเพียงบางส่วนถูกส่งผ่านตัวกรองเข้าสู่หน้าต่างตรงกลาง ภาพนี้สรุปหลักของ Context Engineering: เป้าหมายไม่ใช่ใส่ข้อมูลให้มากที่สุด แต่คือประกอบข้อมูลชุดเล็กที่สุดที่เพียงพอต่อการตัดสินใจในปัจจุบัน

Complete Prompt มีมากกว่าคำสั่ง

สิ่งที่โมเดลได้รับจริงอาจประกอบด้วย System Instruction, เป้าหมายปัจจุบัน, Conversation History, Tool Definition, Retrieved Knowledge, Memory, Tool Result และข้อจำกัดด้านนโยบาย ทุกส่วนแย่งพื้นที่ใน Context Window เดียวกัน

คำบรรยายจัด Context ที่เติบโตระหว่างการทำงานออกเป็นสิ่งที่ควรแยกให้ออก: **Instruction/System Prompt, Tool Definition, Received Data จาก Tool และ Message History** ก่อนแรกกำหนดวิธีทำงาน ก่อนที่สองเปิดพื้นที่การกระทำ ก่อนที่สามนำหลักฐานจากโลกจริงเข้ามา และก่อนที่สี่ทำให้โมเดล Stateless ดูเหมือนจำ Turn ก่อนหน้าได้ แต่ละก่อนมีเจ้าของ อายุ และนโยบายตัดทอนต่างกัน การเรียกรวมทั้งหมดว่า “Prompt” ทำให้ทีมมองไม่เห็นว่าการแก้ปัญหาที่ก่อนไหน [1]

Prompt Engineering มักเน้นการเขียนคำสั่งให้ชัด Context Engineering ขยายขอบเขตไปถึงการตัดสินใจว่า ข้อมูลใดควรเข้ามา เมื่อใด อยู่ในรูปใด และควรถูกนำออกเมื่อหมดประโยชน์ มันจึงเป็นงาน Runtime มากกว่างาน เขียนข้อความครั้งเดียว

Right Altitude: ควบคุมโดยไม่เขียนทุกก้าวแทน Agent

คำบรรยายแบ่ง System Instruction จากระดับสูงไปต่ำ ระดับสูงคือบทบาทและหลักการ ระดับกลางคือ Workflow หรือ Sequence of Tool Use และระดับต่ำคือเงื่อนไขเฉพาะกับรูปแบบผลลัพธ์ Prompt ที่สูงเกินไป เช่น “ช่วยผู้ใช้ให้ดีที่สุด” ชี้นำไม่พอ ส่วน Prompt ที่ต่ำเกินไปจน Hard-code ทุกกรณีจะเปราะเมื่อโจทย์เปลี่ยน Right Altitude คือการกำหนดเกณฑ์และข้อยกเว้นชัดพอ แต่ยังเปิดให้ Agent เลือกวิธีตามหลักฐาน

Context ไม่ใช่ Memory

ถึงข้อมูลด้านซ้ายและขวาของภาพอยู่นอกหน้าต่างโมเดล นี่คือการแตกต่างสำคัญ Context คือข้อมูลที่โมเดลเห็น ในการเรียกปัจจุบัน ส่วน Memory คือข้อมูลที่เก็บไว้ภายนอกและอาจถูกดึงกลับมาในอนาคต

การเก็บทุกอย่างไว้ใน History ไม่ใช่ Memory Design ที่ดี เพราะ History โตขึ้นทุกครั้งและพาข้อมูลเก่ากลับเข้ามาโดยไม่เลือก Memory ที่ดีมีโครงสร้าง แหล่งที่มา เวลา ความสำคัญ และกติกการเรียกคืน เราจึงสามารถเก็บ ข้อมูลจำนวนมากโดยไม่ต้องจ่าย Context ให้ทั้งหมดทุกครั้ง

ปัญหาของ Context ที่มากเกินไป

เมื่อ Context เกินขีดจำกัด ระบบอาจตัดข้อมูลหรือเกิดข้อผิดพลาด แต่ปัญหาเริ่มก่อนถึงจุดนั้น Context Bloat คือภาวะที่บริบทยังใส่ได้แต่มีข้อมูลมากเกินไปจนความจำเป็น ต้นทุนและ Latency เพิ่มขึ้น ขณะที่สัญญาณสำคัญถูก ข้อมูลอื่นรบกวน

Context Rot อธิบายการเสื่อมคุณภาพเมื่อ Context เติบโตและมีสิ่งค้างเก่า ข้อมูลที่อยู่กลางลำดับอาจถูกใช้น้อย กว่าส่วนต้นหรือท้าย ปรากฏการณ์นี้มักเรียกว่า Lost in the Middle การเพิ่มข้อความอีกย่อหน้าจึงไม่ได้รับประกัน ว่าโมเดลจะไขมันได้ดีขึ้น

กฎเชิงปฏิบัติคืออย่าวางแผนใช้ Context Window จนเต็ม ควรเหลือพื้นที่สำหรับ Tool Result การสร้างคำตอบ และเหตุการณ์ที่ไม่คาดคิด พร้อมติดตาม Token จริงจาก Trace แทนการเดา

คำบรรยายอธิบายอาการนี้ว่าเป็นการ “จำเพี้ยน” มากกว่า “จำไม่ได้” เพราะข้อมูลยังอยู่ในตัวแปร แต่สัญญาณถูก เจือจางจนโมเดลตีความผิด การแก้จึงต้องตรงกับสาเหตุ: ใช้ Pruning เมื่อมีข้อมูลที่หมดความเกี่ยวข้อง, ใช้ Just-in-Time Retrieval เมื่อข้อมูลยังไม่จำเป็นต่อรอบนี้, ใช้ Isolation/Pinned State เมื่อข้อเท็จจริงห้ามหาย และใช้ Compaction เมื่อ History เก่าต้องคงความหมายแต่ไม่จำเป็นต้องคงถ้อยคำเต็ม

Decision Guide: ตัด ดึง ตรึง หรือย่อ

- ถ้าข้อมูลไม่เกี่ยวกับ Goal ปัจจุบัน — **ตัด**
- ถ้ายังไม่ต้องใช้แต่ค้นคืนได้ — **ดึงเมื่อจำเป็น**
- ถ้าเป็น Constraint, Approval หรือ Identifier สำคัญ — **ตรึงเป็น Structured State**

- ถ้าต้องรักษาความต่อเนื่องแต่รายละเอียดเกิน Budget — **ย่อพร้อม Source Pointer**

การใช้ Compaction กับทุกอย่างเป็น Default อาจทำให้ State สำคัญกลายเป็นข้อความสรุปที่คลุมเครือ ส่วนการ Pin ทุกอย่างก็สร้าง Context Bloat รูปแบบใหม่ Harness จึงต้องเลือกกลยุทธ์เป็นรายชนิดข้อมูล

ขั้นตอนประกอบ Working Context

ตัวกรองแนวดิ่งในภาพแสดงกระบวนการหลายขั้น

1. **Route** — ระบุว่างานนี้ต้องใช้ข้อมูลและเครื่องมือกลุ่มใด
2. **Retrieve** — ดึงเฉพาะรายการที่สัมพันธ์กับเป้าหมาย
3. **Rank** — จัดลำดับตามความเกี่ยวข้อง ความน่าเชื่อถือ และความใหม่
4. **Prune** — ตัดข้อมูลซ้ำ Error ที่ไม่สำคัญ และรายละเอียดที่ไม่ส่งผลต่อการตัดสินใจ
5. **Compress** — สรุปส่วนยาวโดยรักษาข้อเท็จจริง ข้อจำกัด และสถานะสำคัญ
6. **Assemble** — วางแต่ละส่วนในลำดับและรูปแบบที่โมเดลใช้ได้

Pipeline นี้ควรสังเกตได้ เราต้องย้อนกลับไปได้ว่าเอกสารใดถูกดึง เหตุใดขั้นหนึ่งถูกตัด และสรุปส่วนใดหายไป หากคำตอบผิด การตรวจเฉพาะ Final Output ไม่พอ ต้องตรวจ Context ที่สร้างคำตอบนั้นด้วย

Progressive Disclosure

ไม่จำเป็นต้องเปิดรายละเอียดทั้งหมดตั้งแต่เริ่ม เราสามารถส่ง Metadata หรือสารบัญก่อน แล้วให้ Agent ขอเนื้อหาเฉพาะส่วนเมื่อจำเป็น หลัก Progressive Disclosure ลด Context เริ่มต้นและทำให้โมเดลตัดสินใจจากภาพรวมก่อนลงรายละเอียด

แนวคิดเดียวกันใช้กับ Skills และ Tools ได้ โมเดลเห็นชื่อและคำอธิบายสั้นก่อน เมื่อเลือกแล้วจึงโหลด Instruction, Schema หรือ Reference ที่เกี่ยวข้อง วิธีนี้ทำให้ระบบขยายความสามารถได้โดยไม่ทำให้ทุกความสามารถอยู่ใน Prompt ตลอดเวลา

Compaction และการรักษา State

เมื่อ Agent ทำงานหลายรอบ History จะโตขึ้น Compaction ใช้การสรุปบทสนทนาและผลการทำงานเพื่อสร้าง State ที่กระชับ แต่การสรุปมีความเสี่ยงที่จะทำข้อมูลสำคัญหาย จึงควรแยกข้อมูลบางชนิดออกจากข้อความสรุป

ข้อกำหนดที่ต้องไม่เปลี่ยน ตัวระบุไฟล์ ผลทดสอบล่าสุด การอนุมัติ และรายการงานค้าง ควรถูกเก็บเป็น Structured State หรือ Pinned Facts ส่วนบทสนทนาทั้งหมดประโยชน์สามารถย่อหรือลบได้ Separate Note-Taking ช่วยเก็บหลักฐานและข้อค้นพบโดยไม่บังคับให้มันอยู่ใน Context ทุกครั้ง

Just-in-Time Context

ด้านขวาของภาพแสดงผลลัพธ์จากโมเดลกลับไปอัปเดตแหล่งข้อมูลภายนอก แทนที่จะสะสมทุกอย่างในหน้าต่างเดียว เมื่อรอบใหม่เริ่ม Harness ดึง State และหลักฐานที่จำเป็นกลับมาแบบ Just in Time

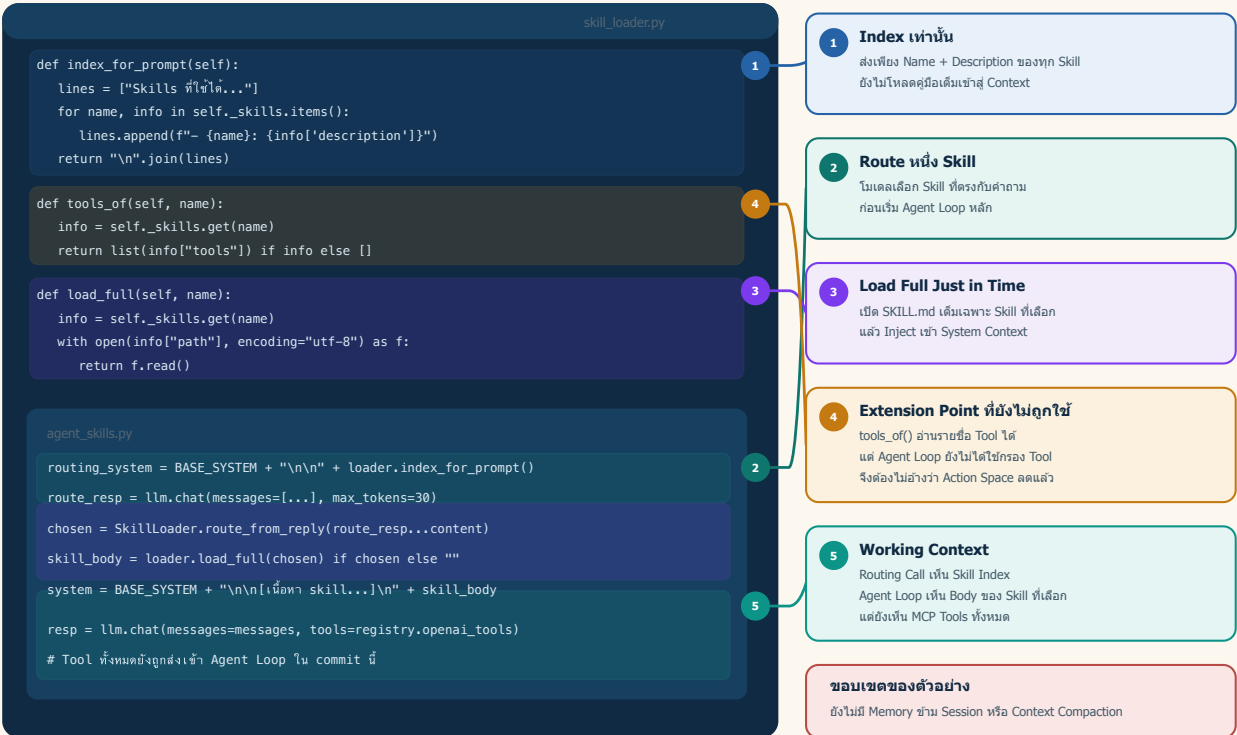
รูปแบบนี้ทำให้ Context เป็นพื้นที่ทำงานชั่วคราว ไม่ใช่คลังข้อมูลถาวร เปรียบได้กับโต๊ะทำงานที่ควรมีเฉพาะเอกสารของงานตรงหน้า ส่วนเอกสารทั้งหมดเก็บในระบบที่ค้นคืนได้

Progressive Disclosure จากโค้ดที่ใช้สอน

Lab 5 ใน Repository ตัวอย่างยังไม่มี Memory ชำม Session หรือ Context Compaction แต่มี Progressive Disclosure ที่ชัดเจนและเหมาะกับการเห็น Context Engineering ในโค้ดจริง [3]

Progressive Disclosure จากโค้ดจริง: Index → Route → Load Skill Body

labs/lab5_skills/skill_loader.py + agent_skills.py · ลด Instruction Context ก่อนเข้า Agent Loop



Source: github.com/aekun2020/Python-Agent-LangGraph · commit c3f95c3

Progressive Disclosure จาก Skill Index ไปสู่คู่มือเต็ม โดย Agent Loop ยังเห็น MCP Tools ทั้งหมด

`SkillLoader.index_for_prompt()` สร้าง Index ที่มีเพียงชื่อและคำอธิบายสั้นของทุก Skill จึงไม่ต้องโหลด SKILL.md ทุกไฟล์เข้าสู่ Context ตั้งแต่ต้น โมเดลใช้ Index นี้เลือก Skill ที่ตรงกับคำถามหนึ่งตัว

หลัง Routing เสร็จ `load_full(chosen)` จึงอ่านคู่มือเต็มของ Skill ที่เลือกแบบ Just in Time และเพิ่มเนื้อหานั้นเข้าสู่ System Context นี่คือการ Progressive Disclosure สองระดับ: เห็น Metadata ก่อน แล้วค่อยเปิด Instruction เต็มเมื่อจำเป็น

ในรูปแบบที่สมบูรณ์กว่า Disclosure ทำได้สามระดับ: Metadata ใช้เลือก, Body ให้ Workflow ตัวอย่าง และ Tool Guideline ส่วน Script ถูกเปิดหรือซ่อนเฉพาะเมื่อ Skill ต้องใช้ Logic เพิ่มเติม การโหลด Script แบบ Dynamic เพิ่ม Trust Boundary ใหม่ จึงต้องตรวจสอบ Source และ Permission แยกจาก Markdown

คำบรรยายเรียกออกการที่โมเดลตอบเองโดยไม่เลือก Skill หรือ Tool ว่า Agent “นอน” สาเหตุหนึ่งคือ Base System Prompt ไม่กำหนดว่าต้อง Route ก่อน โค้ดใน `agent_skills.py` แก่ด้วย Model Call แยกที่ประกอบ `routing_system` จาก Base Prompt และ Skill Index แล้วใช้ `SkillLoader.route_from_reply()` อ่านชื่อ

Skill ก่อนโหลด Body วิธีนี้ทำให้การเลือก Context เป็นขั้นตอนที่มองเห็นและทดสอบแยกได้

ต้องอ่านขอบเขตของโค้ดให้ตรงกับของจริง: `SkillLoader` มีเมธอด `tools_of()` แต่ `agent_skills.py` ที่ commit นี้ยังไม่ได้ใช้เมธอดดังกล่าวเพื่อจำกัด Tool Agent Loop ยังส่ง `registry.openai_tools` ทั้งหมดเข้า `llm.chat()` สิ่งที่ลดลงจริงใน Lab 5 จึงเป็น **Instruction Context** จากการโหลด Skill Body เพียงตัวเดียว ไม่ใช่ Action Space ของ Tool

Working Context หลัง Routing ประกอบด้วย Base Instruction และคู่มือเต็มของ Skill ที่เลือก ส่วน Skill Index ใช้ใน Routing Call ก่อนหน้าและไม่ได้ถูกส่งเข้า Agent Loop การจำกัด Tool Subset เป็นส่วนต่อ ยอดที่ควรเพิ่มภายหลัง หากต้องการลดทั้ง Token และความสับสนในการเลือก Tool

อย่างไรก็ตาม ต้องแยกขอบเขตให้ชัด โค้ดนี้แสดง Skill Routing และ Progressive Disclosure แต่ยังไม่ได้ทำ Retrieval จาก Memory, การสรุป History หรือการจัดลำดับเอกสารหลายแหล่ง ส่วนเหล่านั้นเป็นชั้น Context Engineering ที่ต้องเพิ่มในบทและ Lab ถัดไป

วัดคุณภาพ Context แยกจากคุณภาพโมเดล

เมื่อ Agent ตอบผิด เรามักสรุปว่าโมเดลไม่เก่ง แต่สาเหตุอาจเป็น Retrieval ไม่พบเอกสาร Tool Result ถูกตัด System Instruction ขัดกัน หรือ Memory เก่าแทรกเข้ามา การทดสอบจึงควรบันทึกทั้ง Input Context, Source, Token Usage และผลลัพธ์

หลักสำคัญของบทนี้คือ:

Context ที่ดีไม่ใช่ Context ที่ใหญ่ที่สุด แต่เป็น Context ที่เลือกพอให้โฟกัส และครบพอให้ตัดสินใจ

Context Budget: ทุก Token ต้องมีหน้าที่

Anthropic ให้นิยามเชิงปฏิบัติของ Context Engineering ว่าเป็นการค้นหาชุด Token ที่มีสัญญาณสูงและเล็ก ที่สุดซึ่งยังเพิ่มโอกาสให้เกิดผลลัพธ์ที่ต้องการ [2] คำว่า “เล็กที่สุด” ไม่ได้แปลว่าสั้นที่สุด แต่แปลว่า Context ทุก ก่อนต้องอธิบายได้ว่ามีไว้เปลี่ยนการตัดสินใจใด

Budget	หน้าที่	เงื่อนไขออก
Instruction	กำหนดบทบาท กฎ และ Workflow	เปลี่ยนเมื่อ Policy version เปลี่ยน
Tool definition	บอก Action Space ที่ใช้ได้ในรอบนี้	ตัด Tool ที่ไม่เกี่ยวข้อง
Evidence	ให้ข้อเท็จจริงสำหรับการตัดสินใจ	แทน Raw data เก่าด้วย Reference
Working state	รักษา Goal, Todo และ Constraint	ปิดเมื่อภารกิจสิ้นสุด
History	ให้ความต่อเนื่องระหว่าง Turn	Prune หรือ Compact เมื่อหมดผลต่ออนาคต

การมี Budget ทำให้ Context เป็นสิ่งที่วัดและ Review ได้ แทนการต่อข้อความเพิ่มไปเรื่อย ๆ เมื่อ Agent ผิด Harness ควรบอกให้ได้ว่าปัญหาเกิดจากข้อมูลจำเป็นไม่เข้า Context, Noise ไม่ถูกนำออก หรือข้อมูลสำคัญอยู่ในตำแหน่งที่โมเดลใช้ไม่ได้

Harness Design Check

- **Harness รับผิดชอบ:** เลือก, จัดลำดับ, Compact และเปิดเผย Context แบบ Just in Time
- **Model ตัดสินใจ:** ใช้ข้อมูลที่มองเห็นเพื่อตีความและเลือกก้าวถัดไป
- **Evidence ที่ต้องเก็บ:** Context snapshot, Source, Token count, Compaction และสิ่งที่ถูกตัด
- **Failure ที่ต้องทดสอบ:** Lost in the Middle, Context Rot, Instruction conflict และข้อมูลจำเป็นถูกตัด

เอกสารอ้างอิง

[1] Thongtae, Aekanun. “Complete Context, Context Rot, and Progressive Disclosure.” *Author’s Lecture Transcript*, July 2026.

[2] Anthropic. “Effective Context Engineering for AI Agents.” September 2025.

[3] Thongtae, Aekanun. *Python-Agent-LangGraph*, [labs/lab5_skills/agent_skills.py](#) and [skill_loader.py](#), commit [c3f95c3](#), GitHub repository, 2026.

บทนิยามหลักการประกอบ Working Context สำหรับหนึ่งรอบ โดยยังไม่ลงรายละเอียดว่าความรู้ภายนอกถูกค้นมาอย่างไรหรือ History ถูกเก็บข้าม Turn อย่างไร ก่อนถึงสองเรื่องนั้น บทถัดไปจะจัดการ State ของงานหลายชั้นเพื่อให้ Agent มองเห็นแผนและความคืบหน้าโดยไม่ต้องฝากไว้กับถ้อยคำของโมเดล

บทที่ 7 — Planning และ Todo State: ทำให้งานหลายขั้นมองเห็นได้

TodoWrite จากโค้ดจริง: Plan → State → Tool → Evidence

labs/lab6_todo/agent_todo.py · Python-Agent-LangGraph · commit c3f95c3

```
30 class TodoState:
35 def write(self, items):
36 self.items = [{index, task, status: "todo"}, ...]
39 def update(self, index, status): # normalize index
```

```
60 def build_tools(registry):
63 name: "todo_write" → items: array[string]
68 name: "todo_update" → index: integer
71 status: enum[todo,doing,done]
```

```
93 if name == "todo_write":
94 result = todo.write(args.get("items", []))
96 elif name == "todo_update":
97 result = todo.update(args.get("index"), args.get("status"))
102 messages.append({role: "tool", content: result})
```

1 State อยู่นอก LLM

รายการและสถานะคงอยู่ระหว่างรอบ
update() Normalize index ก่อนเปลี่ยน State

2 State มี Contract

Schema จำกัดชนิดข้อมูลและ enum
โมเดลเสนอ Transition ผ่าน Tool Call

3 Harness เป็นผู้เปลี่ยน State

Dispatch แล้วคืน Todo เป็น Observation
รอบถัดไปเห็นว่ามีงานใดเสร็จจริง
ควรผูก done กับ Evidence เพิ่มเติม

TodoWrite เปลี่ยนแผนให้เป็น State จากโค้ดจริง

คำสั่งว่า “ช่วยจัดการเรื่องนี้ให้เสร็จ” อาจซ่อนงานย่อยหลายขั้น หากปล่อยให้แผนอยู่ในข้อความภายใน Context อย่างเดียว Agent อาจข้ามขั้น ทำซ้ำ หรือสรุปก่อนงานจริงเสร็จ Harness จึงควรยกแผนขึ้นมาเป็น State ที่อ่านและอัปเดตได้

แผนไม่ใช่คำตอบล่วงหน้า

Planning มีหน้าที่แยกเป้าหมายออกเป็นก้าวที่ตรวจสอบได้ แต่ไม่ได้รับรองว่าแต่ละก้าวสำเร็จ สถานะ **todo**, **doing** และ **done** ต้องเปลี่ยนตาม Observation จากการลงมือจริง ไม่ใช่เปลี่ยนเพราะโมเดลคาดว่าน่าจะสำเร็จ

โค้ดใน `labs/lab6_todo/agent_todo.py` ใช้ `TodoState` เป็นหน่วยความจำระหว่างรอบของ Agent Loop รายการแต่ละข้อมี `index`, `task` และ `status` เมธอด `write()` สร้างแผนใหม่ ส่วน `update()` Normalize index แล้วเปลี่ยนสถานะที่ละข้อ [3]

```
class TodoState:
    def write(self, items):
        self.items = [
            {"index": i + 1, "task": task, "status": "todo"}
            for i, task in enumerate(items)
        ]

    def update(self, index, status):
        ...

    def render(self):
        mark = {"todo": "[ ]", "doing": "[~]", "done": "[x]"}
        return "\n".join(
            f"{mark.get(i['status'], '[ ]')} {i['index']}. {i['task']}"
            for i in self.items
        )
```

สิ่งสำคัญคือ Todo ไม่ได้เก็บอยู่ใน LLM แต่อยู่ในโปรแกรม Python ต่อให้โมเดลเปลี่ยนถ้อยคำในรอบถัดไป Harness ยังมี State เดิมเป็นหลักอ้างอิง

ทำให้ State กลายเป็น Tool

`todo_write` และ `todo_update` ถูกประกาศด้วย Function Tool Schema โมเดลจึงไม่ได้แก้ State โดยตรง แต่เสนอคำขอที่มีโครงสร้าง แล้ว Harness เป็นผู้ดำเนินการ

```
{
  "name": "todo_update",
  "parameters": {
    "type": "object",
    "properties": {
      "index": {"type": "integer"},
      "status": {"type": "string",
        "enum": ["todo", "doing", "done"]}
    },
    "required": ["index", "status"]}
}
```

`enum` จำกัดภาษาของ State ไม่ให้เกิดสถานะตามอำเภอใจ เช่น `almost_done` หรือ `probably_finished` ส่วน System Prompt กำหนดกติกาว่างานตั้งแต่สามขั้นขึ้นไปต้องเขียน Todo ก่อน เปลี่ยนเป็น `doing` ก่อนลงมือ และเป็น `done` หลังได้รับผลแล้ว

ความทนทานเล็ก ๆ ที่สำคัญ

ใน `update()` มีการรองรับทั้ง Index แบบ 1-based และ 0-based เพราะ LLM อาจส่งเลขต่างจากรูปแบบที่แสดง การ Normalize นี่เป็นตัวอย่างของ Harness Engineering: เราไม่หวังให้โมเดลแม่นยำทุกครั้ง แต่สร้าง Boundary ที่รับความคลาดเคลื่อนซึ่งคาดการณ์ได้

อย่างไรก็ตาม Todo ที่เป็น **done** ยังไม่เท่ากับ Business Outcome ที่ได้รับการยืนยัน ระบบที่เข้มข้นควรผูกแต่ละข้อกับหลักฐาน เช่น Tool Call ID, Result, Record ID หรือเงื่อนไขตรวจสอบ แล้วอนุญาตให้ปิดข้อได้เมื่อ Evidence ครบเท่านั้น

จากรายการงานสู่ State Machine

TodoWrite คือ State Machine ขนาดเล็ก:

todo → **doing** → **done**

To-Do Right: แผนต้องกลับเข้า Loop

ในคำบรรยาย จุดสำคัญไม่ใช่เพียงสร้าง Checklist แต่ต้อง Inject แผนและสถานะล่าสุดกลับเข้า Context ภายใน Tool Loop เพื่อให้โมเดลทบทวนว่าเพิ่งทำข้อใด ได้หลักฐานอะไร และก้าวถัดไปคืออะไร โค้ดทำสิ่งนี้ผ่านผลของ **todo.write()** และ **todo.update()** ซึ่งถูกเพิ่มเป็น Message **role: "tool"** ทุกครั้ง [1]

กติกาที่ใช้สอนคือ งานหลายขั้นให้แตก Task ก่อน เปลี่ยนสถานะเป็น **doing** ก่อนลงมือ และ **done** หลังมี Result หาก Agent ปิดหลายข้อพร้อมกันโดยไม่มี Observation คั่นกลาง Checklist กำลังกลายเป็นเรื่องเล่าของโมเดล ไม่ใช่ State ของงาน

Maintain State แยก “สิ่งที่ตั้งใจทำ” ออกจาก “สิ่งที่เกิดขึ้นแล้ว”

ลำดับการสอนวาง To-Do-Write ก่อน Lab ที่เน้น Maintain State อย่างชัดเจน ประเด็นจึงไม่ใช่ให้โมเดลเขียนแผนสวยขึ้น แต่ให้โปรแกรมถือสถานะข้ามหลายรอบของการคิดและการเรียก Tool ได้ หากมีเพียงข้อความว่า “กำลังทำข้อ 2” ในคำตอบของโมเดล ข้อความนั้นอาจหายไปเมื่อ Context ถูกตัดหรือสรุป แต่ State ใน Harness ยังตรวจ อ่าน และส่งกลับเข้าโมเดลได้

แต่ละ Transition ควรมีส่วนน้อยสี่ส่วน: สถานะเดิม การเปลี่ยนที่ร้องขอ Observation ที่รองรับ และสถานะใหม่ ด้วยเหตุนี้ **todo → done** โดยไม่มี Tool Result หรือหลักฐานจึงควรถูกปฏิเสธหรือทำเครื่องหมายให้ตรวจสอบ ไม่ใช่เชื่อเพราะโมเดลประกาศว่าเสร็จแล้ว

จุดนี้ทำให้ Planning เชื่อมกับ Context Engineering โดยตรง: Todo ล่าสุดคือ Context ที่ควรคงอยู่ ส่วนรายละเอียด Tool Output เก่าที่ใช้ยืนยันขั้นก่อนหน้าอาจย่อเป็น Receipt หรือ Reference ได้ เราจึงรักษา “สถานะของภารกิจ” โดยไม่ต้องแบก “ประวัติทุกอย่างคำ” ไปตลอดงาน

Checkpoint คือการตัดสินใจจาก State และ Ground Truth

Checkpoint ไม่ใช่การหยุดทุกครั้งเพื่อถามมนุษย์ แต่เป็นจุดที่ Harness ประเมินว่า Evidence ใน State เพียงพอให้เปลี่ยนขั้นหรือยัง งานความเสี่ยงต่ำอาจผ่านด้วยผลตรวจอัตโนมัติ งานที่ย้อนกลับยากอาจต้อง Human Approval และเมื่อพบ Blocker ซึ่ง Agent แก่เองไม่ได้ควรหยุดเพื่อขอข้อมูลหรือคำตัดสิน [2]

ดังนั้น Checkpoint ที่ดีต้องระบุ Trigger, Evidence requirement, Decision maker และ Next state อย่างชัดเจน ถ้ามีเพียงคำสั่งว่า “ถามผู้ใช้เมื่อจำเป็น” โมเดลต้องตีความความจำเป็นเองทุกครั้ง แต่ถ้า Harness กำหนดว่า **needs_approval** ต้องมี Preview, Risk level และผู้อนุมัติ การควบคุมจะตรวจย้อนหลังและทดสอบได้

เมื่อกติกาซับซ้อนขึ้น เราอาจเพิ่ม `blocked`, `failed` หรือ `needs_approval` แต่ยิ่งเพิ่มสถานะก็ยิ่งต้องกำหนด Transition ให้ชัด มิฉะนั้น LLM จะกลายเป็นผู้ตีความ Workflow เองทั้งหมด เป้าหมายของ Planning Harness จึงไม่ใช่ทำแผนให้ละเอียดที่สุด แต่ทำให้ก้าวสำคัญมองเห็น ตรวจสอบ และดำเนินต่อได้อย่างมีหลักฐาน

Harness Design Check

- **Harness รับผิดชอบ:** เก็บ Todo/State, ตรวจ Transition และผูกสถานะกับ Observation
- **Model ตัดสินใจ:** แต่งงานและเสนอการเปลี่ยนสถานะทีละก้าว
- **Evidence ที่ต้องเก็บ:** State version, Transition, ผู้อนุมัติ, Result และรายการค้าง
- **Failure ที่ต้องทดสอบ:** ข้ามขั้น, ปิดงานพร้อมกัน, Index คลาดเคลื่อน และ State ไม่ตรง External Reality

เมื่อการลงมือบางขั้นไม่สำเร็จ State ต้องบอกได้ว่า Error นั้นแก้ไขและลองใหม่ได้หรือควรหยุด บทถัดไปจึงพาออกแบบ Error ให้เป็น Observation ที่ช่วย Agent ไปต่อ แทนข้อความล้มเหลวที่ทำให้วงจรจบโดยไม่มีทางเลือก

เอกสารอ้างอิง

[1] Thongtae, Aekanun. “To-Do-Write, Planning, and Maintaining Agent State.” *Author’s Lecture Transcript*, August 2026.

[2] Anthropic. “Building Effective Agents.” December 2024.

[3] Thongtae, Aekanun. *Python-Agent-LangGraph*, `labs/lab6_todo/agent_todo.py`, commit `c3f95c3`, GitHub repository, 2026.

บทที่ 8 — Recoverable Error: ออกแบบความล้มเหลวให้ Agent ไปต่อได้

Recoverable Error: Exception ต้องข้าม Boundary มาเป็น Observation

labs/core/mcp_client.py + labs/lab6_todo/agent_todo.py · commit c3f95c3

```
61 resp = self._client.post(self.url, ...)
62 resp.raise_for_status()

# HTTP error → Exception หลุดออกจาก Agent Loop
# ยังไม่มี error_code / hint / retryable
```

```
100 result = registry.dispatch(name, args)
102 messages.append({
    role: "tool", tool_call_id: call.id,
    content: result })
```

```
84 for step in range(1, max_steps + 1): ...
110 # ขนเพดาน max_steps
112 "ห้ามเรียก tool เพิ่ม — สรุปจากข้อมูลที่ได้"
```

1 Fail-fast Boundary

ตรวจ HTTP Failure ได้ทันที แต่ยังไม่แปลงเป็นผลที่ Agent อ่านได้ นี่คือช่องว่างของโค้ดปัจจุบัน

2 Observation Channel

Tool Result ถูกส่งกลับด้วย role=tool ถ้าตก Exception เป็น JSON ที่มี Hint ไม่เคยจะปรับ Argument แล้วลองใหม่ได้

3 Stopping Boundary

เพดานรอบหยุด Retry ที่ไม่สิ้นสุด ต้องรายงาน unfinished อย่างชัดแจ้ง

Error กลายเป็น Observation จากโค้ดจริง

โปรแกรมทั่วไปมักแบ่งผลเป็น Success กับ Exception แต่สำหรับ Agent คำว่า Error อย่างเดียวมีข้อมูลไม่พอ ถ้า Tool เพียงตอบว่า **failed** โมเดลไม่รู้ว่าจะลองใหม่ ลดขอบเขตข้อมูล เปลี่ยน Argument ขอสิทธิ์จากมนุษย์ หรือหยุด

Error คือส่วนหนึ่งของ Observation

ในคำบรรยาย ผมแยก Error เป็นอย่างน้อยสองระดับ

- **Recoverable** — Agent มีสิ่งที่ทำได้ เช่น Timeout พร้อม Hint ให้ลดช่วงวัน
- **Unexpected** — ระบบยังไม่มีวิธีแก้อัตโนมัติ ต้องเก็บ Log และส่งให้ผู้พัฒนาตรวจสอบ

คำบรรยายเชื่อม Error Handling กับ Observation โดยตรง: หาก Function ล้มแล้ว Program Terminate โมเดลไม่ได้เรียนรู้อะไรจากความล้มเหลวนั้น แต่หาก Harness แปลง Exception เป็น Observation ที่มี Error Code และ Hint Agent สามารถทบทวน Action แล้วสร้างคำขอใหม่ได้ Recoverability จึงไม่ได้หมายถึง “จับ Exception ไม่ให้ Crash” เท่านั้น แต่หมายถึงการสร้างข้อมูลสำหรับการตัดสินใจรอบถัดไป [1]

Recovery Ladder

1. **Normalize** — แปลง Error จากระบบปลายทางเป็น Code และ Cause ที่เสถียร
2. **Constrain** — บอกขอบเขตการแก้ เช่น ลดช่วงเวลา ลดจำนวนรายการ หรือแก้ไขชนิดข้อมูล
3. **Retry safely** — ตรวจ Budget, Idempotency และ Side Effect ก่อนลองใหม่
4. **Change strategy** — เปลี่ยน Tool หรือขอข้อมูลเพิ่มเมื่อการลองแบบเดิมไม่มีประโยชน์
5. **Escalate** — ส่ง State, Attempts และ Evidence ให้มนุษย์เมื่อเกินขอบเขต

ตัวอย่างจากชั้นเรียนคือค่าขอช่วงเวลา 30 วันทำให้ Timeout แทนที่จะคืนเพียง `timeout` Harness ควรบอกว่า Retry ได้และเสนอให้ลดเหลือช่วงสั้นกว่า Hint ไม่จำเป็นต้องเฉลยทุกอย่าง แต่ต้องลดพื้นที่ค้นหาคำตอบให้ Agent อย่างปลอดภัย

ผลลัพธ์ที่ช่วยให้ Agent พันทวีควรมีโครงสร้าง เช่น

```
{
  "ok": false,
  "error_code": "UPSTREAM_TIMEOUT",
  "message": "เรียกข้อมูล 30 วันไม่สำเร็จ",
  "hint": "ลดช่วงเหลือ 3 วันแล้วลองใหม่",
  "retryable": true
}
```

`message` อธิบายสิ่งที่เกิดขึ้น ส่วน `hint` บอกการกระทำถัดไป และ `retryable` ช่วยให้ Harness บังคับนโยบายได้โดยไม่ต้องให้ LLM เดา

อ่านเส้นทาง Error จากโค้ดจริง

ใน `labs/core/mcp_client.py` เมธอด `_rpc()` ส่ง HTTP Request แล้วเรียก `raise_for_status()` ทันที นี่เป็น Boundary ที่ดีสำหรับตรวจ HTTP Failure แต่ในเวอร์ชันปัจจุบัน Exception จะหลุดออกจาก Agent Loop และจบโปรแกรม [2]

```
resp = self._client.post(
    self.url, headers=self._headers(), json=body
)
resp.raise_for_status()
```

อีกด้านหนึ่ง เมื่อ Tool ทำงานและคืนข้อความได้ `labs/lab6_todo/agent_todo.py` จะนำ `result` ใส่กลับเข้า `messages` ด้วย Role `tool` นี่คือช่องทาง Observation ที่โมเดลใช้คิดรอบถัดไป

```
result = registry.dispatch(name, args)
messages.append({
    "role": "tool",
    "tool_call_id": call.id,
    "content": result,
})
```

ดังนั้นช่องว่างไม่ได้อยู่ที่ Agent Loop ไม่มีทางรับ Error แต่อยู่ที่ Exception จาก Execution Boundary ยังไม่
ได้ถูกแปลงเป็น Tool Result ที่มีความหมาย หากดัก Timeout หรือ Validation Error แล้ว Serialize เป็น
Observation แบบมี `error_code` และ `hint` วงจรเดิมก็สามารถให้โมเดลแก้ Argument และลองใหม่ได้

อย่า Retry ทุกอย่าง

Retry ที่ไม่มีนโยบายอาจทำให้เสียค่าใช้จ่าย เรียก API ซ้ำ หรือเขียนข้อมูลซ้ำ Harness ควรแยกอย่างน้อยว่าเหตุ
ใด Retry ได้ จำนวนครั้งสูงสุดเท่าใด และการกระทำนั้นเป็น Read หรือ Write

- Timeout ของการอ่านข้อมูลอาจ Retry ด้วย Backoff
- Validation Error ควรแก้ Argument ก่อน
- Authentication Error ควรหยุดและขอ Credential
- การเขียนข้อมูลที่ไม่ทราบว่าจะสำเร็จหรือไม่ ต้องใช้ Idempotency Key หรืออ่านกลับไปตรวจสอบก่อนทำซ้ำ

Error มีอายุการใช้งาน

คำบรรยายเรื่อง Context Clearing ช่วยเติมอีกด้านหนึ่งของ Recoverability: Error ที่ยังบอกวิธีแก้ต้องถูกเก็บ
เป็น Observation แต่ Error เก่าที่แก้แล้วไม่ควรติดอยู่ใน Context ตลอดไป มิฉะนั้นโมเดลอาจตีความว่างานยัง
ล้มเหลวและ Context จะบวมจาก Stack Trace หรือข้อความซ้ำ

Harness จึงควรสกัด Error ให้เหลือ Code, Cause, Retryability และ Hint แล้วผูกกับ Step ที่เกิด เมื่อ Step
สำเร็จหรือเปลี่ยนเส้นทาง Error เดิมสามารถย้ายไป Trace และตัดออกจาก Working Context ได้ การ Clear
ไม่ใช่การลบหลักฐาน แต่เป็นการแยก Operational Log ออกจากข้อมูลที่โมเดลจำเป็นต้องใช้รอบถัดไป

เพดานรอบคือ Safety Boundary

โค้ดจริงกำหนด `max_steps=30` และเมื่อชนเพดานจะห้ามเรียก Tool เพิ่ม แล้วบังคับให้สรุปจากข้อมูลที่มี

```
for step in range(1, max_steps + 1):
    ...

messages.append({
    "role": "user",
    "content": "ถึงขีดจำกัดขั้นตอนนี้แล้ว ห้ามเรียก tool เพิ่ม ..."
```

นี้ช่วยป้องกัน Infinite Loop แต่คำตอบเมื่อชนเพดานควรบอกสถานะที่ยังไม่เสร็จอย่างตรงไปตรงมา ไม่ควรเปลี่ยน “หยุดเพราะหมดรอบ” ให้ดูเหมือน “งานสำเร็จ” Recoverability ที่ดีจึงต้องทำงานคู่กับ Stopping Condition และสถานะผลลัพธ์ที่ชัดเจน

Harness Design Check

- **Harness รับผิดชอบ:** Normalize Error, กำหนด Retry/Backoff/Budget และ Escalation
- **Model ตัดสินใจ:** เลือกแก้ Arguments, เปลี่ยนเส้นทาง หรือขอความช่วยเหลือจาก Hint
- **Evidence ที่ต้องเก็บ:** Error code, Cause, Attempt, Retry decision, Side Effect และ Final status
- **Failure ที่ต้องทดสอบ:** Retry ไม่สิ้นสุด, ทำ Write ซ้ำ, กลบ Fatal error และรายงานสำเร็จเมื่อชนเพดาน

เมื่อเส้นทางการลงมือมีขอบเขตแล้ว Harness ยังต้องจัดหลักฐานภายนอกให้พอกับการตัดสินใจ บทถัดไปจึงพาไปออกแบบ Retrieval ให้สร้าง Evidence Package ที่ตรวจแยกจากคำตอบของ LLM ได้

เอกสารอ้างอิง

[1] Thongtae, Aekanun. “Recoverable Errors, Error Codes, Hints, and Self-correction.” *Author’s Lecture Transcript*, August 2026.

[2] Thongtae, Aekanun. *Python-Agent-LangGraph*, [labs/core/mcp_client.py](#) and [labs/lab6_todo/agent_todo.py](#), commit [c3f95c3](#), GitHub repository, 2026.

บทที่ 9 — Retrieval Engineering: สร้าง Evidence จากความรู้ภายนอก

Retrieval จากโค้ดจริง: Query → Hybrid Search → Graph → Evidence

retrieval.py + vector_index.py · ตั้ดรายละเอียตโดเมนออก แต่รักษากลไกของโค้ด

```
44 keywords = extract_query_keywords(query)
45 low_query = join(keywords["low_level"])
46 high_query = join(keywords["high_level"])
```

1 Query Decomposition

คำเฉพาะช่วยค้นรายละเอียต
คำกว้างช่วยค้นแนวคิดและความสัมพันธ์

```
51 profiles = index.search_profiles(high_query, top_k)
58 expanded = _expand_from_graph(profiles, graph,
max_expand=3, max_hops=2)
62 chunks = index.search_chunks(chunk_query, max_chunks)
```

2 ส่องเส้นทางของ Evidence

Profile เป็น Seed ให้ Graph Traversal
Chunk รักษาข้อความต้นฉบับ
max_hops / max_chunks คือ Context Budget

```
vector_index.py:
Prefetch(dense) + Prefetch(sparse)
query = FusionQuery(fusion=Fusion.RRF)
```

3 Dense + Sparse + RRF

รวมความหมายกับคำตรงโดยรวมอันดับ

```
64 context = _build_context(profiles, expanded, chunks)
67 return {keywords, profiles, chunks, context}
```

4 Evidence Package

เก็บ Candidate และ Context เพื่อตรวจสอบหลัง

Retrieval Pipeline จากโค้ดจริง

เมื่อคำตอบต้องอ้างอิงข้อมูลภายนอก ปัญหาไม่ได้จบที่ “มี Vector Database แล้ว” Harness ต้องเตรียมข้อมูล ค้นหา จัดอันดับ ตรวจสอบ Source และประกอบ Context ที่เพียงพอให้ LLM ตอบโดยไม่เติมช่องว่างเอง

อ่าน Retrieval Pipeline จากโค้ดจริง

โค้ดที่ใช้สอนแยก Retrieval ออกเป็นขั้นที่ตรวจได้ เริ่มจากสกัด Keyword สองระดับ แล้วค้นทั้ง Knowledge Profile และเอกสาร ก่อนขยายความสัมพันธ์ใน Graph และประกอบ Context ภาพใช้ชื่อย่อเพื่อให้อ่านได้ในหน้าเดียว ส่วน Listing ต่อไปนี้รักษาชื่อ Function และตัวแปรหลักจาก Source [2]:

```
keywords = extract_query_keywords(query)
profile_results = index.search_profiles(high_query, top_k=top_k)
expanded = _expand_from_graph(
    profile_results, knowledge_graph, max_expand=3, max_hops=2
)
chunk_results = index.search_chunks(chunk_query, top_k=max_chunks)
context = _build_context(profile_results, expanded, chunk_results)
```

บรรทัดเหล่านี้คือ Harness ไม่ใช่ความสามารถภายใน LLM: Harness กำหนดจำนวน Candidate, จำนวน Hop, จำนวน Chunk และรูปแบบ Evidence ที่โมเดลจะเห็น หากเปลี่ยน Budget หรือวิธีรวม Candidate พฤติกรรมของ Agent ย่อมเปลี่ยน แม้ใช้โมเดลเดิม

ใน `VectorIndex.search_profiles()` การค้น Dense และ Sparse ทำงานแยกกัน แล้ว Qdrant รวมอันดับด้วย Reciprocal Rank Fusion:

```
prefetch=[
    Prefetch(query=dense_vec.tolist(), using="dense", limit=top_k * 2),
    Prefetch(query=sparse_vec, using="sparse", limit=top_k * 2),
],
query=FusionQuery(fusion=Fusion.RRF),
limit=top_k,
```

Dense Search ช่วยจับความใกล้เคียงความหมาย ส่วน Sparse Search รักษาเฉพาะหรือรหัสที่ต้องตรง การรวมอันดับด้วย RRF หลีกเลี่ยงการนำ Score จากคนละระบบมาเปรียบเทียบตรง ๆ

Retrieval มีสอง Flow

คำบรรยายแยกสถาปัตยกรรมเป็นสองเส้นทาง

Admin Flow เตรียม Corpus ก่อนใช้งาน:

Document → Parse → Chunk → Embed → Store

เอกสารถูก Parse เป็นรูปแบบที่รักษาโครงสร้าง แบ่งเป็น Chunk แล้วแปลงแต่ละ Chunk เป็น Vector ก่อนเก็บทั้งข้อความ Metadata และ Vector ลง Store

End-user Flow ทำงานเมื่อมีคำถาม:

Question → Query Embedding → Search → Rank → Context → LLM

คำบรรยายแยก “Retrieval Pipeline” ออกจาก “Tool Surface” อย่างชัดเจน Pipeline แบบ GraphRAG อาจบังคับให้เริ่มจาก Vector Search แล้วขยาย Graph เพื่อสร้าง Context แต่เมื่อความสามารถถูกเปิดผ่าน MCP แต่ละ Tool สามารถเลือกเส้นทางต่างกันได้ Tool หนึ่งอาจรัน Hybrid Pipeline เต็มชุด อีก Tool รับ Identifier แล้วอ่าน Graph โดยตรง และอีก Tool อาจค้น Relational Database โดยไม่แตะ Vector Store เลย [1]

ดังนั้นคำว่า “ต่อ MCP แล้วได้ RAG” ไม่ถูกต้อง MCP กำหนดวิธี Discover และ Invoke ความสามารถ ส่วน Retrieval Logic อยู่หลัง Tool Contract การแยกสองชั้นนี้ทำให้ทีมเปลี่ยน Index หรือ Traversal ได้โดยไม่เปลี่ยน Agent Loop และทำให้ทดสอบ Retrieval Function โดยไม่ต้องเรียก Agent ได้

จุดที่มักพลาดคือใช้ Embedding Model คนละตัวหรือคนละ Version ระหว่าง Index กับ Query แม้ Vector มี Dimension เท่ากัน ความหมายใน Space อาจไม่ตรงกันและ Retrieval จะเสียโดยไม่เกิด Error ชัดเจน

สิ่งที่ส่งให้ LLM คือข้อความ ไม่ใช่ Vector

Vector ใช้วัดความใกล้เคียงใน Retrieval Layer แต่ LLM ต้องได้รับ Chunk ที่เป็นข้อความ พร้อม Metadata เช่น Source ID, Section, Score และ Timestamp Harness ควรรักษาข้อมูลเหล่านี้ไว้เพื่อให้ Final Answer อ้างหลักฐานและย้อนกลับไปได้

โค้ดเชิงโครงสร้างของ Flow มีหน้าตาประมาณนี้:

```
query_vector = embed(question)
hits = vector_store.search(query_vector, limit=20)
ranked = rerank(question, hits)
context = [
    {"source": h.id, "text": h.text, "score": h.score}
    for h in ranked[:5]
]
answer = llm.chat(messages=build_prompt(question, context))
```

โค้ดนี้เป็นการถอด Pipeline จากคำบรรยายเพื่อชี้ Boundary ไม่ใช่ Code Listing จาก Repository: `embed()` สร้างตัวแทน Query, `search()` หา Candidate, `rerank()` จัดลำดับ และ `build_prompt()` แปลงผลค้นให้เป็น Complete Context

Dense, Sparse และ Hybrid Search

Dense Search ใช้ Embedding จับความคล้ายเชิงความหมาย จึงค้นข้อความที่ใช้คนละคำแต่หมายถึงเรื่องเดียวกันได้ Sparse Search เช่น BM25 ให้ความสำคัญกับคำที่ตรงกัน เหมาะกับรหัส ชื่อเฉพาะ และศัพท์ที่ต้อง Exact Match

Hybrid Search ใช้ทั้งสองทางแล้วรวมอันดับ เช่น Reciprocal Rank Fusion (RRF) ซึ่งรวม Rank แทนการบวก Score ดิบ เพราะ Dense Score กับ BM25 Score อยู่คนละสเกล ผลลัพธ์จาก Hybrid ไม่ได้รับประกันว่าถูก แต่ลด Blind Spot ของการค้นแบบเดียว

HNSW เร็วเพราะไม่ได้มองทุกจุด

Approximate Search อย่าง HNSW ใช้โครงสร้างกราฟช่วยเดินไปหาเพื่อนบ้านที่น่าจะใกล้เคียง จึงเร็วเมื่อข้อมูลใหญ่ แต่มีโอกาสพลาด Candidate ส่วน Exact KNN เปรียบเทียบกว้างกว่าและช้ากว่า Retrieval Evaluation จึงต้องวัดทั้ง Recall, Latency และ Cost ไม่ใช่ดูเพียงว่าระบบคืนผลได้

ข้อจำกัดของ Vector-only Retrieval

Similarity Search ค้นหาข้อมูลที่คล้ายคำถามโดยตรง แต่คำตอบบางชนิดต้องใช้ข้อมูลที่เกี่ยวข้องผ่านความสัมพันธ์หลายชั้น Chunk ที่จำเป็นอาจไม่คล้าย Query มากพอและไม่ติด Top-k เมื่อ Context ขาด LLM มีแนวโน้มเติมช่องว่างด้วยภาษาที่สมจริง

คำบรรยายสรุป Graph Retrieval ไว้อย่างกระชับว่า:

GraphRAG = Similarity Search + Graph Traversal

ระบบใช้ Similarity Search หา Seed Node ก่อน แล้วเดินตาม Relationship ใน Knowledge Graph จำนวน Hops ที่กำหนดเพื่อรวบรวมข้อมูลประกอบ การเดินกราฟจะมีประโยชน์ก็ต่อเมื่อ Node Type, Relationship Type, Direction และ Property ถูกออกแบบตามความหมายของโดเมน หาก Schema ไม่ดี Multi-hop จะเพียงลากข้อมูลเพิ่มโดยไม่ช่วยตอบคำถาม

Context ที่มากขึ้นอาจทำให้คำตอบแย่ลง

การเพิ่ม **top_k** หรือจำนวน Hops ทำให้ Recall สูงขึ้นได้ แต่ก็เพิ่ม Noise และ Context Bloat Retrieval Harness จึงต้องกำหนด Budget: Candidate ที่ขึ้น, Rerank เหลือเท่าใด, Traversal ที่ Hops และส่ง Token เข้า LLM ได้เท่าใด

Result ที่ดีควรตอบได้ว่า:

- Chunk นี้มาจากเอกสารและตำแหน่งใด
- ถูกค้นพบด้วย Dense, Sparse หรือ Traversal
- Score และ Rank คือเท่าใด
- ข้อมูลใหม่หรือหมดอายุแล้วหรือไม่
- ผู้ใช้มีสิทธิ์อ่าน Source นี้หรือไม่

ทดสอบ Retrieval แยกจาก Generation

ก่อนถามว่า LLM ตอบดีหรือไม่ ต้องตรวจว่า Retrieval ดึง Evidence ที่ต้องใช้มาครบหรือยัง ใช้ชุดคำถามที่มี Expected Sources แล้ววัด Recall@k, Precision@k และตำแหน่ง Rank จากนั้นจึงทดสอบว่าเมื่อ Evidence ครบ LLM สรุปและอ้าง Source ถูกต้องหรือไม่

repo ฝั่ง Evaluation ทำให้การแบ่งนี้ชัดเจนด้วยสี่เมตริก: **context_precision** และ **context_recall** ตรวจ Retrieval ส่วน **faithfulness** และ **answer_relevancy** ตรวจ Generation การแยกคะแนนทำให้รู้ว่าควรแก้ Index, Query, Context Assembly หรือ Prompt แทนการเปลี่ยนโมเดลโดยเดา

Harness Design Check

- **Harness รับผิดชอบ:** สกัด Query, ค้น Candidate, รวมอันดับ, จำกัด Budget และประกอบ Evidence Package
- **Model ตัดสินใจ:** ช่วยตีความ Query และสร้างคำตอบจาก Evidence ที่ได้รับ
- **Evidence ที่ต้องเก็บ:** Query, Keywords, Candidate IDs, Scores, Rank, Source Metadata และ Context ที่ส่งจริง
- **Failure ที่ต้องทดสอบ:** หาไม่ครบ, Noise สูง, Source หาย, Graph ขยายเกิน Budget และคำตอบอ้างเกิน Context

หาก Retrieval ไม่คืนหลักฐานที่ต้องใช้ การแก้ Prompt ปลายทางไม่สามารถสร้างข้อเท็จจริงที่หายไปได้ หลักสำคัญของบทนี้คือ:

Retrieval ไม่ได้มีหน้าที่หาข้อความที่ดูคล้ายที่สุด แต่มีหน้าที่สร้างชุดหลักฐานที่เล็กพอให้โมเดลโฟกัสและครบพอให้ตอบโดยไม่เผลอ

Retrieval ตอบคำถามว่า “ควรนำความรู้ภายนอกชิ้นใดเข้ามาตอนนี้” แต่ยังไม่ตอบว่า “ข้อมูลใดจากบทสนทนาก่อนหน้านี้ควรคงอยู่” บทถัดไปจึงเปลี่ยนจากการค้น Evidence ภายนอกไปสู่ Memory Compaction และ Note-taking สำหรับรักษาความต่อเนื่องข้าม Turn

เอกสารอ้างอิง

- [1] Thongtae, Aekanun. “Hybrid Retrieval, Graph Traversal, and MCP Tool Surfaces.” *Author’s Lecture Transcript*, July 2026.
- [2] Thongtae, Aekanun. *2026-AI-Testing-and-Evaluation*, commit [9eb207dc](#), GitHub repository, 2026.

บทที่ 10 — Memory Engineering: จำสิ่งสำคัญโดยไม่ทำให้ Context เหน้

Memory จากโค้ด Lab 7: History → Compaction → Notes → Context

agent_memory.py · Pure Python · โค้ดที่ใช้สอน

```
30 class ConversationMemory:
33     self.history: list[dict] = []
34     self.notes: list[str] = []
37     self.history.append(message)

42 def context(self):
44     sys_msg = {"role": "system", "content": SYSTEM}
46     sys_msg["content"] += "[บันทึกที่ต้องจำ]" + notes
47     return [sys_msg] + self.history

49 def maybe_compact(self):
51     if len(history) < COMPACT_AFTER_MESSAGES: return
54     keep = self.history[-4:]
55     old = self.history[:-4]
58     summary = llm.chat(... old ...)
62     self.history = [summary] + keep

68 mem.add({"role": "user", "content": question})
70 llm.chat(messages=mem.context(), tools=...)
82 mem.maybe_compact() # หลังจบแต่ละ turn
```

1 แยก History กับ Notes

History โดตามบทสนทนา
Notes คือ Fact ที่ต้องรอดจาก Compaction

2 ประกอบ Complete Context

System + Notes + History ถูกสร้างใหม่
แล้วส่งเข้า LLM ทุกครั้ง
LLM จึงดูเหมือนจำ Turn ก่อนหน้าได้

3 Summary + Raw Tail

เก็บ 4 Messages ล่าสุดแบบไม่ย่อ
ให้ LLM สรุปส่วนที่เก่ากว่า
จากนั้นเขียน History ใหม่
ค่า 4 และ Threshold ต้องทดลองจริง
Structured State ไม่ควรถูกฝากใน Summary

4 ทำงานร่วมกับ Agent Loop

เพิ่ม Message → ใช้ Context → Compact
เมื่อ Turn จบ เพื่อลด Context Bloat

Source: labs/lab7_memory/agent_memory.py · ตัดรายละเอียด โดเมนและข้อมูลตัวอย่างออกจากภาพ

Memory Compaction และ Note-taking จากโค้ด Lab 7

LLM ไม่มีความทรงจำต่อเนื่องอยู่ในตัวเอง ทุกครั้งที่เรียกโมเดล มันเห็นเฉพาะ Context ที่ระบบส่งเข้าไป หาก Agent ดูเหมือนจำบทสนทนาท่อนหน้าได้ นั่นเป็นเพราะ Harness เก็บ Message History แล้วส่งกลับเข้าโมเดลอีกครั้ง

ปัญหาคือการส่ง History ทั้งหมดกลับไปเรื่อย ๆ ไม่สามารถขยายได้ไม่สิ้นสุด Memory Engineering จึงไม่ใช่แค่การ “เก็บ” แต่ต้องตัดสินใจว่าอะไรควรเก็บแบบเต็ม อะไรควรย่อ อะไรควรตัด และอะไรสำคัญจนต้องแยกไว้นอก History

Context Window Exceeded, Bloat และ Rot ไม่ใช่เรื่องเดียวกัน

ในการบรรยาย ผมแยกปัญหาของ Context ออกเป็นสามระดับ

- **Context Window Exceeded** — ข้อมูลมากเกินไปขนาดที่โมเดลรับได้ ส่วนที่เกินอาจถูกตัดหรือทำให้ Request ล้มเหลว
- **Context Bloat** — Context ยังไม่เกินไปขนาด แต่มีข้อมูลมากและฟุ่มเฟือย ทำให้ช้าลงและมีต้นทุนสูงขึ้น

- **Context Rot** — Context ใหญ่จนความสามารถในการจับประเด็นและความสัมพันธ์เริ่มลดลง แม้ Request ยังส่งเข้าโมเดลได้

เมื่อ Token จำนวนมากแข่งขันกัน ใน Attention ข้อมูลสำคัญอาจได้รับน้ำหนักเจือจาง โดยเฉพาะข้อมูลที่ถูกดันไปอยู่กลางบทสนทนา ซึ่งเชื่อมกับปรากฏการณ์ Lost in the Middle ดังนั้น Context Window ที่ใหญ่ไม่ใช่ใบอนุญาตให้ใส่ทุกอย่าง

ตัวเลขสัดส่วน Context ที่เหมาะสมไม่ควรถูกมองเป็นค่าคงที่สากล ต้องทดลองกับ โมเดล ประเภทงาน และรูปแบบข้อมูลของระบบจริง หลักสำคัญคือวัด Token, คุณภาพคำตอบ, Latency และ Cost พร้อมกัน

ในการบรรยาย Memory Compaction ไม่ได้เริ่มจากคำถามว่า “จะทำให้ Agent จำได้มากขึ้นอย่างไร” แต่เริ่มจาก Context ที่บวมจนคุณภาพตก ลำดับเหตุผลคือ History โต → Tool Results เก่าสะสม → สัญญาณสำคัญเจือจาง → Lost in the Middle → จำเพี้ยน → จึงต้อง Prune, Retrieve แบบ Just in Time หรือ Compact การเริ่มจาก Causal Chain นี้ช่วยไม่ให้ทีมติดตั้ง Memory Store แล้วส่งทุกอย่างที่ค้นได้กลับเข้า Context ซึ่งจะสร้างปัญหาเดิมในรูปแบบใหม่ [1]

Memory ที่ดีจึงวัดสองด้านพร้อมกัน: **Retention** รักษาสิ่งที่ห้ามหาย และ **Selectivity** ไม่นำทุกสิ่งที่เคยเก็บกลับมาในทุก Turn หากมีแต่ Retention ระบบจะบวม หากมีแต่ Selectivity ระบบอาจลิม Constraint สำคัญ Harness ต้องพิสูจน์ทั้งสองด้านด้วย Test ที่ต่างกัน

สื่กลยุทธ์จัดการ Context

1. **Pruning** — ตัดข้อมูลที่หมดประโยชน์ เช่น Error เก่าหรือผล Tool ระหว่างทางที่มีผลสรุปใหม่กว่าแล้ว
2. **Just-in-Time Retrieval** — ยังไม่โหลดข้อมูลจนกว่าจะต้องใช้ แล้วดึงมาเฉพาะส่วนที่เกี่ยวข้อง
3. **Isolate** — แยก State หรือ Fact สำคัญออกจาก History เพื่อไม่ให้ถูกย่อหรือลืม
4. **Compaction** — สรุปส่วนเก่าให้เล็กลงโดยรักษาข้อเท็จจริง การตัดสินใจ และสถานะงานที่จำเป็น

วิธีนี้ทำหน้าที่คนละอย่าง Retrieval ลดสิ่งที่เข้ามาตั้งแต่ต้น Pruning เอาส่วนเกินออก Isolate ป้องกันข้อมูลสำคัญ และ Compaction ลดขนาดสิ่งที่ยังต้องเก็บ

อ่านจากโค้ดจริง: ConversationMemory

โค้ด Lab 7 ที่ใช้สอนสร้าง Memory ด้วย Pure Python เพื่อให้เห็นกลไกโดยไม่ซ่อนอยู่หลัง Framework [3]

```
class ConversationMemory:
    def __init__(self):
        self.history: list[dict] = []
        self.notes: list[str] = []

    def add(self, message: dict):
        self.history.append(message)

    def add_note(self, fact: str):
        self.notes.append(fact)
```

history เก็บ User Message, Assistant Message, Tool Call และ Tool Result ข้าม Turn ส่วน **notes** เก็บ Fact ที่ต้องคงอยู่แม้ History จะถูก Compact ทั้งสองส่วนยังเป็น In-memory หาก Process ปิดข้อมูลจะหายไป จึงเป็น Conversation Memory ข้าม Turn แต่ยังไม่ใช้ Persistent Memory ข้าม Process

ประกอบ Context ใหม่ทุกครั้ง

ก่อนเรียก LLM เมธอด `context()` จะประกอบ System Prompt, Notes และ History เป็น Complete Context

```
def context(self) -> list[dict]:
    sys_msg = {"role": "system", "content": SYSTEM}
    if self.notes:
        sys_msg["content"] += (
            "\n\nบันทึกที่ต้องจำ\n- "
            + "\n- ".join(self.notes)
        )
    return [sys_msg] + self.history
```

ตำแหน่งของ Note มีความหมาย เพราะข้อมูลถูกนำไปต่อกับ System Prompt ก่อน History ทุกครั้ง วิธีนี้ช่วยให้ Fact สำคัญไม่ถูกดันหายไปพร้อมบทสนทนาเก่า แต่ก็หมายความว่า Note ทุกข้อกิน Token ทุกรอบ จึงต้องมีนโยบายเพิ่ม แก่ และลบ Note

Compaction: เก็บส่วนล่าสุดแบบเต็ม สรุปลส่วนเก่า

หัวใจของ Lab อยู่ที่ `maybe_compact()` เมื่อ History ยาวถึงเกณฑ์ โค้ดแยกข้อมูลออกเป็นสองส่วน

```
if len(self.history) < COMPACT_AFTER_MESSAGES:
    return

keep = self.history[-4:]
old = self.history[:-4]
```

keep คือสี่ข้อความล่าสุดที่เก็บครบทุกตัวอักษร เพราะมักเกี่ยวข้องกับงานปัจจุบันมากที่สุด ส่วน **old** คือข้อความก่อนหน้านั้นที่จะถูกส่งให้ LLM สรุป ค่า 4 ไม่ใช่เลขวิเศษ ต้องทดลองเปลี่ยนตามความลึกของงานและจำนวน Tool Calls ต่อ Turn

ก่อนสรุป โค้ดเลือกเฉพาะ Message ที่มี **content**:

```
transcript = "\n".join(
    f'{m["role"]}: {m.get("content", "")}'
    for m in old
    if m.get("content")
)
```

นี่ทำหน้าที่คล้าย Tool-result clearing เบื้องต้น เพราะก่อน Tool Call ที่ไม่มีข้อความจะไม่ถูกนำไปสรุป แต่ยังคงระวังว่าผล Tool ที่มีข้อความยาวจะติดไปด้วย Production Harness ควรเลือกเก็บเฉพาะผลสุดท้าย Record ID ข้อเท็จจริงที่ยืนยันแล้ว และ Error ที่ยังมีผลต่อสถานะงาน

จากนั้น LLM สรุปส่วนเก่า แล้ว Harness เขียน History ใหม่เป็น `summary + keep`

```
summary = llm.chat(messages=[
    {"role": "system", "content":
        "สรุปบทสนทนาต่อไปนี้เป็นย่อหน้าเดียว "
        "เก็บข้อเท็จจริงสำคัญไว้"},
    {"role": "user", "content": transcript},
], max_tokens=300).choices[0].message.content

self.history = [
    {"role": "assistant",
     "content": f"[สรุปบทสนทนาก่อนหน้า] {summary}"},
] + keep
```

เป้าหมายของ Compaction ไม่ใช่ย่อให้สั้นที่สุด แต่ลด Token โดยไม่ทำ State สำคัญหาย ความเสี่ยงคือ LLM อาจสรุปตกหรือสรุปผิด จึงไม่ควรใช้ Summary เป็นที่เก็บข้อมูลที่ต้องแม่นยำ เช่น Approval, จำนวนเงิน, Identifier หรือสถานะ Transaction ข้อมูลเหล่านี้ควร Isolate เป็น Structured State

หลักการนี้สอดคล้องกับแนวทาง Context Engineering สำหรับงานระยะยาว ซึ่งแยก Compaction, Structured Note-taking และ Just-in-Time Retrieval ออกจากกัน การล้าง Tool Result เก่าคือทางเลือกที่เบาและปลอดภัยกว่าการสรุปทุกอย่างใหม่ ขณะที่ Compaction ต้องเริ่มโดยเน้น Recall ของรายละเอียดสำคัญก่อน แล้วจึงปรับ Precision เพื่อตัดส่วนเกิน [2]

Note-taking: สิ่งห้ามหายไปกับ Summary

Note-taking แก้ปัญหาดนลแบบกับ Compaction

- Compaction รับ History เก่าแล้วสังเคราะห์เป็น Summary ใหม่
- Note-taking รับ Fact ที่ผู้ใช้ ผู้พัฒนา หรือ Policy ระบุว่าควรจำ และเก็บแยกจาก History
- Compaction เขียนทับส่วนเก่า ส่วน Notes เพิ่มเติมเนื่องจากจะมีกลไกแก้ไขหรือลบ

ตัวอย่างที่เหมาะสมกับ Note ได้แก่คำศัพท์เฉพาะ รูปแบบรายงานที่ผู้ใช้งานต้องการ หรือข้อจำกัดประจำวัน ส่วนข้อมูลชั่วคราวและผล Tool ที่ตรวจใหม่ได้ไม่ควรถูกยกเป็น Note ถาวรโดยอัตโนมัติ

Memory ทำงานร่วมกับ Agent Loop อย่างไร

ใน `turn()` ทุก Message ถูกเพิ่มเข้า Memory และทุกครั้งที่เรียก โมเดลจะใช้ `mem.context()`

```
mem.add({"role": "user", "content": question})

resp = llm.chat(
    messages=mem.context(),
    tools=registry.openai_tools,
)

mem.add({
    "role": "tool",
    "tool_call_id": call.id,
    "content": result,
})
```

เมื่อ Agent ตอบจบ Turn จึงเรียก `mem.maybe_compact()` ทำให้คำถามถัดไปอ้างอิงสิ่งที่เพิ่งกล่าวไปได้ ขณะเดียวกัน History เก่าไม่โตขึ้นอย่างไม่มีเพดาน

สิ่งที่โค้ด Lab ตั้งใจทำให้ง่าย

โค้ดนี้เหมาะกับการเรียนรู้ แต่ก่อนใช้จริงยังต้องเติมอย่างน้อยสี่เรื่อง

- นับ Token จริงแทนการนับจำนวน Message
- เก็บ Structured State แยกจาก Summary ที่ LLM สร้าง
- Persist History และ Notes พร้อม Session/User Scope
- มีนโยบายแก้ไข ลบ หมดอายุ และป้องกันข้อมูลอ่อนไหว

Compaction เป็นกระบวนการแบบ Lossy ต่อให้ Summary อ่านลื่นก็อาจทำรายละเอียดหาย จำนวนข้อความล่าสุดที่เก็บหรือคำแนะนำเชิงสัดส่วนของ Context จากชั้นเรียนจึงเป็นค่าเริ่มทดลอง ไม่ใช่มาตรฐานตายตัว ระบบจึงควรหาค่าเหมาะสมด้วย Evaluation Set ของงานตนเอง

กรอบต่อยอดเชิงวิศวกรรม

การแบ่งต่อไปนี้เป็นกรอบสังเคราะห์สำหรับออกแบบระบบ ไม่ใช่การแบ่งประเภทที่ยกมาจากคำบรรยายโดยตรง

- **Working Memory** — History, Tool Results และ State ของงานปัจจุบัน
- **Episodic Memory** — บันทึกว่า Run ก่อนเกิดอะไรขึ้น ทำอะไร และได้ผลอย่างไร
- **Semantic Memory** — Fact หรือความรู้ที่สกัดและผ่านการตรวจสอบแล้ว

ไม่ว่าจะเรียก Memory ชั้นใด วงจรที่ต้องออกแบบยังคงเป็น `write` → `store` → `retrieve` → `validate` → `update/delete` ระบบที่จำได้แต่แก้ไขหรือลืมไม่ได้ จะสะสมทั้งข้อมูลล้าสมัย ความขัดแย้ง และความเลื่องด้านความเป็นส่วนตัว

Memory Engineering ที่ดีจึงไม่วัดจากจำนวนข้อมูลที่จำ แต่จากความสามารถในการนำสิ่งที่จำเป็นกลับมาใช้ในเวลาที่เหมาะสม โดยไม่ทำให้ Context บวม ไม่ทำ State สำคัญหาย และยังอธิบายได้ว่าความทรงจำนั้นมาจากไหน

Harness Design Check

- **Harness รับผิดชอบ:** เลือกสิ่งที่จะเขียน เก็บ ดึง ตรวจสอบ Compact และลบ
- **Model ตัดสินใจ:** เสนอ Fact หรือ Summary ที่ควรจำ และใช้ Memory ในการตอบ
- **Evidence ที่ต้องเก็บ:** Source, Timestamp, Scope, Version, Retrieval reason และการแก้ไข/ลบ
- **Failure ที่ต้องทดสอบ:** ความจำเก่า, Fact ขัดกัน, ข้ามขอบเขตผู้ใช้, Context Rot และข้อมูลที่ลืมไม่ได้

เมื่อ Harness จัด Tool, State, Retrieval และ Memory ครบแล้ว คำถามสุดท้ายคือเราจะพิสูจน์อย่างไรว่าทั้งระบบทำงานถูก บทถัดไปจึงเปลี่ยน Trace ให้เป็น Detector, Evaluation และ Gate ที่หยุดการ Deploy ได้จริง

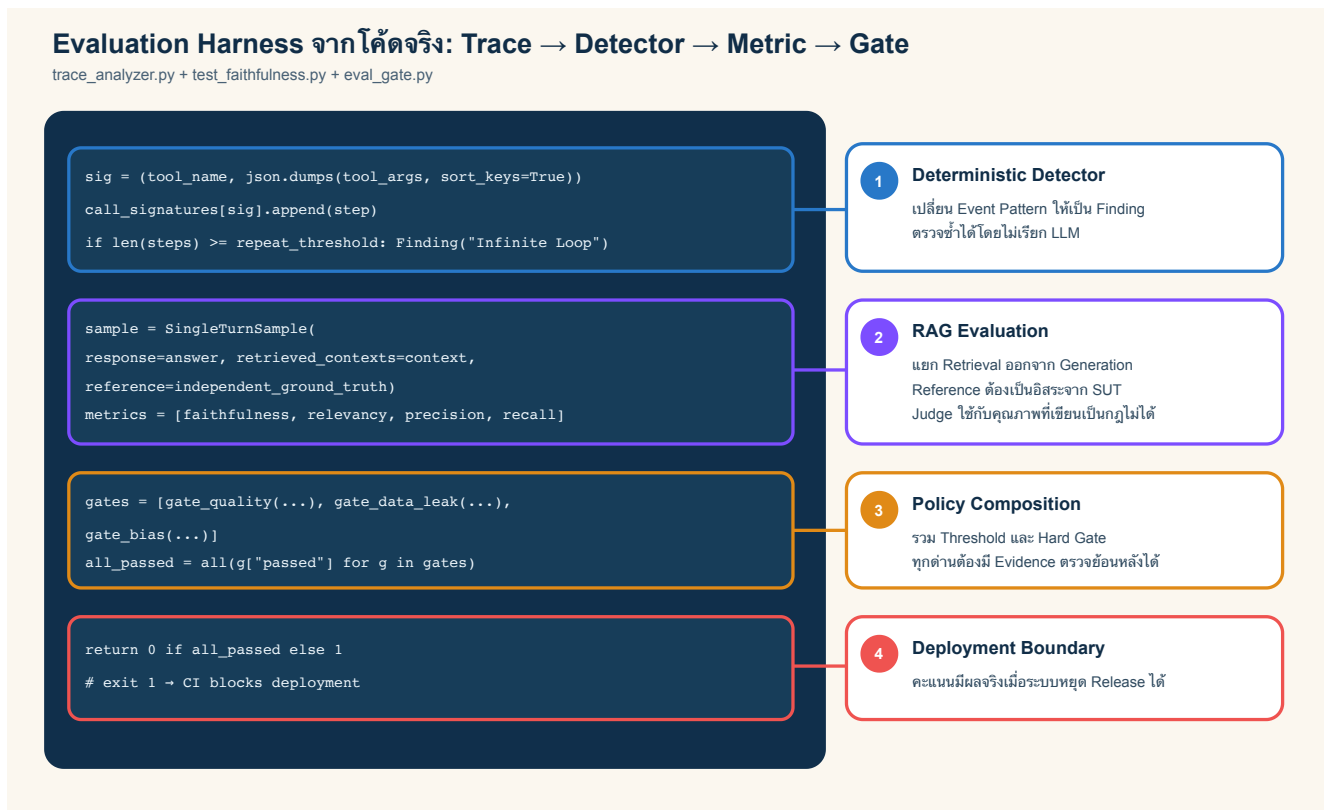
เอกสารอ้างอิง

[1] Thongtae, Aekanun. “Context Rot, Memory Compaction, and Note-taking.” *Author’s Lecture Transcript*, July–August 2026.

[2] Anthropic. “Effective Context Engineering for AI Agents.” September 2025.

[3] Thongtae, Aekanun. *Python-Agent-LangGraph*, [labs/lab7_memory/agent_memory.py](#), commit [c3f95c3](#), GitHub repository, 2026.

บทที่ 11 — Evaluation Engineering: จาก Trace สู่ Deployment Gate



Evaluation Harness จาก Trace สู่ Deployment Gate

Agent สามารถเรียก Tool สำเร็จและตอบอย่างมั่นใจ แต่ยังเลือก Tool ผิด วนซ้ำ ใช้หลักฐานไม่ครบ เปิดเผยข้อมูล หรือสร้างข้ออ้างที่ไม่ปรากฏใน Observation ได้ Evaluation ของ Agent จึงต้องตรวจทั้งเส้นทาง ไม่ใช่ดูเพียง Final Answer หรือสถานะว่าโปรแกรมไม่ Crash

ภาพเปิดบทตัดรายละเอียดโดเมนและย่อชื่อบางตัวเพื่อให้เห็นโครงสร้างในหน้าเดียว แต่กลไกทั้งสองช่วง—Detector, RAG Evaluation, Policy Composition และ Exit Code—มาจาก Source ที่ใช้สอนจริง ส่วน Snippet ในเนื้อหาจะรักษาโครงสร้างสำคัญที่ใช้อธิบาย [3]

Trace คือวัตถุดิบของการตรวจ

Production Log มักเขียนไว้เพื่อให้คนอ่าน แต่ Evaluation Trace ต้องรักษาโครงสร้างที่โปรแกรมวิเคราะห์ต่อได้ เช่น `event_type`, `tool_name`, `tool_args`, `content`, `tokens_used` และ `step`

โค้ด `trace_analyzer.py` แบ่งงานเป็นสามชั้น:

1. **Parser Layer** อ่าน Event โดยยังไม่ตัดสิน
2. **Detector Layer** ตรวจ Pattern ของ Failure แต่ละชนิด
3. **Coverage Layer** สรุปว่า Failure Mode ใดมี Test ครอบคลุม

การแยกนี้สำคัญ เพราะ Trace ชุดเดียวสามารถถูกตรวจด้วย Detector ใหม่ได้โดยไม่ต้องรัน Agent ซ้ำ และ Detector แต่ละตัวทดสอบแยกจาก LLM ได้

คำบรรยายแยก **Error** ออกจาก **Failure** อย่างมีนัยสำคัญ Error เป็นเหตุการณ์ภายในที่ระบบอาจฟื้นตัวได้ แต่ Failure หมายถึงพฤติกรรมผิดได้หลุดไปสร้างผลกระทบแล้ว เช่นผู้ใช้เห็นคำตอบ Hallucination หรือ Agent เรียก Tool ผิดลำดับ การมี Error Handling จึงยังไม่เท่ากับมี Assurance เพราะระบบอาจรันจบโดยไม่มี Exception แต่ให้ผลผิดได้ [1]

ทำไม Parser ต้องแยกจาก Detector

แหล่ง Observability ต่างกันอาจใช้ Trace, Span และ Event คนละ Schema Parser มีหน้าที่แปลงความต่างเหล่านั้นเป็น Event Object กลาง ส่วน Detector อ่าน Object กลางเพื่อตรวจ Failure Pattern เมื่อเปลี่ยนผู้ให้บริการ Log ทีมจึงแก้ Parser โดยไม่ต้องเขียน Detector ทั้งชุดใหม่ และสามารถ Replay Traceเก่าเพื่อทดสอบ Detector Version ใหม่ได้

Detector เปลี่ยนพฤติกรรมให้เป็น Assertion

ตัวตรวจ Infinite Loop สร้าง Signature จากชื่อ Tool และ Arguments แล้วนับการเรียกซ้ำ:

```
sig = (tool_name, json.dumps(tool_args, sort_keys=True))
call_signatures[sig].append(step)

if len(steps) >= repeat_threshold:
    findings.append(Finding(failure_mode="Infinite Loop", ...))
```

การเรียก Tool เดิมสองครั้งอาจเป็นการลองใหม่ที่สมเหตุผล แต่เมื่อ Signature เดิมเกิดซ้ำถึง Threshold ระบบสามารถ Flag ได้โดยไม่ต้องถาม LLM ว่า “นี่คือ Loop หรือไม่” หลักเดียวกันใช้กับ Tool Misuse, Cost Runaway, Prompt Injection และ Sensitive Data Leakage

ตัวตรวจ Hallucination ในโค้ดจริงตรวจสอบสองชั้น ชั้นแรกตรวจสอบข้ออ้างที่เป็นไปไม่ได้ตามโครงสร้างข้อมูล ชั้นที่สองตรวจ Grounding ว่าข้ออ้างใน Final Answer เคยปรากฏใน Observation ของ Run เดียวกันหรือไม่ ดังนั้น “ข้อเท็จจริงนั้นมีอยู่จริง” กับ “Agent มีหลักฐานพอที่จะกล่าวใน Run นี้” เป็นคนละคำถาม

Coverage Map ไม่ใช่ Code Coverage

การมี Test จำนวนมากยังไม่บอกว่า Failure สำคัญถูกตรวจครบหรือไม่ Coverage Map จึงควรจัดตาม Failure Mode เช่น:

- Hallucination
- Tool Misuse
- Infinite Loop
- Prompt Injection
- Sensitive Data Leakage

- Cost Runaway

รายการนี้ทำหน้าที่เป็น Risk Coverage: ทุก Failure Mode ต้องโยงกลับไปหา Dataset, Detector, Test และ ผลล่าสุดได้ เมื่อระบบเพิ่ม Tool หรือสิทธิ์ใหม่ Coverage Map ต้องเปลี่ยนตาม Threat Surface

ค่า `covered=true` ควรแปลงอย่างระมัดระวังว่า “มี Test Case ที่ตั้งใจออกแบบสำหรับความเสี่ยงนี้” ไม่ได้แปลว่า Detector จับได้ครบทุก Variant และไม่ได้แปลว่าระบบปลอดภัยแล้ว Coverage Map จึงควรเก็บอย่างน้อย `failure_mode`, `test_case_id`, `expected_finding`, `detector_version` และ `last_result` เพื่อแยกการมี Test ออกจากการผ่าน Test

Golden Set ต้องไม่เจลดจากแหล่งเดียวกับ SUT

การประเมิน RAG ต้องแยกวัตถุสี่ชิ้น:

- **Knowledge Base** — ข้อมูลที่ระบบมีสิทธิ์ใช้ตอบ
- **Golden Reference** — คำตอบหรือหลักฐานที่ผู้ประเมินกำหนดอย่างอิสระ
- **Retrieved Context** — Evidence ที่ Retrieval ดึงมาใน Run นั้น
- **Actual Answer** — คำตอบที่ Agent สร้างจริง

โค้ดทดสอบเลือก `ground_truth` ก่อน และเตือนเมื่อจำเป็นต้องใช้ Corpus เดียวกับ SUT เป็น Fallback เพราะการใช้แหล่งเดียวกันทั้งตอบและเจลดอาจทำให้คะแนนสูงเกินจริง

```
reference = case.get("ground_truth")
if not reference:
    reference = load_corpus_fallback(...)
    warnings.warn("evaluation is not independent")
```

แยก Retrieval Failure ออกจาก Generation Failure

RAG Evaluation ใช้เมตริกสี่มิติ:

- `context_precision` — Context ที่ดึงมามี Noise มากเพียงใด
- `context_recall` — Evidence ที่จำเป็นถูกดึงมาครบหรือไม่
- `faithfulness` — ข้ออ้างในคำตอบมี Grounding ใน Context หรือไม่
- `answer_relevancy` — คำตอบตรงกับคำถามหรือไม่

สองตัวแรกชี้ปัญหาที่ Retrieval Harness ส่วนสองตัวหลังชี้ปัญหาที่ Generation การรวมทั้งหมดเป็นคะแนนเดียวเร็วเกินไปจะทำให้ทีมไม่รู้ว่าควรแก้ส่วนไหน

Deterministic First, Judge When Necessary

สิ่งที่เขียนเป็นกฎได้ควรตรวจด้วยโค้ดก่อน เช่น Schema, Tool Order, Duplicate Write, Permission, Token Budget และข้อมูลที่ห้ามปรากฏ ส่วนคุณภาพที่เป็นนามธรรมจึงค่อยใช้ LLM-as-a-Judge พร้อม Rubric และ Golden Reference

Judge ไม่ใช่ผู้พิพากษาสูงสุดโดยอัตโนมัติ ต้องบันทึก Judge Model, Prompt, Rubric, Threshold และ Version เพราะการเปลี่ยนองค์ประกอบใดองค์ประกอบหนึ่งทำให้คะแนนเปลี่ยนได้ แม้ SUT ไม่เปลี่ยน

Evaluation Gate ทำให้คะแนนมีผลต่อระบบ

รายงานที่ไม่มีผลต่อการปล่อยระบบเป็นเพียงข้อมูลประกอบ `eval_gate.py` เปลี่ยนผลประเมินหลายด้านให้เป็นคำตัดสินที่เครื่องอ่านได้:

```
gates = [gate_quality(min_pass_rate)]
gates.append(gate_sensitive_data_leak(...))
gates.append(gate_bias(threshold))

all_passed = all(gate["passed"] for gate in gates)
return 0 if all_passed else 1
```

บาง Metric ใช้ Threshold เช่นอัตราเสถียรภาพที่ผ่านขั้นต่ำ แต่บางความเสี่ยงควรเป็น Hard Gate เช่นข้อมูลอ่อนไหวรั่วแม้เพียงเคสเดียว การกำหนด Policy เป็นหน้าที่ของ Harness และต้องสอดคล้องกับผลกระทบของระบบ ไม่ควรใช้ Threshold เดียวกับทุก Use Case

Test ก่อน Deploy และ Observe หลัง Deploy

Offline Evaluation ช่วยเปรียบเทียบ Version กับ Golden Set ก่อนปล่อย ส่วน Online Observability ช่วยค้นหา Drift, Failure ใหม่ และเคสจริงที่ควรถูกนำกลับมาเพิ่มในชุดทดสอบ วงจรจึงเป็น:

```
Observe → Curate → Test → Gate → Deploy → Observe
```

ต้องปกปิด Secret และข้อมูลอ่อนไหวใน Trace พร้อมรักษา `run_id`, `tool_call_id`, Timestamp, Latency, Token Usage, Error Code, Model Version และ Source IDs ให้พอสำหรับการตรวจย้อนหลัง

Eval ก่อน Improve: เริ่มจาก Failure จริง

การปรับ Prompt, Model หรือ Harness โดยยังไม่มี Baseline ทำให้ทีมรู้เพียงว่าระบบ “เปลี่ยน” แต่ไม่รู้ว่า “ดีขึ้น” Anthropic แนะนำว่า Eval เริ่มต้นไม่จำเป็นต้องมีหลายร้อยงาน ชุดงานง่าย 20–50 งานซึ่งดึงจากความล้มเหลวจริงก็สร้างสัญญาณสำหรับระบบระยะแรกได้ [2]

Real failure → Minimal reproducible task → Reference solution
→ Grader → Regression suite → Deployment gate

แต่ละ Task ต้องไม่กำกวมและควรมี Reference Solution ที่พิสูจน์ว่าโจทย์แก้ได้และ Grader ทำงานถูก ชุดทดสอบต้องสมดุลทั้ง Positive และ Negative Cases เช่น ทดสอบว่า Agent เรียก Search เมื่อจำเป็น และไม่เรียกเมื่อข้อมูลที่มีเพียงพอ มิฉะนั้นการ Optimize เพื่อลด Miss อาจกลายเป็น Agent ที่เรียก Search ทุกครั้ง

Eval Environment ก็เป็นส่วนของ Harness: แต่ละ Trial ควรเริ่มจาก State ที่สะอาด ลด Cached data หรือไฟล์ค้างที่ทำให้ผลรอบต่าง ๆ พังพากัน และควรเลือก Deterministic Grader ก่อนเมื่อ Outcome ตรวจสอบด้วยโค้ดได้ ส่วน LLM Judge ใช้เมื่อเกณฑ์ต้องอาศัยความหมาย พร้อม Calibration เทียบมนุษย์และทางเลือก **Unknown** เมื่อหลักฐานไม่พอ [2]

Harness Design Check

- **Harness รับผิดชอบ:** เก็บ Trace, รัน Detector/Judge, คำนวณ Metric, บังคับ Threshold และหยุด Deploy
- **Model ตัดสินใจ:** ตอบหรือเลือก Tool ภายใน SUT; เมื่อเป็น Judge ให้ประเมินเฉพาะ Rubric ที่กำหนด
- **Evidence ที่ต้องเก็บ:** Dataset version, SUT/Judge version, Trace, Scores, Findings, Threshold และ Gate decision
- **Failure ที่ต้องทดสอบ:** Hallucination, Tool Misuse, Loop, Injection, Data Leakage, Cost Runaway, Retrieval miss และ Judge drift

AI Harness Engineering จึงไม่ได้จบเมื่อ Agent ทำงานได้ แต่จบเมื่อเรารู้ว่ามันได้รับข้อมูลอะไร ตัดสินใจอย่างไร ลงมือผ่านขอบเขตใด เกิดผลจริงอะไร และผ่านหลักฐานชุดใดจึงได้รับอนุญาตให้ทำงานต่อ ความไม่แน่นอนของโมเดลยังอยู่ แต่ระบบรอบมันทำให้ความไม่แน่นอนนั้นมีขอบเขต วัดได้ และรับผิดชอบได้

เอกสารอ้างอิง

[1] Thongtae, Aekanun. “Trace Parsing, Failure Detection, and Evaluation Coverage.” *Author’s Lecture Transcript*, July 2026.

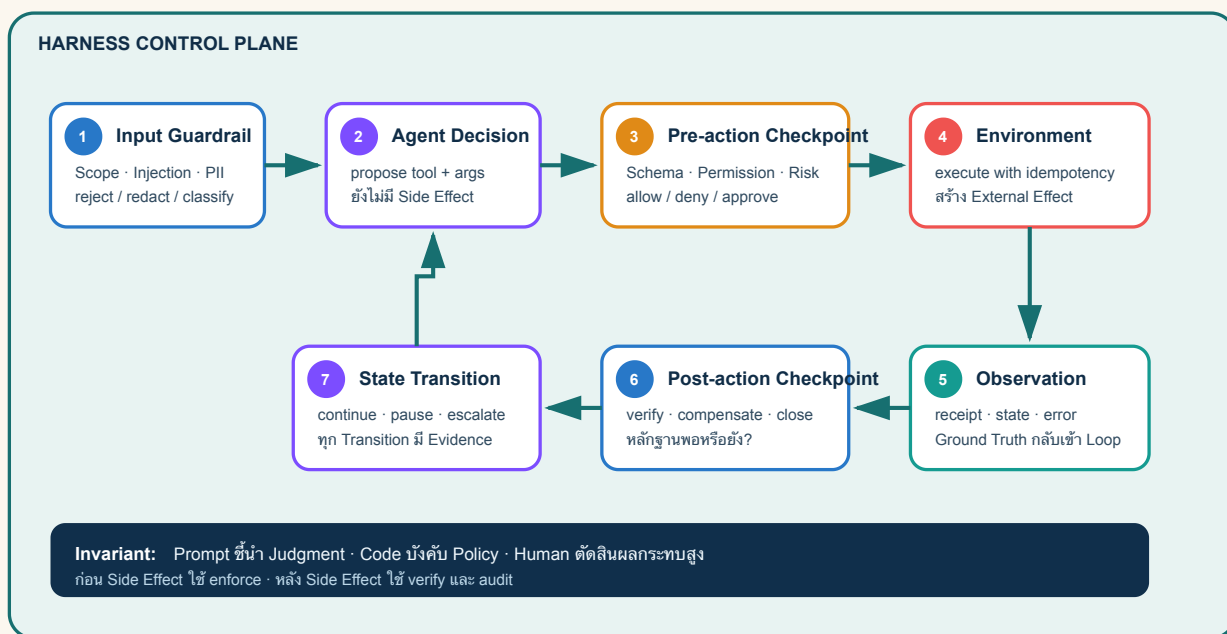
[2] Anthropic. “Demystifying Evals for AI Agents.” 2026.

[3] Thongtae, Aekanun. *2026-AI-Testing-and-Evaluation*, commit [9eb207dc](#), GitHub repository, 2026.

บทที่ 12 — Control, Guardrails และ Checkpoints: บังคับขอบเขตก่อนผลกระทบเกิดขึ้น

Control Plane: Guardrail และ Checkpoint รอบ Agent Loop

Model เสนอการกระทำ · Harness บังคับขอบเขต · Environment ยืนยันผลจริง



Control Plane รอบ Agent Loop แสดง Guardrail และ Checkpoint ก่อนและหลัง Side Effect

ภาพเปิดบททาง Agent Loop ไว้กลางระบบ แต่เพิ่ม Control Plane คร่อมทุก Boundary โมเดลยังเสนอการกระทำได้อย่างยืดหยุ่น ทว่าโค้ดเป็นผู้ตรวจ Input, Tool Call, Permission, Budget และหลักฐานก่อนอนุญาตให้โลกภายนอกเปลี่ยน

Control ไม่ใช้การเขียนคำเตือนใน Prompt

Prompt ช่วยชี้นำพฤติกรรม แต่ไม่ใช่จุดบังคับ หากกฎว่า “ห้ามลบข้อมูลโดยไม่ได้รับอนุมัติ” อยู่ใน System Prompt เพียงแห่งเดียว ความปลอดภัยยังขึ้นกับการทำตามคำสั่งของโมเดล Control ที่บังคับได้ต้องอยู่ใน Runtime Boundary ซึ่งสามารถ Allow, Deny, Transform, Pause หรือ Escalate ได้โดยไม่ต้องขอให้โมเดลเลือกใช้ [1]

หลักแบ่งความรับผิดชอบคือ:

- ให้โมเดลใช้ Judgment เมื่องานต้องตีความ
- ให้โค้ดบังคับ Invariant ที่ห้ามผิด
- ให้นุชนัยตัดสินเมื่อผลกระทบสูงหรือหลักฐานไม่พอ

Guardrail อยู่ตรงไหนของวงจร

คำว่า Guardrail มีความหมายกว้าง จึงควรระบุทั้งตำแหน่งและอำนาจของมัน

Boundary	สิ่งที่ตรวจ	การตอบสนอง
Input	รูปแบบ, Injection, Scope, ข้อมูลอ่อนไหว	Reject, redact หรือ classify
Before model	Policy context, tenant, budget	Inject หรือ restrict
Before tool	Schema, permission, risk, approval	Allow, deny, modify หรือ pause
After tool	Result contract, side effect, leakage	Normalize, quarantine หรือ compensate
Before final	Evidence coverage, disclosure policy	Revise, abstain หรือ escalate
Before stop	Completion condition, unresolved state	Continue หรือ close run

Guardrail หลัง Tool เหมาะกับ Audit และตรวจ Result แต่เข้าไปสำหรับ Authorization เพราะ Side Effect เกิดขึ้นแล้ว กฎที่ป้องกันความเสียหายต้องอยู่ก่อน Execution Boundary

Checkpoint คือจุดตัดสินใจ ไม่ใช่จุดหยุดทุกครั้ง

Checkpoint รับ State และ Evidence แล้วตัดสินใจว่า Run ควรเดินหน้า หยุด ขออนุมัติ หรือเปลี่ยนเส้นทาง Anthropic อธิบายว่า Agent ควรรับ Ground Truth จากสภาพแวดล้อมทุกชั้น และสามารถหยุดรับ Human Feedback เมื่อถึง Checkpoint หรือพบ Blocker [2]

Checkpoint ที่ทดสอบได้ควรมี Contract สี่ส่วน:

Trigger → Required Evidence → Decision Authority → Next State

ตัวอย่างก่อนการเขียนข้อมูล:

```
{
  "trigger": "before_external_write",
  "required_evidence": ["validated_payload", "permission", "preview"],
  "decision_authority": "policy_or_human",
  "next_states": ["approved", "denied", "needs_revision"]
}
```

การเขียนว่า “ถามผู้ใช้เมื่อจำเป็น” ไม่มี Contract เพราะโมเดลต้องตีความคำว่า “จำเป็น” เองทุกครั้ง ในทางกลับกัน Rule ที่ระบุ Risk level, Approval threshold และ Evidence requirement สามารถ Unit Test และ Audit ได้

Risk-tiered Control

การกระทำไม่ควรถูกควบคุมเท่ากันทั้งหมด มีงาน Read ธรรมดาจะเข้าไป ขณะที่ Write สำคัญอาจถูกป้องกันไม่พอ

1. **Observe** — อ่านข้อมูลที่ไม่อ่อนไหว ใช้ Logging และ Rate Limit
2. **Constrain** — จำกัด Query, Scope, Token และ Result size
3. **Confirm** — แสดง Preview ก่อน Write ที่ย้อนกลับได้
4. **Approve** — ต้องมีผู้มีอำนาจก่อน Side Effect สูง
5. **Prohibit** — ปิดความสามารถที่อยู่นอก Policy โดยเด็ดขาด

ระดับควบคุมต้องอิง Impact และ Reversibility ไม่ใช่ความมั่นใจของโมเดล โมเดลที่ตอบอย่างมั่นใจยังไม่มีสิทธิ์ข้าม Approval

Fail-open หรือ Fail-closed

เมื่อ Guardrail เองล้มเหลว ต้องตัดสินใจล่วงหน้าว่าระบบไปต่อหรือหยุด:

- Policy service ล่มก่อนการโอนหรือการลบ → **Fail closed**
- Metrics backend ล่ม → อาจ **Fail open** พร้อม Buffer telemetry
- Cache miss → ไปยัง Source จริง
- Redaction service ล่มก่อนส่งข้อมูลอ่อนไหวเข้า Model → **Fail closed**

ไม่มีค่าเริ่มต้นเดียวที่ถูกสำหรับทุกชั้น สิ่งสำคัญคือบันทึก Decision และเหตุผล เพื่อไม่ให้ Failure ของ Control Plane กลายเป็นช่องโหว่ที่มองไม่เห็น

Idempotency, Retry และ Compensation

Retry เพิ่ม Reliability เฉพาะ Operation ที่ทำซ้ำได้ปลอดภัย หาก Write สำเร็จแต่ Network ขาดก่อนรับ Response การ Retry อาจสร้าง Side Effect ซ้ำ Harness ควรผูก Idempotency key กับ Logical Tool Call เดิมและเก็บ Receipt จากระบบปลายทาง

เมื่อย้อนกลับไม่ได้จริง ต้องออกแบบ Compensation แทน Rollback เช่นสร้างรายการแก้ไขหรือยกเลิกที่อ้างถึง Transaction เดิม Checkpoint หลัง Execution จึงต้องอ่าน External State กลับมา ไม่ใช่เชื่อเพียง HTTP Status

Hook เป็นจุดแทรกของ Control Plane

Hook คือ Callback ที่ Runtime เรียกเมื่อเกิด Event เช่นก่อนใช้ Tool หลัง Tool สำเร็จ ก่อน Compact หรือก่อนหยุด [1] Hook หนึ่งตัวอาจทำได้สามระดับ:

- **Observe** — บันทึก Event โดยไม่เปลี่ยน Flow

- **Transform** — Normalize, redact หรือเติม Context
- **Control** — Allow, deny, retry, pause หรือ stop

Middleware คือการนำ Hook หลายจุดมาประกอบเป็น Policy ที่ใช้ซ้ำ เช่น `HumanApprovalMiddleware`, `PIIRedactionMiddleware` หรือ `RetryWithIdempotencyMiddleware` หนังสือเล่มนี้ใช้คำสองคำนี้เป็น Mental Model ไม่ผูกกับชื่อ API ของ Framework ใด

Harness Design Check

- **Harness รับผิดชอบ:** Enforce Policy, วาง Checkpoint, ตรวจ Permission/Budget และควบคุม Side Effect
- **Model ตัดสินใจ:** เสนอ Action, อธิบายเหตุผล และปรับแผนหลังถูกปฏิเสธ
- **Evidence ที่ต้องเก็บ:** Policy version, Risk tier, Input/Decision, Approval, Receipt และ External verification
- **Failure ที่ต้องทดสอบ:** Guardrail timeout, Bypass, Approval reuse, Duplicate write, Partial success และ Stop โดยงานยังไม่ครบ

บทถัดไปจะเปลี่ยนจากคำถามว่า “จะควบคุมตรงไหน” ไปเป็น “จะเห็นได้อย่างไรว่าในแต่ละ Boundary เกิดอะไรขึ้น” ซึ่งเป็นหน้าที่ของ Observability Engineering

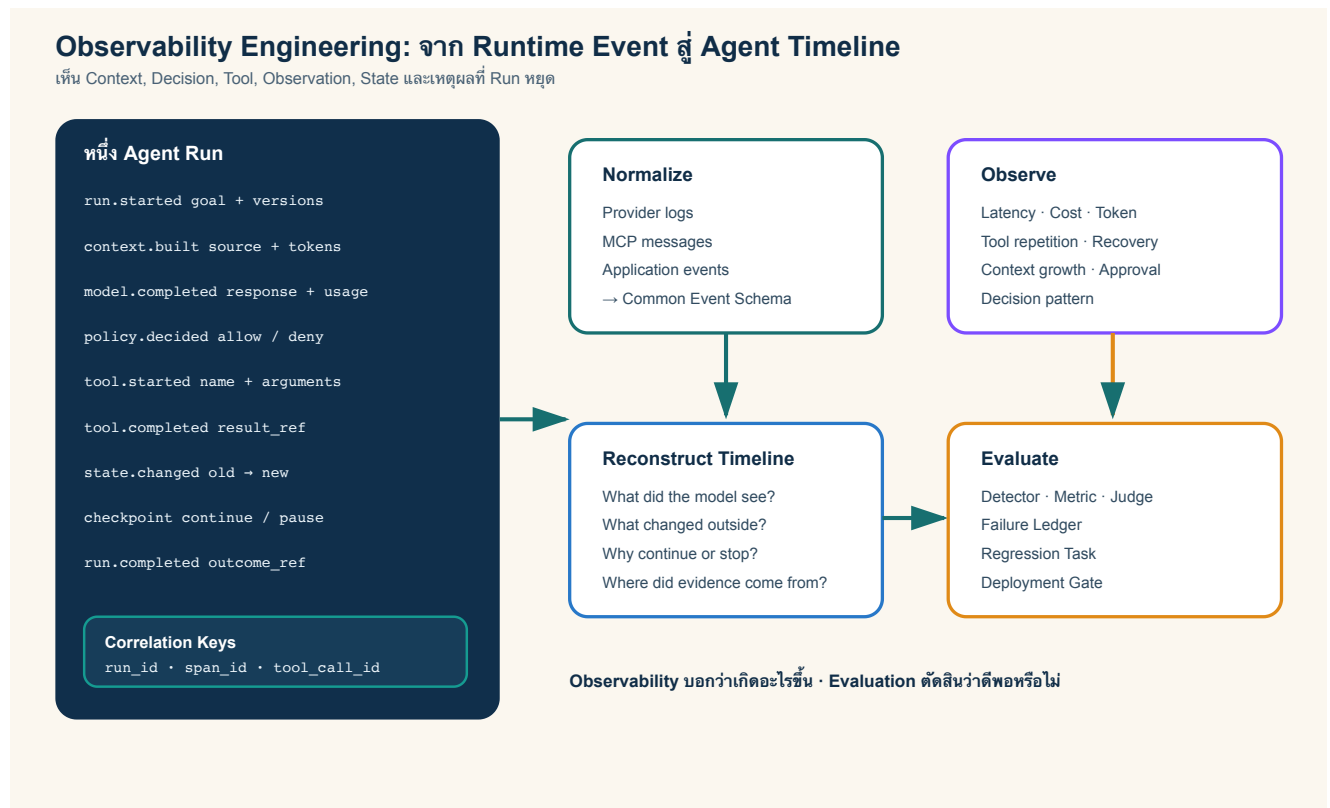
เอกสารอ้างอิง

[1] Anthropic. “Claude Agent SDK: Hooks.” Accessed August 2026.

[2] Anthropic. “Building Effective Agents.” December 2024.

[3] Thongtae, Aekanun. “Recoverable Errors, Agent State, and MCP Tool Boundaries.” *Author’s Lecture Transcript*, August 2026.

บทที่ 13 — Observability Engineering: ทำให้การตัดสินใจของ Agent มองเห็นได้



เส้นทาง Run, Trace, Span และ Event จาก Agent ไปยัง Timeline, Detector และ Dashboard

ระบบ Agent อาจให้ Final Answer เหมือนกันจากเส้นทางที่ต่างกันมาก รอบหนึ่งใช้ Tool เดียวและหลักฐานครบ อีกรอบวนซ้ำหลายครั้งแล้วตอบจากข้อมูลไม่ครบ การเก็บเฉพาะคำตอบสุดท้ายจึงไม่พอ Observability ต้องทำให้เราเห็น Interaction Structure และ Decision Pattern ของทั้ง Run [1]

Observability ต่างจาก Evaluation

Observability ตอบว่า เกิดอะไรขึ้น ส่วน Evaluation ตอบว่า สิ่งที่เกิดขึ้นดีพอหรือไม่

Runtime events → Trace → Normalized facts → Detector/Metric → Evaluation decision

หากไม่มี Trace การประเมินจะเหลือเพียง Final Answer หากไม่มี Evaluation Trace ก็เป็นเพียงข้อมูลจำนวนมากที่ยังไม่บอกว่าระบบผ่านเกณฑ์หรือไม่ สองขั้นนี้ต้องเชื่อมกันแต่ไม่ควรถูกรวมเป็นสิ่งเดียว

Run, Trace, Span และ Event

- Run — หนึ่งภารกิจตั้งแต่รับ Goal จนจบหรือหยุด

- **Trace** — ลำดับหลักฐานทั้งหมดของ Run
- **Span** — Operation ที่มีเวลาเริ่มและจบ เช่น Model Call หรือ Tool Call
- **Event** — เหตุการณ์ ณ จุดหนึ่ง เช่น State transition, Retry หรือ Policy denial
- **Observation** — ข้อมูลจาก Environment ที่กลับเข้า Agent Loop

ทุกชิ้นควรเชื่อมด้วย `run_id`, `span_id`, `parent_span_id` และ `tool_call_id` เพื่อสร้าง Timeline ย้อนกลับได้ หาก Tool Result ไม่มี ID เชื่อมกับค่าขอเดิม เราจะรู้ว่ามีข้อมูลกลับมาแต่ไม่รู้ว่าเป็นผลของ Action ใด

Event Schema กลาง

การเขียน Log เป็นประโยชน์เหมาะกับคน แต่ Parser ต้องเดารูปแบบทุกครั้ง ระบบควรมี Event Schema กลาง ก่อนส่งไปยัง Backend ใด:

```
{
  "event_type": "tool.completed",
  "run_id": "run-184",
  "span_id": "span-9",
  "tool_name": "docs_search",
  "attempt": 2,
  "duration_ms": 418,
  "status": "ok",
  "evidence_ref": "result-88",
  "policy_decision": "allowed"
}
```

Schema ควรรักษาข้อเท็จจริงดิบที่จำเป็นต่อการตรวจ และแยกข้อความสำหรับคนออกจาก Field ที่เครื่องอ่าน ห้ามบันทึก Secret หรือข้อมูลอ่อนไหวแบบดิบเพียงเพราะต้องการ Debug

Timeline ที่อ่าน Agent ได้

Timeline ที่มีประโยชน์ต้องตอบอย่างน้อยหกคำถาม:

1. Model เห็น Context version ใด
2. Model เสนอ Action อะไร
3. Guardrail ตัดสินอย่างไร
4. Environment คืบ Ground Truth อะไร
5. State เปลี่ยนจากอะไรเป็นอะไร
6. เหตุใด Run จึงเดินหน้าหรือหยุด

Standard Metric เช่น Latency, Error rate และ Token usage ยังจำเป็น แต่ Agent ต้องเพิ่ม Metric เชิงโครงสร้าง เช่นจำนวน Tool Call ต่อ Task, Repeat signature, Context growth, Recovery success, Approval frequency และ Evidence coverage

Parser และ Detector ต้องแยกจากกัน

คำบรรยายเรื่อง AI Testing แยก Parser ซึ่งแปลง Trace/Span/Event ให้เป็น Object ออกจาก Detector ซึ่งค้น Failure Pattern [2] การแยกนี้ทำให้เราเปลี่ยน Log source หรือ Provider ได้โดยไม่เขียน Detector ใหม่ทั้งหมด

```
Provider logs ┌
MCP logs ───┴──> Parsers → Normalized Events → Detectors
App events ───┴──
```

Detector แบบ Deterministic อาจตรวจ Tool Call ซ้ำ, Invalid transition, Missing observation หรือ Cost budget เกิน ส่วนพฤติกรรมที่ต้องอาศัยความหมายจึงค่อยส่งให้ Model Judge

Observe โดยไม่ทำให้ระบบหลักล้ม

Telemetry มี Failure Policy ของตัวเอง Logging ที่ล้มไม่ควรทำให้ Read-only Tool ล้มตาม แต่ Security Audit สำหรับ Side Effect สำคัญอาจต้อง Fail closed หากไม่สามารถสร้างหลักฐานได้ ควรส่ง Event ผ่าน Buffer หรือ Queue และกำหนด Backpressure ไม่ให้ Agent Loop รอ Network Call ของ Dashboard ทุกชั้น

Sampling ใช้กับ Event ปริมาณสูงได้ แต่ Decision สำคัญ เช่น Policy denial, Human approval, External write และ Fatal failure ไม่ควรถูกสุ่มทิ้ง Trace ต้องมี Retention และ Redaction Policy แยกตามความเสี่ยงของข้อมูล

จาก Observability สู่ Failure Ledger

Trace ที่พบปัญหาไม่ควรจบที่ Dashboard ให้แปลงเป็น Failure Ledger:

```
Incident → Trace slice → Root cause → Minimal task
→ Expected outcome → Regression test
```

วงจรมีทำให้ Production Observation ไหลกลับไปเสริม Evaluation Suite และทำให้คำว่า “ปรับปรุง” มี Baseline วัด ไม่ใช่แค่ Prompt ตามความรู้สึก

Harness Design Check

- **Harness รับผิดชอบ:** สร้าง Correlation ID, Event Schema, Trace, Redaction, Retention และ Export
- **Model ตัดสินใจ:** ไม่มีสิทธิ์เลือกว่าหลักฐานบังคับใดจะถูกบันทึกหรือไม่
- **Evidence ที่ต้องเก็บ:** Context/Model/Tool version, Action, Observation, State transition, Policy decision, Cost และเวลาที่เกิด

- **Failure ที่ต้องทดสอบ:** Missing span, Broken correlation, PII leakage, Duplicate event, Telemetry outage และ Trace ที่ Reconstruct Run ไม่ได้

บทสุดท้ายจะนำ Control, State, Evidence, Observability และ Evaluation มาประกอบเป็น Production Harness เดี่ยวซึ่งหยุดและกลับมาทำงานต่อได้

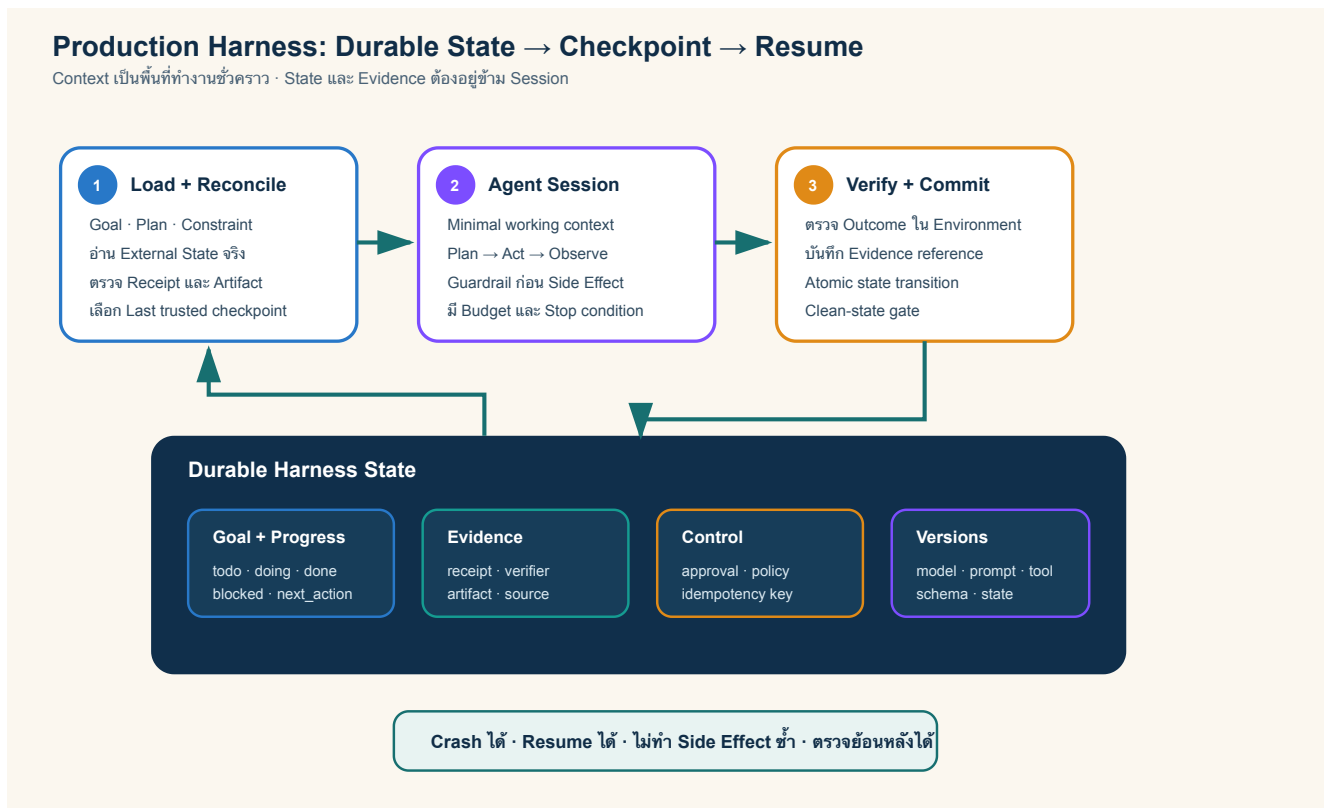
เอกสารอ้างอิง

[1] Anthropic. “How We Built Our Multi-Agent Research System.” June 2025.

[2] Thongtae, Aekanun. “Trace Parsing, Failure Detection, and Evaluation Coverage.” *Author’s Lecture Transcript*, July 2026.

[3] Thongtae, Aekanun. *2026-AI-Testing-and-Evaluation*, commit [9eb207dc](#), GitHub repository, 2026.

บทที่ 14 — Production Harness: ประกอบ Agent ให้ทำงานระยะยาวและกลับมาทำต่อได้



Production Harness รวม Goal, State, Agent Session, Checkpoint, Evidence และ Resume

ภาพเปิดบทแยก Durable State ออกจาก Context ของ Agent Session แต่ละ Session มีหน้าตาทำงานจำกัด เมื่อใกล้เต็มหรือเกิด Failure ระบบสร้าง Checkpoint แล้ว Session ใหม่อ่าน Goal, Progress, Evidence และรายการค้างเพื่อทำงานต่อ ไม่จำเป็นต้องแบก Transcript ทั้งหมดกลับมา

Long-running Agent ไม่ใช่ Loop ที่ยาวขึ้น

การเพิ่ม `max_steps` ทำให้ Agent อยู่ได้นานขึ้น แต่ไม่ได้ทำให้ Recover ได้ งานระยะยาวต้องออกแบบให้ Context สูญหาย Process restart หรือ Version เปลี่ยนแล้วภารกิจยังมี State ที่เชื่อถือได้ Anthropic เสนอ Harness ที่แยก Initializer Session ออกจาก Coding Session และใช้ Progress file ร่วมกับ Git history เพื่อให้ Session ใหม่เข้าใจงานได้รวดเร็ว [1]

หลักสำคัญคือ **Context เป็น Cache ของการทำงาน ส่วน State ต้อง Durable** สิ่งที่ทำสูญหาย เช่น Goal, Constraint, Approval, External receipt และ Completion criteria ไม่ควรฝากไว้ใน Conversation Summary เพียงแห่งเดียว

Durable State Model

Production Harness ควรมี State อย่างน้อย:

```
{
  "goal": "ผลลัพธ์ที่ต้องส่งมอบ",
  "status": "in_progress",
  "plan": ["..."],
  "completed": ["..."],
  "blocked": [],
  "evidence_refs": ["..."],
  "approvals": ["..."],
  "artifacts": ["..."],
  "next_action": "...",
  "state_version": 18
}
```

State update ต้องเป็น Atomic หรือมี Version check เพื่อป้องกัน Session สองตัวเขียนทับกัน หากมีหลาย Worker แต่ละตัวควรคืนผลแบบ Artifact + Evidence + Uncertainty ไม่ใช่เพียง Summary ที่ผ่านการถ่ายทอดหลายทอด

Checkpoint และ Resume Protocol

Checkpoint ที่ดีไม่ใช้การ Dump Memory ทั้งหมด แต่เป็น Handoff Package ซึ่งตอบว่า:

- Goal และ Definition of Done คืออะไร
- งานใดเสร็จแล้วพร้อมหลักฐาน
- External State เปลี่ยนอะไรไปแล้ว
- งานใดยังค้างหรือ Blocked
- Decision สำคัญและ Constraint ใดห้ามหาย
- Session ใหม่ควรตรวจอะไรเป็นอันดับแรก

Resume ต้องเริ่มด้วย Reconciliation: อ่าน Durable State แล้วตรวจ Environment จริง เช่น Artifact ยังอยู่ Test ยังผ่าน หรือ Transaction ยังมีสถานะเดิม ก่อนเชื่อ State ที่เขียนไว้ วิธีนี้ป้องกัน Progress Note เก่า กลายเป็นความจริงเทียม

Verifiable Work Units

งานใหญ่ควรถูกแตกเป็นหน่วยที่มี Outcome ตรวจได้ Anthropic ยกตัวอย่าง Feature list ซึ่งทุกข้อเริ่มจากสถานะ Failing แล้ว Agent ค่อยเปลี่ยนเป็น Passing หลังตรวจจริง [1] หลักนี้เข้ากับ Todo State ในบท 7 แต่เพิ่มเงื่อนไขสำคัญ: **done** ต้องมี Verifier และ Evidence reference

```
Work unit = Task + Preconditions + Action boundary
            + Verifier + Evidence + Completion state
```

หน่วยงานที่เล็กเกินไปสร้าง Overhead มาก หน่วยที่ใหญ่เกินไปทำให้ Resume ไม่รู้ว่าจะข้างในสำเร็จถึงไหน
ขอบเขตที่ดีคือจุดซึ่งตรวจ Outcome ได้อิสระและ Commit ได้ใน Clean State

Clean State Discipline

ก่อน Session จบ ระบบควรตรวจว่า Artifact อยู่ในสภาพที่ผู้พัฒนาหรือ Session ถัดไปเริ่มต่อได้ ไม่ทิ้งไฟล์ชั่วคราว State ครึ่งเดียว หรือ Test ที่เสียโดยไม่บอก หากยัง Clean ไม่ได้ต้องบันทึก Blocker และ Recovery step อย่างตรงไปตรงมา ไม่ประกาศสำเร็จเพื่อให้ Session ปิดสวย

Clean State เป็น Checkpoint เชิงคุณภาพ ไม่จำเป็นต้องหมายถึงภารกิจทั้งหมดเสร็จ แต่หมายถึงความคืบหน้าปัจจุบันไม่สร้างหนี้ลับให้รอบต่อไป

Recovery หลัง Failure

Recovery เริ่มจากจำแนก:

1. **Transient** — Retry ได้ภายใน Budget
2. **Recoverable** — ต้องแก้ Arguments หรือเปลี่ยน Strategy
3. **Blocked** — ต้องการข้อมูลหรือ Approval ภายนอก
4. **Fatal** — Invariant เสียหรือ State เชื้อถือไม่ได้ ต้องหยุด

เมื่อ Process กลับมา Harness ต้องรู้ Last confirmed checkpoint ไม่ใช่เพียง Last log line Operation ที่มี Side Effect ต้องตรวจ Receipt ก่อน Retry และใช้ Idempotency key เดิมตลอด Logical Action

Version-aware Execution

Agent Run อาจยาวกว่ารอบ Deploy หากเปลี่ยน Model, Prompt, Tool Schema หรือ Policy กลาง Run ผลลัพธ์อาจ Reproduce ไม่ได้ Trace จึงต้องบันทึกทุก Version ที่มีผลต่อ Decision และ Resume Policy ต้องกำหนดว่าจะ:

- ทำ Run เดิมต่อด้วย Version เดิม
- Migrate State ไป Version ใหม่
- หยุด Run เก่าอย่างปลอดภัยแล้วเริ่มใหม่

สำหรับระบบที่มี Run ยาว การค่อยย้าย Traffic โดยรักษา Version เก่าไว้จน Run เดิมจบช่วยไม่ตัดงานกลางทาง [2]

Reference Production Loop

```
Load durable state
→ Reconcile environment
→ Build minimal working context
→ Model proposes next action
→ Guardrail and checkpoint
→ Execute with idempotency
→ Observe and record evidence
→ Update state atomically
→ Evaluate completion
→ Continue, pause, resume or close
```

ลูปนี้แสดงว่าทั้งลิบคำบนปกไม่ได้เป็น Component น้ำหนักเท่ากันทั้งหมด บางคำเป็นกลไก เช่น Context, Tools, State และ Memory ส่วน Control, Evidence, Checkpoints, Observability และ Guardrails เป็นคุณสมบัติข้ามระบบที่ครอบคลุมกลไกเหล่านั้น

Harness Design Check

- **Harness รับผิดชอบ:** Durable state, Checkpoint/Resume, Reconciliation, Versioning, Idempotency และ Clean-state gate
- **Model ตัดสินใจ:** เลือก Next action ภายใน Work unit และอธิบาย Blocker/Uncertainty
- **Evidence ที่ต้องเก็บ:** State version, Artifact, Verifier result, Receipt, Approval, Runtime version และ Resume decision
- **Failure ที่ต้องทดสอบ:** Crash กลาง Write, Stale checkpoint, Concurrent update, Version migration, Lost approval และ False completion

AI Harness Engineering จึงไม่ใช่การทำให้ LLM ทำทุกอย่างเอง แต่เป็นการสร้างระบบซึ่งรู้ว่าอะไรควรให้โมเดลตัดสินใจ อะไรต้องบังคับด้วยโค้ด หลักฐานใดทำให้งานเดินหน้าได้ และเมื่อเกิดความผิดพลาดจะหยุดหรือกลับมาทำต่อจากจุดที่เชื่อถือได้อย่างไร

เอกสารอ้างอิง

[1] Anthropic. “Effective Harnesses for Long-Running Agents.” November 2025.

[2] Anthropic. “How We Built Our Multi-Agent Research System.” June 2025.

[3] Thongtae, Aekanun. “Planning, Memory, Error Recovery, and Agent Harness.” *Author’s Lecture Transcript*, July–August 2026.